



**POLSKO-JAPOŃSKA AKADEMIA
TECHNIK KOMPUTEROWYCH**

Wydział Informatyki

Katedra Systemów Inteligentnych i Data Science

Data Science

Michał Chojna

Nr albumu: s29758

**Analiza efektywności różnych architektur
wieloagentowych w analizie dokumentów
ubezpieczeniowych**

Praca magisterska

dr hab. Andrzej Wodecki, prof. PJATK

Warszawa, czerwiec 2026

Streszczenie

W pracy zbadano wpływ topologii systemów agentowych na jakość odpowiedzi na pytania wymagające analizy dokumentów ubezpieczeniowych – Ogólnych Warunków Ubezpieczenia (OWU) oraz dokumentów zawierających informacje o produkcie ubezpieczeniowym (IPID). Środowisko testowe wybrano ze względu na jego złożoność: specjalistyczny język prawniczo-ubezpieczeniowy łączy się z wielopoziomową strukturą warunków i wyłączeń rozproszonych w wielu paragrafach. Wymagało to od badanych architektur skutecznego wykorzystania mechanizmów wyszukiwania i wnioskowania.

Porównano sześć architektur opartych na dużych modelach językowych (*ang. Large Language Models*, LLM): deterministyczną, planer–wykonawca, router–specjalista, tablicową, hierarchiczną oraz ReAct. Bazę wiedzy systemu stanowiły 22 polskie dokumenty ubezpieczeniowe pochodzące od trzech głównych towarzystw ubezpieczeniowych. Opracowano autorski zestaw 90 pytań w czterech poziomach trudności: bardzo łatwym, łatwym, trudnym i bardzo trudnym – każde pytanie wymagało odniesienia się do treści zgromadzonych dokumentów. Łącznie przeprowadzono 540 uruchomień, które oceniono według wspólnego protokołu ewaluacyjnego uwzględniającego wierność, poprawność, zwięzłość, trafność pobieranych dokumentów, opóźnienie oraz zużycie tokenów.

Wyniki wskazują, że większa złożoność orkiestracji nie gwarantuje wyższej jakości odpowiedzi. Architektura router–specjalista osiąga najwyższą średnią wierność (0,811), a deterministyczna – najwyższą średnią poprawność (0,716), obie przy koszcie porównywalnym z najprostszymi układami. Architektury hierarchiczna i ReAct zużywają kilkakrotnie więcej tokenów bez proporcjonalnego przyrostu jakości – analiza jakościowa wskazuje, że dłuższe ścieżki przetwarzania prowadzą do utraty kontroli nad kontekstem i propagacji błędów między modułami. Sformułowano wnioski praktyczne dotyczące doboru architektury do przypadku użycia, opłacalności złożoności orkiestracji oraz roli przygotowania danych jako czynnika warunkującego skuteczność systemu niezależnie od topologii.

Słowa kluczowe: architektury agentowe, duże modele językowe, ewaluacja systemów, generowanie wspomagane wyszukiwaniem, systemy wieloagentowe

[Tekst dedykacji]

Podziękowania

[Tekst podziękowań]

Spis treści

1	Wprowadzenie	1
1.1	Motywacja i problem badawczy	1
1.2	Pytania badawcze	3
1.3	Cele badawcze	4
1.4	Zakres i ograniczenia	4
1.5	Struktura pracy	5
2	Podstawy teoretyczne	7
2.1	Systemy agentowe	7
2.1.1	Pojęcie agenta	7
2.1.2	Klasyfikacja architektur agentowych	8
2.1.3	Architektury jednoagentowe	9
2.1.4	Architektury wieloagentowe	10
2.1.5	Interfejsy narzędziowe agentów	13
2.1.6	Protokoły komunikacji agentów	13
2.1.7	Systemy pamięci agentów	14
2.1.8	Ewaluacja agentów	15
2.1.9	Przegląd istniejących środowisk agentowych	18
2.2	Generowanie wspomagane wyszukiwaniem	20
2.2.1	Pojęcie RAG	20
2.2.2	Problem halucynacji w modelach językowych	20
2.2.3	Problem okna kontekstowego w modelach językowych	22
2.2.4	Architektury RAG	22
2.2.5	Metody wyszukiwania dokumentów	24
2.2.6	Metody fragmentacji dokumentów	25
2.2.7	Ewaluacja RAG	26
2.2.8	Agentowy RAG	29
2.3	Środowisko dokumentów ubezpieczeniowych	30
2.3.1	Przetwarzanie języka naturalnego dla dokumentów ubezpieczeniowych	30
2.3.2	Ogólne Warunki Ubezpieczenia	33
2.3.3	Informacyjny Dokument Produktu Ubezpieczeniowego	33
2.3.4	Klasyfikacja pytań	35
2.4	Przegląd prac pokrewnych	36
2.4.1	Istniejące rozwiązania	36
2.4.2	Luki badawcze i wkład niniejszej pracy	38

3	Metodologia	41
3.1	Metoda badawcza	41
3.2	Projekt eksperymentu	42
3.2.1	Zmienna niezależna	42
3.2.2	Zmienne zależne	44
3.2.3	Warunki kontrolowane	44
3.2.4	Plan porównawczy	44
3.3	Materiał badawczy	45
3.4	Kryteria oceny	47
3.4.1	Miary ilościowe	47
3.4.2	Miary jakościowe	49
3.4.3	Protokół ewaluacji	49
3.5	Procedura badawcza	50
3.5.1	Etapy eksperymentu	50
3.5.2	Analiza ilościowa	50
3.5.3	Analiza jakościowa	51
3.5.4	Perspektywa produkcyjna	51
3.6	Odtwarzalność	51
4	Implementacja	52
4.1	Stos technologiczny	52
4.2	Architektura modułowa systemu	54
4.3	Elementy architektury współdzielone	55
4.3.1	Stan grafu	56
4.3.2	Konfiguracja uruchomieniowa	57
4.3.3	Schematy wyjść strukturalnych	57
4.3.4	Rejestr narzędzi i fabryka grafów	58
4.4	Implementacja promptów	58
4.4.1	Prompt ekstrakcyjny	58
4.4.2	Prompty strukturalne	59
4.4.3	Prompty odpowiedziowe	59
4.4.4	Prompty ewaluacyjne	59
4.5	Implementacja narzędzi	60
4.5.1	Parser pytania	61
4.5.2	Przepisywanie zapytania	61
4.5.3	Wyszukiwanie	61
4.5.4	Ponowne szeregowanie	61
4.5.5	Selektor dowodów	62
4.5.6	Generator cytowań	62

4.5.7	Selektor promptu	62
4.5.8	Synteza odpowiedzi	62
4.6	Implementacja architektur agentowych	63
4.6.1	Architektura deterministyczna	63
4.6.2	Architektura planer–wykonawca	64
4.6.3	Architektura router–specjalista	64
4.6.4	Architektura tablicowa	65
4.6.5	Architektura hierarchiczna	65
4.6.6	Architektura ReAct	65
4.7	Protokół przetwarzania dokumentów	66
4.7.1	Ekstrakcja	67
4.7.2	Fragmentacja	67
4.7.3	Indeksowanie	67
4.8	Infrastruktura ewaluacyjna	68
4.8.1	Uruchamianie architektur	68
4.8.2	Ocena odpowiedzi	68
4.8.3	Śledzenie przebiegów	68
4.9	Wybrane zagadnienia inżynierskie	69
4.9.1	Zarządzanie rozmiarem okna kontekstowego	69
4.9.2	Agregacja metadanych wykonania	69
4.9.3	Obsługa niepoprawnych wyjść modeli	69
4.9.4	Strategia wielojęzycznych promptów	69
4.9.5	Stabilność identyfikatorów fragmentów	70
4.9.6	Izolacja warstwy ewaluacyjnej	70
5	Eksperyment i wyniki	71
5.1	Przebieg wykonania	71
5.1.1	Czas trwania	71
5.1.2	Kompletność i błędy wykonania	72
5.1.3	Artefakty wynikowe	72
5.2	Wyniki operacyjne	73
5.2.1	Zużycie tokenów i koszt	73
5.2.2	Wywołania narzędzi i iteracje	74
5.2.3	Fragmenty z wyszukiwania i cytowania	74
5.3	Wyniki jakościowe	75
5.3.1	Wierność	75
5.3.2	Poprawność	77
5.3.3	Zwiężłość	78
5.3.4	Trafność dokumentów	80

5.4	Parametry konfiguracyjne eksperymentu	81
5.4.1	Parametry modeli językowych	81
5.4.2	Parametry wyszukiwania	82
5.4.3	Parametry fragmentacji	83
5.4.4	Parametry architektury	83
5.4.5	Parametry wykonania	84
6	Wnioski i zakończenie	85
6.1	Wnioski ilościowe	85
6.1.1	Wierność	85
6.1.2	Poprawność	86
6.1.3	Zwiężłość	87
6.1.4	Trafność dokumentów	87
6.1.5	Koszty operacyjne	88
6.2	Wnioski jakościowe	89
6.2.1	Charakterystyka niepowodzeń	90
6.2.2	Systematyczne pomijanie klauzul wyłączeń	90
6.2.3	Niedobór kontekstu definicyjnego	91
6.2.4	Bezwarunkowa dekompozycja w architekturze hierarchicznej	91
6.2.5	Rozszerzanie zakresu wyszukiwania w architekturze ReAct	91
6.2.6	Niespójność autorytetu między OWU a IPID	92
6.2.7	Samokorekta w architekturze tablicowej	92
6.3	Rozstrzygnięcie problemu badawczego i realizacja celów	92
6.3.1	Rozstrzygnięcie głównego problemu badawczego	92
6.3.2	Wpływ topologii na jakość odpowiedzi	93
6.3.3	Kompromis między jakością a kosztem	93
6.3.4	Wpływ planowania, trasowania i dekompozycji	93
6.3.5	Wzorce niepowodzeń	94
6.3.6	Odpowiedzi na pytania badawcze	94
6.3.7	Realizacja celów badawczych	95
6.4	Kierunki dalszych badań	95
6.5	Rekomendacje dla praktyków	96
6.6	Zakończenie	97
	Wykaz tabel	100
	Wykaz rysunków	101
	Wykaz wykresów	102
	Bibliografia	103

A	Prompty systemu	108
B	Diagramy architektur agentowych	118

1. Wprowadzenie

W ostatnich latach badania w dziedzinie sztucznej inteligencji przesunęły się z analizy pojedynczych modeli na złożone systemy. Duże modele językowe (ang. *Large Language Models*, LLM) oparte na architekturze Transformer osiągają wysoką skuteczność w generowaniu tekstu na podstawie ogólnej wiedzy [1], jednakże w złożonych zadaniach wymagających wieloetapowego wnioskowania i wiedzy dziedzinowej generacja tekstu jest niewystarczająca [2]. Rozwiązanie takich zadań wymaga wyszukiwania i integracji informacji oraz formułowania wniosków na podstawie pobranych źródeł. Zmiana paradygmatu ukształtowała dziedzinę architektur agentowych, wykorzystywania zewnętrznych narzędzi [3], generowania wspomaganego wyszukiwaniem (ang. *Retrieval-Augmented Generation*, RAG) [4] oraz protokołów orkiestracji [5]. W literaturze brakuje jednak kontrolowanych porównań wskazujących, kiedy dana architektura orkiestracji poprawia jakość wyników, a kiedy generuje jedynie dodatkowy narzut obliczeniowy i koszty operacyjne.

1.1. Motywacja i problem badawczy

Projektowanie systemów agentowych opartych na dużych modelach językowych do odpowiadania na pytania na podstawie obszernych zbiorów dokumentów pozostaje otwartym problemem badawczym. Modele wykazują wyższą skuteczność po wyposażeniu w odpowiednie narzędzia i ustrukturyzowany schemat wnioskowania [5].

W architekturze ReAct zaobserwowano, że przeplatanie etapów wnioskowania (ang. *reasoning traces*) z wywołaniami narzędzi (ang. *tool calls*) zmniejsza liczbę halucynacji i zwiększa skuteczność w porównaniu do podejścia łańcucha myśli (ang. *chain-of-thought*) [6]. Systemy wieloagentowe, takie jak AutoGen, udowodniły skuteczność podziału zadań [7], natomiast system MetaGPT potwierdził wpływ specjalizacji ról na wyniki w złożonych zadaniach [8].

Architektury te rzadko podlegają jednak bezpośrednim porównaniom. Nowe rozwiązania zazwyczaj zestawia się z modelem bazowym – wyposażonym w prostą instrukcję systemową (ang. *system prompt*) lub w architekturze jednorzeczowego systemu RAG (ang. *single-pass RAG*). Podejście to uniemożliwia obiektywną ocenę struktur agentowych w identycznych warunkach – przy wykorzystaniu tego samego modelu bazowego, zestawu narzędzi, korpusu danych i metryk ewaluacyjnych.

Brak ujednoliconych testów utrudnia syntezę wiedzy naukowej oraz projektowanie systemów produkcyjnych. Według Wang i in. porównawcza ewaluacja topologii agentów w stałych warunkach pozostaje problemem otwartym [5]. Xi i in. wskazują, że niespójność zadań, modeli bazowych i metryk uniemożliwia porównywanie wyników pomiędzy publikacjami [9].

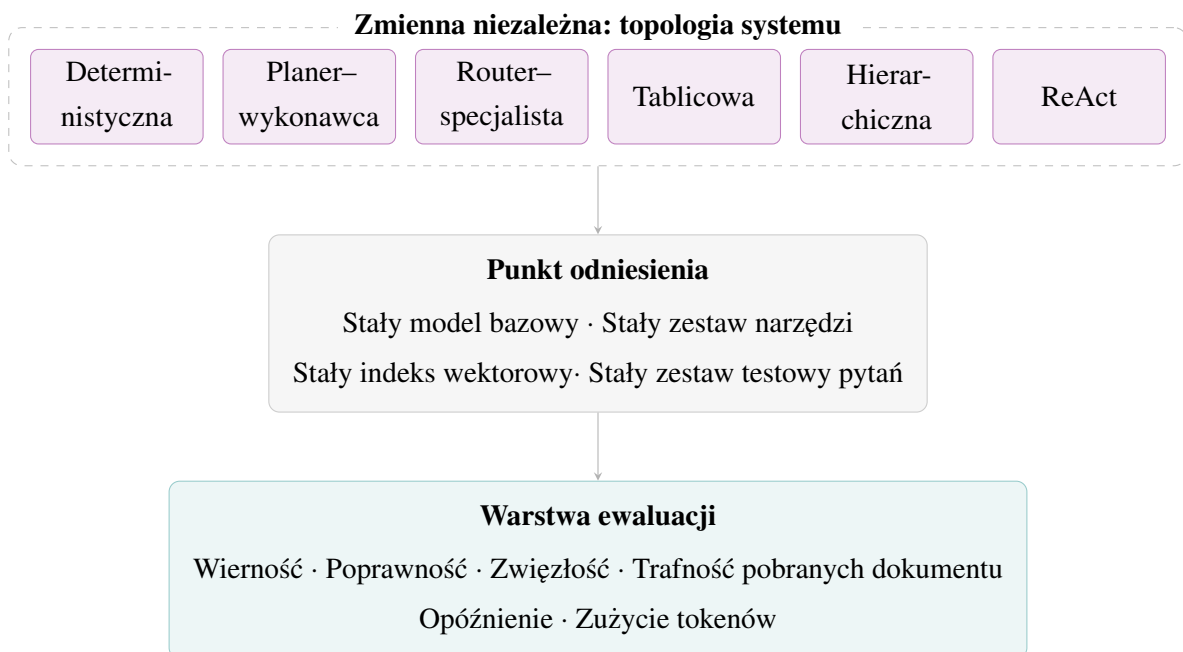
Niniejsza praca podejmuje problem braku kontrolowanego porównania izolującego wpływ struktury agenta na poprawność odpowiedzi oraz koszty operacyjne podczas analizy ustruk-

turyzowanych dokumentów regulacyjnych. Badaniu poddano zależność między złożonością systemów wieloagentowych a jakością uzyskiwanych odpowiedzi w kontrolowanym środowisku testowym.

Jako środowisko testowe wybrano polską dokumentację ubezpieczeniową. Ogólne Warunki Ubezpieczenia (OWU) oraz dokumenty zawierające informacje o produkcie ubezpieczeniowym (IPID) stanowią trudną klasę tekstów dla systemów pytań i odpowiedzi (ang. *Question Answering*, QA). Dokumenty te charakteryzują się specjalistycznym językiem prawniczo-ubezpieczeniowym, zawierają rozbudowane struktury warunków i wyłączeń, a kluczowe informacje są w nich rozproszone. Wymusza to stosowanie mechanizmów wyszukiwania wieloskokowego i wnioskowania warunkowego, co pozwala na miarodajne zróżnicowanie badanych architektur agentowych.

Rysunek 1.1 przedstawia główne założenie eksperymentu: jedyną zmienną niezależną jest topologia agenta, podczas gdy model, narzędzia, korpus i warstwa ewaluacyjna pozostają wspólne dla wszystkich wariantów.

Rysunek 1.1 Projekt eksperymentu



Źródło: opracowanie własne.

1.2. Pytania badawcze

Pytania badawcze wynikają bezpośrednio z luki opisanej w sekcji 1.1 i odpowiadają czterem obszarom porównania: jakości odpowiedzi, stosunkowi kosztu do jakości, wpływowi strategii rozwiązywania pytań trudnych oraz charakterystyce błędów.

Pytanie badawcze 1 (PB1): W jakim stopniu poszczególne architektury różnią się pod względem wierności źródeł? Weryfikacji poddano poprawność merytoryczną i trafność pobieranych dokumentów w kontrolowanym środowisku ze stałym modelem, zestawem narzędzi, korpusem i metrykami.

Pytanie badawcze 2 (PB2): Która topologia agentowa wykazuje optymalny stosunek jakości do kosztów operacyjnych? Koszty operacyjne wyrażono poprzez opóźnienie oraz zużycie tokenów. Analizie poddano również zmiany tego stosunku w zależności od stopnia trudności zadanego pytania.

Pytanie badawcze 3 (PB3): Jak mechanizmy planowania, trasowania (ang. *routing*) i podziału zadań wpływają na rozwiązywanie złożonych problemów, w tym na zestawianie warunków ochrony z wyjątkami w dokumentach? Skuteczność mechanizmów porównano z układami realizującymi wnioskowanie w pojedynczej ścieżce.

Pytanie badawcze 4 (PB4): Jakie rodzaje błędów wykazują poszczególne struktury podczas analizy tekstów ustrukturyzowanych? Określono potencjalne źródła tych błędów, w tym ograniczenia okna kontekstowego, zakłócenia komunikacji między modułami oraz kaskadową propagację błędów.

Tabela 1.1 zestawia każde pytanie badawcze z odpowiadającą mu hipotezą oraz wskazuje sekcje, w których zgromadzono dane niezbędne do jej weryfikacji.

Tabela 1.1 Zestawienie pytań badawczych, hipotez i źródeł danych

Pytanie	Hipoteza	Źródło dowodów
PB1	Bardziej złożone topologie zapewniają wyższą wierność odpowiedzi w porównaniu do prostych układów jednoagentowych	Oceny jakościowe i ilościowe, przekroje według trudności pytań — sekcja 5.3
PB2	Wzrost kosztów operacyjnych w systemach wieloagentowych nie przekłada się proporcjonalnie na przyrost jakości odpowiedzi	Analiza zużycia tokenów i opóźnień, oceny jakościowe — sekcja 5.2, 5.3
PB3	Dekompozycja zadań w różnym stopniu poprawia skuteczność poszczególnych architektur w pytaniach wieloskokowych	Przekroje według trudności pytań, analiza jakościowa przebiegów — sekcja 5.3
PB4	Różne architektury agentowe wykazują odmienne charakterystyki błędów przy identycznych zadaniach	Analiza jakościowa przebiegów, klasyfikacja niepowodzeń — sekcja 6.2.1

Źródło: opracowanie własne.

1.3. Cele badawcze

Odpowiedź na postawione pytania badawcze wymagała zrealizowania czterech szczególnych celów.

Cel badawczy 1 (CB1): Opracowano środowisko orkiestracji oparte na frameworku LangGraph i platformie ewaluacyjnej Arize Phoenix, w którym zaimplementowano sześć architektur agentowych. Umożliwiło to izolację topologii systemu jako jedynej zmiennej niezależnej w badaniu.

Cel badawczy 2 (CB2): Skonstruowano zestaw testowy dla branży ubezpieczeniowej, składający się z 90 pytań opartych na dokumentach OWU i IPID, odzwierciedlających rzeczywiste zapytania użytkowników.

Cel badawczy 3 (CB3): Przeprowadzono ewaluację wszystkich wariantów przy użyciu jednolitych metryk jakościowych (wierność, poprawność) i operacyjnych.

Cel badawczy 4 (CB4): Przeanalizowano uzyskane dane w celu identyfikacji cech charakterystycznych badanych struktur oraz sformułowania rekomendacji dla projektantów systemów produkcyjnych.

1.4. Zakres i ograniczenia

Eksperyment przeprowadzono w zamkniętym środowisku testowym. Bazę wiedzy stanowiły 22 polskojęzyczne dokumenty ubezpieczeniowe, podzielone na 5108 fragmentów tekstowych

(ang. *chunks*). Architektury generowały odpowiedzi wyłącznie na podstawie tej bazy, bez dostępu do internetu i zewnętrznych źródeł, co zagwarantowało izolację topologii agenta jako zmiennej niezależnej. Procedura testowa objęła zestaw 90 pytań oraz sześć architektur agentowych. Wszystkie systemy korzystały z tego samego modelu językowego, identycznych narzędzi oraz identycznej bazy wektorowej, a instrukcje systemowe zdefiniowano w języku angielskim.

Wyniki eksperymentu poddano analizie z trzech perspektyw. Pierwsza obejmuje pomiary ilościowe (wierność, poprawność, zwięzłość, trafność pobieranych dokumentów oraz koszty operacyjne) dla wszystkich architektur. Druga perspektywa dotyczy analizy jakościowej przebiegów: sposobu konstrukcji odpowiedzi, utraty spójności kontekstu oraz charakterystycznych wzorców błędów. Trzecia perspektywa obejmuje rekomendacje praktyczne dotyczące doboru architektury, progu opłacalności orkiestracji oraz roli przygotowania danych w zapewnieniu skuteczności systemu.

W badaniu oddzielono celowe decyzje projektowe od obiektywnych ograniczeń. Wybór jednego modelu bazowego i stałego zestawu narzędzi stanowił warunek konieczny do izolacji wpływu struktury agentowej. Z tego powodu zrezygnowano z dostrajania modeli (ang. *fine-tuning*), obsługi wielu języków, indeksowania w czasie rzeczywistym oraz testów z udziałem użytkowników końcowych.

Badanie ma trzy główne ograniczenia. Po pierwsze, zestaw 90 pytań nie wyczerpuje pełnego zakresu zapytań w środowisku produkcyjnym. Po drugie, każdą architekturę uruchomiono jednokrotnie dla każdego pytania. Wszystkie wywołania (540 przebiegów agentowych i ewaluacja metodą *LLM-as-a-judge*) zrealizowano przez płatne API OpenAI, co przy ograniczonym budżecie wykluczyło powtórzenia. Użycie modeli open-source lokalnie odrzucono ze względu na wymagania sprzętowe (infrastruktura GPU) dla dużych modeli oraz niewystarczającą jakość analizy polskich tekstów prawniczych przez modele mniejsze. Ustawienie temperatury modelu na zero oraz powtarzalny potok wyszukiwania miały na celu minimalizację wpływu losowości. Trzecim ograniczeniem jest automatyczna ewaluacja z użyciem modelu jako sędziego (ang. *LLM-as-a-judge*), co wprowadza ryzyko błędów poznawczych modelu i nie zastępuje weryfikacji eksperckiej. Wyników nie należy uogólniać na inne dziedziny bez dodatkowych badań.

1.5. Struktura pracy

Układ rozdziałów odpowiada kolejnym fazom badań.

Rozdział 2 – Podstawy teoretyczne: Omawia kluczowe pojęcia z zakresu architektur agentowych i systemów generowania wspomaganego wyszukiwaniem oraz charakteryzuje polską dokumentację ubezpieczeniową jako dziedzinę badawczą.

Rozdział 3 – Metodologia: Przedstawia założenia eksperymentu, sposób budowy korpusu i zestawu testowego, przyjęte kryteria i metryki oceny oraz warunki zapewniające porównywalność wyników między architekturami.

Rozdział 4 – Implementacja: Opisuje sposób realizacji sześciu architektur agentowych, zastosowane narzędzia, budowę bazy wektorowej oraz protokół ewaluacji przyjęty na potrzeby eksperymentu.

Rozdział 5 – Eksperyment i wyniki: Prezentuje wyniki 540 uruchomień z podziałem na poziomy trudności pytań wraz z analizą kosztów operacyjnych i zestawieniem zaobserwowanych zachowań architektur.

Rozdział 6 – Wnioski i zakończenie: Odpowiada na postawione pytania badawcze, formułuje rekomendacje dla praktyków projektujących systemy agentowe oraz wskazuje kierunki dalszych badań.

2. Podstawy teoretyczne

W rozdziale omówiono trzy obszary wiedzy stanowiące podstawę teoretyczną dla dalszej części pracy: ewolucję systemów agentowych, mechanizmy generowania wspomaganego wyszukiwaniem oraz specyfikę dokumentów prawnych ze szczególnym uwzględnieniem polskich dokumentów ubezpieczeniowych jako środowiska ewaluacyjnego.

2.1. Systemy agentowe

Systemy oparte na dużych modelach językowych przeszły ewolucję od potoków przetwarzających proste zapytania do autonomicznych systemów realizujących złożone cele. Kolejne podrozdziały porządkują tę dziedzinę – punktem wyjścia jest definicja agenta, po której następuje przegląd wariantów architektonicznych, od układów pojedynczych po złożone topologie rozproszone.

2.1.1 Pojęcie agenta

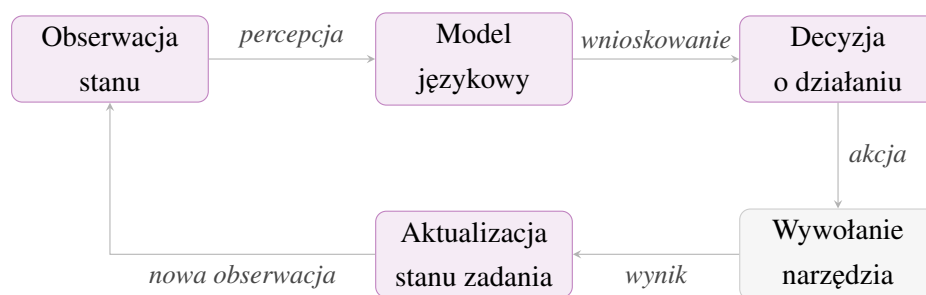
Pojęcie agenta pochodzi z wczesnych badań nad sztuczną inteligencją i wyprzedza powstanie dużych modeli językowych. Wooldridge i Jennings określili go jako byt obliczeniowy osadzony w środowisku, który potrafi realizować założone cele bez nadzoru. Definicja ta zakłada cztery główne cechy: autonomię, zdolność społeczną, reaktywność i proaktywność [10]. System operuje bez ciągłej ingerencji operatora, komunikuje się z innymi podmiotami, reaguje na zmiany otoczenia i inicjuje działania celowe. Kryteria te, sformułowane pierwotnie z myślą o systemach robotycznych i symulacjach wieloagentowych, pozostają aktualne w kontekście współczesnych agentów językowych – zmianie podlega tylko środowisko, które nie jest już przestrzenią fizyczną, lecz cyfrową: interfejsy programistyczne, repozytoria dokumentów, bazy wiedzy.

Wang i in. wyodrębnili cztery współpracujące elementy agenta: profil, pamięć, planowanie oraz działanie [5]. Profil nadaje systemowi konkretną rolę operacyjną i ustala cele nadrzędne. Pamięć zachowuje zebrane informacje między kolejnymi krokami. Planowanie dzieli złożony problem na wykonalne etapy. Moduł działania zamienia wynik rozumowania na operacje techniczne, takie jak odpytanie bazy danych lub wywołanie zewnętrznego skryptu. Podział ten wymusza zaprojektowanie niezależnych komponentów – różnice między poszczególnymi architekturami agentowymi wynikają z decyzji podjętych w tych czterech obszarach.

Integracja modelu z narzędziami nie gwarantuje jego pełnej autonomii, co podkreślają Xi i in. [9]. System wyposażony w dostęp do sieci może działać wyłącznie jako reaktywny generator tekstu. Warunkiem autonomii jest umiejętność podtrzymywania celu, monitorowania postępów oraz korygowania strategii w razie wystąpienia błędu. Wiele istniejących rozwiązań stanowią modele z dostępem do zewnętrznych interfejsów, niespełniające kryteriów proaktywności i działania zorientowanego na cel.

Pętla decyzyjna stanowi fundament działania agenta. Agent odbiera bieżący stan środowiska, interpretuje go za pomocą modelu językowego i wybiera kolejny krok – na przykład wywołanie narzędzia. Po wykonaniu akcji obserwuje jej wynik i aktualizuje stan zadania. Pętla powtarza się aż do zakończenia zadania [6]. Architektury agentowe różnią się sposobem organizacji pętli decyzyjnej, liczbą jej iteracji oraz podziałem zadań między agentów [5], [9]. Schemat blokowy przedstawiony na rysunku 2.1 ilustruje fazy pojedynczego cyklu decyzyjnego.

Rysunek 2.1 Pętla decyzyjna agenta



Źródło: opracowanie własne.

Funkcjonowanie agenta warunkują trzy elementy: instrukcja systemowa (ang. *system prompt*), okno kontekstowe (ang. *context window*) i schemat narzędzi. Instrukcja systemowa określa rolę agenta, ograniczenia i format wyjścia. Okno kontekstowe pełni rolę pamięci roboczej – gromadzi historię dialogu, wyniki narzędzi, pobrane dowody i pośrednie kroki rozumowania. Schemat narzędzi opisuje, jakie operacje agent może wywołać i w jaki sposób. Niejasna instrukcja systemowa, zbyt małe okno kontekstowe oraz niepełny opis narzędzi stanowią główne przyczyny błędów w implementacji architektur agentowych [3], [5].

2.1.2 Klasyfikacja architektur agentowych

Architektury agentowe klasyfikuje się według różnych kryteriów. Dwa najczęściej stosowane podziały dotyczą liczby agentów uczestniczących w realizacji zadania oraz stopnia autonomii systemu podczas podejmowania decyzji [5], [9], [11]. Kryteria te są ortogonalne – system jednoagentowy może być w pełni autonomiczny, a system wieloagentowy może działać według ściśle deterministycznego schematu [12].

Najprostszym wariantem jest architektura jednoagentowa, w której pojedynczy model językowy samodzielnie obsługuje cały cykl: interpretuje zapytanie, wywołuje narzędzia i formułuje odpowiedź. Zaletą tego podejścia jest prostota implementacji i przewidywalność zachowania – każdy krok jest realizowany przez ten sam model z tym samym oknem kontekstowym. Wadą jest natomiast ograniczona zdolność do równoległego przetwarzania oraz trudność w obsłudze zadań wymagających jednoczesnej specjalizacji w wielu dziedzinach [5]. W praktyce architektury jednoagentowe są odpowiednie dla zadań o dobrze zdefiniowanym zakresie, takich jak odpowiadanie na pytania na podstawie zamkniętego korpusu dokumentów.

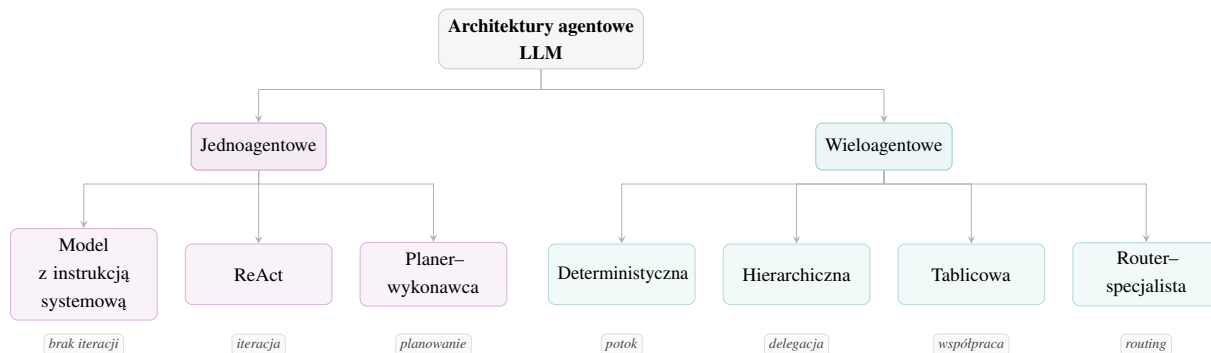
Architektury wieloagentowe dzielą zadanie między kilka współpracujących instancji modelu,

z których każda pełni odrębną rolę – planisty, wykonawcy, weryfikatora lub specjalisty dziedzinowego [11]. Podział ten pozwala na równoległe przetwarzanie niezależnych podzadań i ogranicza ryzyko przeciążenia pojedynczego okna kontekstowego. Złożoność ta skutkuje jednak większą liczbą wywołań modelu, wyższym zużyciem tokenów oraz trudniejszą diagnostyką błędów, ponieważ awaria dowolnego węzła utrudnia lokalizację przyczyny niepowodzenia [9], [11].

Niezależnie od liczby agentów, architektury różnią się również stopniem autonomii w podejmowaniu decyzji **weng2023agentm wang2024survey**, [9]. Wyróżnia się układy deterministyczne, realizujące z góry zdefiniowaną sekwencję kroków, oraz układy w pełni autonomiczne, w których agent samodzielnie planuje i modyfikuje działania na podstawie uzyskanych wyników. Przykładem pierwszego podejścia jest klasyczny potok RAG, drugiego – architektura ReAct, w której model werbalizuje tok rozumowania i wywołuje narzędzia, definiując przebieg zadania [6]. W praktyce powszechnie stosowane są podejścia hybrydowe: deterministyczny szkielet orkiestracji ogranicza przestrzeń decyzyjną, a model dobiera argumenty narzędzi i interpretuje wyniki.

Rysunek 2.2 przedstawia klasyfikację omawianych topologii według dwóch głównych kryteriów. Gałąź lewa obejmuje układy jednoagentowe – od modelu sterowanego wyłącznie instrukcją systemową, przez architekturę ReAct z iteracyjnym rozumowaniem, po układ planer–wykonawca z jawnym etapem planowania. Gałąź prawa grupuje architektury wieloagentowe według dominującego wzorca przepływu informacji: potokowego, hierarchicznego, tablicowego i opartego na routingu. Węzeł *Model z instrukcją systemową* pełni rolę punktu odniesienia.

Rysunek 2.2 Podział architektur agentowych



Źródło: opracowanie własne na podstawie literatury.

2.1.3 Architektury jednoagentowe

Najprostszą architekturą jednoagentową jest model sterowany wyłącznie instrukcją systemową, który odpowiada na podstawie wiedzy odwzorowanej w wagach modelu podczas trenowania. Taki wariant stanowi naturalny punkt odniesienia (ang. *baseline*), ponieważ pozwala oddzielić wpływ użycia narzędzi, wyszukiwania i sterowania od samej jakości modelu bazowego. Sprawdza się przy pytaniach, których odpowiedź została odwzorowana w parametrach modelu, ale wykazuje trudności przy pytaniach wymagających danych aktualnych lub

ściśle dziedzinowych – skutkuje to generowaniem nieprawdziwych informacji, określanych jako halucynacje [13]. Ograniczenie to jest szczególnie istotne w dziedzinach takich jak prawo czy ubezpieczenia, gdzie precyzja i aktualność informacji mają bezpośrednie znaczenie dla użytkownika.

Jedną z propozycji rozwiązania tego problemu jest topologia ReAct, która włącza do procesu decyzyjnego ślad rozumowania (ang. *reasoning trace*). Agent naprzemiennie werbalizuje plany, wykonuje akcję i przetwarza jej rezultat, a decyzja o kolejnym kroku opiera się na pozyskanych dowodach, a nie na z góry określonym scenariuszu [6]. Metoda ta osiąga wysoką skuteczność przy zadaniach wieloetapowych (ang. *multi-hop*), w których odpowiedź wymaga połączenia informacji z kilku niezależnych fragmentów źródłowych. Ma jednak charakterystyczne wady: agent może utracić pierwotny cel lub wpaść w pętlę podczas wielokrotnego odpytywania tego samego zasobu, a narastający zapis kroków rozumowania stopniowo wypełnia okno kontekstowe, ograniczając dostępną przestrzeń dla nowych dowodów [5], [6].

Architektura planer–wykonawca proponuje odmienną strategię – przenosi ciężar obliczeniowy na etap przygotowawczy, w którym najpierw powstaje jawny plan zadania, a dopiero potem wykonywane są kolejne kroki [14]. Rozdzielenie fazy planowania od fazy wykonania ułatwia audytowanie procesu i zwiększa jego przewidywalność. Każdy krok realizowany przez moduł wykonawcy wynika wprost z wcześniej zatwierdzonego planu. Podatnym na błędy punktem jest sam moment tworzenia strategii – błędne rozpoznanie struktury problemu na etapie planowania skutkuje wygenerowaniem fałszywych założeń, które przenoszone są na wszystkie kolejne kroki i istotnie utrudniają późniejszą adaptację. W odróżnieniu od architektury ReAct, gdzie błąd w jednym kroku może być skorygowany w następnym, błąd planisty propaguje się przez cały przebieg zadania.

Zaletą wszystkich układów jednoagentowych jest prostota. Utrzymanie jednego, scentralizowanego kontekstu ułatwia diagnozowanie błędów i wdrożenia produkcyjne, a brak narzutu komunikacyjnego minimalizuje opóźnienia i zużycie tokenów. Ceną za tę prostotę jest pełne obciążenie jednego agenta całym zadaniem – model musi jednocześnie utrzymywać cel, syntetyzować fakty, obsługiwać narzędzia i planować kolejne kroki, co przy wieloetapowych zadaniach szybko obniża jakość odpowiedzi [5], [9]. Ograniczenie to staje się szczególnie widoczne przy pytaniach wymagających zestawienia informacji rozproszonych w wielu dokumentach – właśnie takich, jakie dominują w korpusie użytym w niniejszym eksperymencie.

2.1.4 Architektury wieloagentowe

Systemy wieloagentowe opierają się na założeniu, że podział pracy między wyspecjalizowane role może poprawić jakość odpowiedzi albo obniżyć koszt rozwiązania złożonego zadania. Zyski ze specjalizacji wiążą się jednak z nowymi wyzwaniami: kosztami koordynacji, błędami w przesyłaniu danych między modułami i zarządzaniem współdzielonym stanem aplikacji. Zasadność wdrożenia takiego systemu zależy od możliwości spójnego podzielenia problemu na mniejsze, względnie niezależne części – jeśli zadanie jest trudno dekomponowalne, dodatkowa

złożoność orkiestracji obniża ogólną wydajność systemu [5], [7], [11].

Podstawowym wariantem jest architektura sekwencyjna, w której kilku agentów tworzy łańcuch przetwarzania – wyjście jednego modułu staje się wejściem kolejnego. Układ ten gwarantuje czytelny podział obowiązków i przewidywalny przepływ sterowania, co ułatwia diagnozowanie błędów i analizę poszczególnych etapów przetwarzania. Brak pętli zwrotnej może jednak powodować kaskadowe narastanie błędów – jeśli pierwszy moduł zgromadzi niewystarczający lub błędny materiał, kolejne etapy nie mają możliwości jego weryfikacji ani uzupełnienia [5], [7].

Topologia hierarchiczna – określana również jako układ orkiestrator–pracownicy – formalizuje podział na nadzór i wykonawstwo. Orkiestrator odbiera pytanie, analizuje jego strukturę i rozdziela je na mniejsze podzadania, które przydziela wyspecjalizowanym agentom-pracownikom. Każdy agent-pracownik wykonuje swój fragment i zwraca wynik orkiestratorowi, który łączy je w spójną odpowiedź końcową. Umożliwia to wdrożenie wąskich specjalizacji – poszczególni pracownicy mogą korzystać z różnych instrukcji systemowych, filtrów metadanych lub zestawów narzędzi dostosowanych do swojej roli. Hong i in. wykazali skuteczność tego podejścia w MetaGPT, przypisując agentom wyspecjalizowane role – m.in. architekta, programisty i testera – i koordynując ich współpracę zgodnie z ustrukturyzowanym przepływem pracy [8]. Głównym ryzykiem architektury hierarchicznej jest przeciążenie orkiestratora: przy złożonych zadaniach moduł koordynujący może stać się wąskim gardłem całego systemu.

Architektura tablicowa (ang. *blackboard*) jest zorganizowana wokół koncepcji współdzielonego stanu zadania, dostępnego dla wszystkich modułów systemu. Rezygnuje ze sztywno narzuconej kolejności działań – system dynamicznie wybiera kolejne akcje na podstawie analizy braków informacyjnych w centralnej bazie. Podejście to jest adaptacyjne i wykazuje skuteczność przy zadaniach o nieprzewidywalnej strukturze, w których sekwencja kroków nie może być z góry określona. Wymaga jednak rygorystycznych mechanizmów sterujących: błędny lub niekompletny wpis na tablicy może zaburzyć proces decyzyjny i prowadzić do trudnych do wykrycia błędów w odpowiedzi końcowej [5].

Architektura oparta na routerze wykorzystuje mechanizm mieszaniny ekspertów (ang. *mixture-of-experts*) [15]. Lekki agent-router klasyfikuje pytanie i kieruje je do najlepiej dopasowanego specjalisty spośród dostępnych modułów. Poszczególni agenci-specjaliści działają niezależnie od siebie, a dla jednego pytania uruchamiany jest tylko jeden agent-specjalista, co przekłada się na niskie zużycie tokenów i krótki czas odpowiedzi. Podejście cechuje zatem wysoka wydajność operacyjna w porównaniu z układami hierarchicznymi czy tablicowymi. Wadą tego podejścia jest ryzyko błędnej klasyfikacji na etapie routingu – nieprawidłowy wybór specjalisty ogranicza dostęp do zasobów i strategii wyszukiwania potrzebnych w danym przypadku [5], [11].

Tabela 2.1 zestawia wszystkie omawiane architektury według jednolitych kryteriów.

Tabela 2.1 Porównanie architektur agentowych

Topologia	Komunikacja	Pamięć	Dekompozycja	Mocne strony	Słabe strony	Źródło
Deterministyczna	Liniowy przepływ sterowania bez rozgałęzień	Wspólne okno kontekstowe dla całego przebiegu	Brak – sekwencja kroków ustalona statycznie	Pełna powtarzalność wyników, niski narzut obliczeniowy	Niemożliwość adaptacji gdy pobrane fragmenty są niewystarczające	[5]
Planer–wykonawca	Jednostronna delegacja: moduł planujący przekazuje kroki wykonawcy	Plan przechowywany jako jawny stan między fazami	Statyczna, realizowana przed pierwszym wywołaniem narzędzia	Każdy krok audytowalny i powiązany z konkretnym założeniem planu	Błąd w fazie planowania propaguje się przez wszystkie kolejne kroki	[14]
Router–specjalista	Dwuetażowa: router klasyfikuje pytanie, specjalista przetwarza niezależnie	Izolowany kontekst lokalny każdego specjalisty	Niejawna – router wybiera jedną ze zdefiniowanych ścieżek	Niskie zużycie tokenów dzięki aktywacji jednego specjalisty na zapytanie	Błędna klasyfikacja blokuje dostęp do właściwego specjalisty	[15]
Tablicowa	Asynchroniczna przez współdzielony obiekt stanu dostępny wszystkim modułom	Centralna tablica jako jedyne źródło prawdy o stanie zadania	Dynamiczna – moduł wybierany na podstawie braków na tablicy	Brak ustalonej kolejności modułów umożliwia obsługę nieoczekiwanych pytań	Niespójny wpis na tablicy może skierować proces na błędne tory	[5]
Hierarchiczna	Dwuopozycyjna: orkiestrator rozdziela podzadania, pracownicy zwracają wyniki	Prywatne okno każdego pracownika, orkiestrator widzi tylko wyniki częściowe	Jawna przez orkiestratora przed wykonaniem	Izolacja kontekstów pracowników redukuje ryzyko wzajemnego zakłócania	Orkiestrator może utracić szczegóły potrzebne do syntezy końcowej	[7], [8]
ReAct	Naprzemienne: rozumowanie przeplatane wywołaniami narzędzi	Okno kontekstowe gromadzi cały ślad rozumowania i wyniki narzędzi	Iteracyjna – kolejny krok wynika z analizy wyniku poprzedniego	Ślad rozumowania pozwala zidentyfikować fragmenty wpływające na odpowiedź	Narastający ślad szybko wypełnia okno kontekstowe przy długich zadaniach	[6]
Debate	Wielostronna wymiana argumentów zakończona głosowaniem lub syntezą	Wspólny wątek dyskusji widoczny dla wszystkich uczestników	Przez równoległe generowanie stanowisk i ich iteracyjną konfrontację	Weryfikacja krzyżowa między agentami ogranicza ryzyko błędnej odpowiedzi	Wielokrotne rundy generują wysoki koszt tokenowy nawet przy prostych pytaniach	[16]
Reflection	Sekwencyjna: agent generuje, krytyk ocenia, agent wprowadza poprawki	Okno kontekstowe przechowuje historię kolejnych wersji odpowiedzi	Przez iteracyjną samokrytykę – każda rewizja jako osobny krok	Wielokrotna weryfikacja redukuje błędy faktyczne i logiczne	Słabość do rewizji poprawnych odpowiedzi zwiększa koszt bez zysku jakościowego	[17]

Źródło: opracowanie własne na podstawie literatury.

2.1.5 Interfejsy narzędziowe agentów

Zdolność do korzystania z narzędzi zewnętrznych stanowi jedną z kluczowych cech odróżniających agenta językowego od klasycznego modelu generatywnego. We wczesnych systemach tę ideę realizowano przez parsowanie wyjścia modelu w poszukiwaniu z góry ustalonych słów kluczowych – podejście podatne na błędy i trudne do skalowania. Współczesne modele językowe obsługują wywoływanie funkcji (ang. *function calling*): model zwraca ustrukturyzowany obiekt JSON zawierający nazwę funkcji i jej argumenty, który warstwa aplikacyjna interpretuje, wykonuje odpowiednią operację i odsyła wynik do modelu jako kolejny komunikat w wątku konwersacji. Schemat ten upowszechnił się i stał się standardem w większości współczesnych platform agentowych.

Schick i in. wykazali, że modele językowe można nauczyć samodzielnego decydowania o tym, kiedy i w jaki sposób wywoływać narzędzia, bez konieczności ręcznego adnotowania każdego przypadku użycia. Kryterium uczenia jest proste: wywołanie narzędzia jest uzasadnione wtedy, gdy zmniejsza stratę predykcji dla kolejnego tokena [3].

W produkcyjnych systemach agentowych narzędzia przyjmują wiele postaci: są to m.in. wyszukiwarki wektorowe, interfejsy do baz danych, kalkulatory, parsery dokumentów oraz wywołania zewnętrznych API. Każde narzędzie opisywane jest schematem zawierającym jego nazwę, opis funkcjonalny i typy argumentów. Opis ten wchodzi w skład okna kontekstowego modelu i bezpośrednio wpływa na jakość podejmowanych decyzji [5], [18]. Niedostatecznie precyzyjny opis narzędzia należy do najczęstszych źródeł błędów w systemach agentowych: model albo pomija wywołanie tam, gdzie jest ono konieczne, albo przekazuje błędne wartości argumentów.

Istotnym czynnikiem wpływającym na skuteczność agenta jest również liczba dostępnych narzędzi. Badania pokazują, że wraz ze wzrostem ich liczby rośnie ryzyko błędnego wyboru – model może wywołać narzędzie nieadekwatne do kontekstu lub niepotrzebnie łączyć kilka narzędzi tam, gdzie wystarczyłoby jedno [19]. Zjawisko to wynika z ograniczonej pojemności okna kontekstowego. Im więcej schematów narzędzi musi przetworzyć model jednocześnie, tym trudniej mu trafnie ocenić, które z nich jest właściwe w danym kroku rozumowania. W praktyce zaleca się zatem ograniczanie zestawu narzędzi do tych rzeczywiście niezbędnych dla danego zadania.

2.1.6 Protokoły komunikacji agentów

Jakość architektury wieloagentowej zależy od kompetencji poszczególnych agentów oraz od skuteczności wymiany informacji między węzłami. Wczesne implementacje przesyłały wiadomości jako niesformatowany tekst, co prowadziło do błędów parsowania i nadinterpretacji semantycznych na styku modułów. Rozwiązaniem okazało się zastosowanie ustrukturyzowanych formatów wyjściowych w postaci walidowanych schematów JSON. Autorzy AutoGen wskazują, że ograniczenie schematu odpowiedzi agentów do określonych struktur ułatwia konsumpcję wyników przez inne węzły systemu [7]. Rozwiązanie to ogranicza jednak swobodę generacji –

model musi przestrzegać narzuconego schematu nawet wtedy, gdy naturalna odpowiedź tekstowa byłaby bardziej precyzyjna.

Wu i in. wyróżnili cztery wzorce przepływu informacji: rozmowę dwuagentową, sekwencyjną, zagnieżdżoną oraz grupową [7]. Wzorce sekwencyjne i zagnieżdżone narzucają z góry określony porządek komunikacji, zapewniając przewidywalność i łatwość debugowania. Rozmowy grupowe pozwalają natomiast na dynamiczny wybór kolejnego uczestnika na podstawie bieżącego kontekstu. Podejście to zwiększa adaptacyjność systemu, jednak jest trudniejsze w kontrolowaniu i może generować niespójne wyniki przy kolejnych uruchomieniach tego samego zadania. Jest to szczególnie problematyczne w kontekście replikowalności wyników eksperymentów, gdzie oczekuje się, że identyczne dane wejściowe prowadzą do porównywalnych przebiegów [5], [7].

W systemach pracujących na wielostronicowych dokumentach konieczna staje się selekcja i kompresja istotnych fragmentów kontekstu przed przekazaniem ich między modułami. Pojedynczy agent rzadko dysponuje oknem kontekstowym wystarczającym do przetworzenia całego tekstu źródłowego, co wymusza stosowanie strategii redukcji: generowania streszczeń, ekstrakcji kluczowych pojęć lub przesyłania wyselekcjonowanych metadanych [5]. Każda z tych strategii wiąże się jednak z ryzykiem utraty niuansów semantycznych – szczególnie istotnym w dokumentach prawnych i ubezpieczeniowych, gdzie zmiana jednego słowa może odwrócić sens klauzuli lub zakres ochrony ubezpieczeniowej.

2.1.7 Systemy pamięci agentów

Zdolność systemu do utrzymywania stanu między krokami decyzyjnymi jest bezpośrednio zależna od zastosowanego układu pamięci. Sumers i in. w ramach koncepcji architektury poznawczej dla agentów językowych (ang. *Cognitive Architectures for Language Agents*, CoALA) wyróżnili cztery kategorie pamięci [20]. Kategorie te nie są rozłączne – w praktycznych implementacjach jeden system może korzystać jednocześnie z kilku rodzajów pamięci, a ich wzajemne relacje determinują sposób aktualizacji wiedzy agenta w toku zadania.

Pamięć robocza: przechowuje aktywne informacje dla bieżącego cyklu decyzyjnego – dane znajdujące się aktualnie w przestrzeni wejściowej modelu, w tym informacje percepcyjne, aktywna wiedza oraz cele agenta. Działa najszybciej spośród wszystkich kategorii, ale jej zawartość ulega zmianie wraz z każdym cyklem przetwarzania i jest ograniczona rozmiarem okna kontekstowego modelu. W praktyce oznacza to, że przy długich zadaniach wieloetapowych wcześniej pobrane fragmenty dokumentów mogą zostać wyparte przez kolejne wyniki narzędzi [20].

Pamięć epizodyczna: gromadzi informacje z wcześniejszych cykli decyzyjnych – historie przepływu zdarzeń czy trajektorie z poprzednich epizodów [20]. Ułatwia unikanie powtarzania błędów poprzez dostęp do wcześniejszych prób, ale wymaga mechanizmu persystencji między sesjami. Większość badanych w niniejszej pracy architektur nie implementuje tego

rodzaju pamięci – każde uruchomienie rozpoczyna się od pustego stanu.

Pamięć semantyczna: przechowuje wiedzę agenta o świecie – tradycyjnie inicjalizowaną z zewnętrznych baz danych lub nieustrukturyzowanego tekstu. W przeciwieństwie do pamięci epizodycznej zawiera uogólnioną, bezkontekstową wiedzę dziedzinową [20]. W systemach RAG rolę pamięci semantycznej pełni indeks wektorowy – agent nie przechowuje wiedzy w swoich parametrach, lecz odpytuje zewnętrzne repozytorium w razie potrzeby.

Pamięć proceduralna: przyjmuje w agentach językowych dwie formy: ukrytą wiedzę przechowywaną w wagach modelu językowego oraz jawną wiedzę zapisaną w kodzie agenta [20]. Pierwsza jest nabywana podczas trenowania i pozostaje niezmienna w trakcie działania systemu, druga może być modyfikowana przez programistę bez konieczności ponownego trenowania modelu.

Sposób dystrybucji tych zasobów determinuje zachowanie systemów agentowych. Architektury rozproszone wymagają precyzyjnego określenia, które dane stanowią wiedzę prywatną węzła, oraz zdefiniowania procedur propagowania ustaleń lokalnych do globalnego stanu aplikacji [5], [20]. Brak takiej specyfikacji prowadzi do niespójności między modułami – sytuacji, w której różne węzły systemu operują na sprzecznych założeniach dotyczących stanu zadania.

2.1.8 Ewaluacja agentów

Weryfikacja jakości działania systemów agentowych jest trudniejsza niż klasyczna ocena modeli przetwarzania języka naturalnego. Narzędzia takie jak BLEU, ROUGE czy BERTScore oceniają wyłącznie podobieństwo leksykalne do wzorca [21], [22] i okazują się nieskuteczne przy sprawdzaniu poprawności merytorycznej w systemach opartych na wieloetapowym rozumowaniu. Model może wygenerować tekst płynny i poprawny gramatycznie, formułując jednocześnie twierdzenia sprzeczne z faktami – klasyczne metryki tekstowe nie są w stanie tego wykryć.

Ewaluacja agentów wykracza również poza ocenę samych modeli językowych, ponieważ agent musi być sprawdzany w dynamicznym środowisku zadaniowym, a nie wyłącznie jako izolowany generator tekstu. Liu i in. zaproponowali zestaw testów porównawczych (ang. *benchmark*) AgentBench obejmujący osiem różnych środowisk interaktywnych – od zadań z kodem i grami po przeglądanie stron internetowych [23]. Badanie to ujawniło, że główne wyzwania w implementacji agentów to słabe długoterminowe rozumowanie, podejmowanie decyzji oraz stosowanie się do instrukcji, a nie sama jakość generowanego tekstu. Zdolność agenta do utrzymania celu przez wiele kroków ma bezpośredni wpływ na jakość odpowiedzi na pytania wieloskokowe wymagające zestawienia informacji z kilku sekcji dokumentu.

Metryki specyficzne dla systemów agentowych koncentrują się na przebiegu działania, a nie tylko na jego wyniku końcowym. Kluczową miarą jest wskaźnik ukończenia zadania (ang. *task completion rate*), określający jaki odsetek zadań agent doprowadził do poprawnego zakończenia bez interwencji zewnętrznej [23]. Uzupełnieniem jest ewaluacja trajektorii (ang. *trajectory*

evaluation), polegająca na ocenie sekwencji kroków podjętych przez agenta – nie tylko tego, czy osiągnął cel, ale również czy obrał właściwą ścieżkę i czy nie wykonał zbędnych lub szkodliwych wywołań [5]. Istotnym wymiarem tej oceny jest analiza wywołań narzędzi: czy agent sięgnął po właściwe narzędzie we właściwym momencie, czy nie pominął wywołania tam gdzie było ono niezbędne, oraz czy nie wykonał nadmiarowych zapytań do bazy wektorowej [18], [23]. Równie ważna jest liczba pętli decyzyjnych zrealizowanych przez agenta dla danego zadania. Niska liczba iteracji przy wysokiej poprawności świadczy o efektywnym planowaniu, natomiast wysoka liczba pętli przy niskiej jakości odpowiedzi wskazuje na trudności z konwergencją lub błędną strategię wyszukiwania. W kontekście systemów odpowiadających na pytania na podstawie dokumentów trajektoria obejmuje w szczególności dobór zapytań do bazy wektorowej, liczbę iteracji wyszukiwania oraz sposób syntezy pobranych fragmentów w odpowiedź końcową.

Obok oceny poprawności odpowiedzi istotnym wymiarem porównania architektur jest efektywność operacyjna, mierzona liczbą kroków pośrednich, wywołań narzędzi, zużyciem tokenów oraz opóźnieniem odpowiedzi [5]. Uzupełnieniem pomiarów ilościowych jest analiza jakościowa śladów wykonania (ang. *traces*) i ich składowych kroków (ang. *spans*) – ręczne przeglądanie kolejnych etapów przebiegu pozwala ustalić, w którym miejscu agent traci kontekst, wykonuje zbędne wywołania lub generuje błędne założenia pośrednie.

Osobnym zagadnieniem metodologicznym jest ewaluacja z wykorzystaniem modelu językowego w roli sędziego (ang. *LLM-as-a-judge*). Zheng i in. wykazali, że modele klasy GPT-4 osiągają ponad 80% zgodności z ocenami ekspertów, co czyni tę metodę praktyczną alternatywą dla kosztownych ręcznych adnotacji – szczególnie przy dużej liczbie uruchomień [24]. Autorzy wskazali jednocześnie na szereg systematycznych odchyleń: stronniczość pozycyjną (ang. *positional bias*), polegającą na faworyzowaniu odpowiedzi prezentowanej jako pierwsza, stronniczość gadatliwości (ang. *verbosity bias*), wyrażającą się preferencją dla dłuższych wypowiedzi, oraz tendencję do faworyzowania stylu zbliżonego do własnego (ang. *self-enhancement bias*). Odchylenia te należy uwzględniać przy interpretacji wyników automatycznej oceny, traktując je jako oszacowanie jakości, a nie rozstrzygnięcie ostateczne [25].

Tabela 2.2 zestawia metryki stosowane w ewaluacji systemów agentowych wraz z opisem mierzonego wymiaru i metodą pomiaru.

Tabela 2.2 Porównanie metryk ewaluacji systemów agentowych

Metryka	Co mierzy	Metoda pomiaru	Źródło
Wskaźnik ukończenia zadania (ang. <i>task completion rate</i>)	Odsetek zadań doprowadzonych do poprawnego zakończenia bez interwencji zewnętrznej	Zliczanie zakończonych przebiegów względem wszystkich uruchomień	[23]
Liczba pętli decyzyjnych	Liczba iteracji pętli wykonanych przez agenta dla danego zadania	Zliczanie kroków w śladzie wykonania	–
Poprawność wywołań narzędzi	Zgodność wywołanego narzędzia i jego argumentów z wymaganiami zadania	Analiza jakościowa śladów wykonania i spanów	[18], [23]
Ewaluacja trajektorii (ang. <i>trajectory evaluation</i>)	Ocena sekwencji kroków pod kątem zbędnych lub szkodliwych wywołań	Porównanie sekwencji kroków ze wzorcową trajektorią	[5]
Analiza śladów wykonania (ang. <i>traces, spans</i>)	Identyfikacja miejsca przebiegu, w którym agent traci kontekst lub generuje błędne założenia pośrednie	Ręczne przeglądanie kolejnych spanów w narzędziu obserwacyjnym	–
Opóźnienie (ang. <i>latency</i>)	Czas upływający od momentu zapytania do dostarczenia odpowiedzi końcowej	Pomiar czasu wykonania w sekundach	[5]
Zużycie tokenów	Całkowity koszt tokenowy przebiegu obejmujący tokeny wejściowe i wyjściowe	Zliczanie przez API modelu	[5]

Źródło: opracowanie własne na podstawie literatury.

2.1.9 Przegląd istniejących środowisk agentowych

Rozwój badań nad systemami wieloagentowymi doprowadził do powstania wyspecjalizowanych bibliotek orkiestracyjnych upraszczających ich implementację. Pakiety tego typu izolują kod zarządzający komunikacją, pamięcią i walidacją wywołań narzędzi, pozwalając programistom skupić się na logice biznesowej systemu. Poszczególne biblioteki różnią się jednak przyjętą filozofią projektową – wybór środowiska determinuje dostępne topologie, sposób zarządzania stanem oraz stopień kontroli nad przepływem sterowania.

Podstawowym środowiskiem wykorzystanym w niniejszej pracy jest biblioteka LangGraph. Reprezentuje ona operacje agentowe za pomocą grafów skierowanych, w których wierzchołki odpowiadają stanom decyzyjnym, a krawędzie implementują logikę przejść między nimi. Biblioteka zapewnia mechanizm kontroli oparty na rygorystycznie typowanych obiektach stanu w języku Python, co umożliwia precyzyjne modelowanie złożonych przepływów pracy i deterministyczne zarządzanie sekwencją operacji. Właściwości te ułatwiają testowanie i debugowanie aplikacji wieloagentowych, a także precyzyjne izolowanie zmiennej niezależnej w eksperymentach porównawczych.

Rozwiązanie AutoGen przyjmuje odmienne podejście projektowe. Wu i in. zaprojektowali środowisko oparte na wzorcach konwersacyjnych między agentami, gdzie informacje przepływają przez przyrastający stos wiadomości tekstowych [7]. Biblioteka oferuje cztery wzorce komunikacji – rozmowy dwuagentowe, sekwencyjne, zagnieżdżone i grupowe – właściwe dla scenariuszy wymagających elastycznej współpracy agentów. Mechanika konwersacyjna utrudnia jednak modelowanie procedur wymagających ścisłej kontroli przepływu sterowania.

Poza tymi dwoma środowiskami na rynku funkcjonuje kilka innych bibliotek orkiestracyjnych. CrewAI przyjmuje podejście deklaratywne, w którym programista definiuje role i zadania agentów, a framework samodzielnie zarządza ich wykonaniem. LlamaIndex opiera się na grafie zdarzeń asynchronicznych, co czyni go szczególnie przydatnym w potokach wyszukiwania i indeksowania dokumentów. Semantic Kernel z kolei integruje się z ekosystemem Microsoft i oferuje model oparty na wtyczkach oraz planerze funkcji.

Tabela 2.3 zestawia omówione środowiska według wspieranych topologii, modelu programowania, sposobu zarządzania stanem oraz typowego zastosowania.

Tabela 2.3 Porównanie środowisk orkiestracji systemów agentowych

Środowisko	Wspierane topologie	Model programowania	Zarządzanie stanem	Typowe zastosowanie
LangGraph	Deterministyczna, hierarchiczna, tablicowa, ReAct	Stanowy graf skierowany, typowany schemat stanu w Pythonie	Współdzielony, silnie typowany obiekt stanu	Kontrolowane eksperymenty, deterministyczne przepływy
AutoGen	Hierarchiczna, konwersacyjna, grupowa	Wzorce konwersacji między agentami, stos wiadomości	Przyrastający kontekst wiadomości	Elastyczna współpraca agentów, prototypowanie
CrewAI	Sekwencyjna, hierarchiczna	Deklaratywna definicja ról i zadań	Pamięć per agent, wspólny wynik	Zadania oparte na rolach, automatyzacja procesów
LlamaIndex	Zdarzeniowa, potokowa	Graf zdarzeń asynchronicznych	Kontekst przekazywany przez zdarzenia	Potoki RAG, indeksowanie i wyszukiwanie dokumentów
Semantic Kernel	Sekwencyjna, planująca	Wtyczki i planer funkcji	Kontekst jądra, pamięć wektorowa	Integracja enterprise, ekosystem Microsoft

Źródło: opracowanie własne na podstawie dokumentacji technicznej projektów.

2.2. Generowanie wspomagane wyszukiwaniem

Statyczny charakter treningu uniemożliwia modelom językowym uwzględnianie informacji powstałych po dacie zamknięcia zbioru uczącego (ang. *knowledge cutoff*). Generowanie wspomagane wyszukiwaniem (ang. *Retrieval-Augmented Generation*, RAG) rozwiązuje ten problem przez dynamiczne uzupełnianie kontekstu modelu fragmentami pobranymi z zewnętrznego korpusu w czasie wnioskowania [4]. Kolejne podrozdziały omawiają mechanizmy pobierania wiedzy, problem halucynacji i ograniczeń okna kontekstowego oraz kierunki rozbudowy klasycznych potoków RAG w stronę systemów agentowych.

2.2.1 Pojęcie RAG

Generowanie wspomagane wyszukiwaniem opiera się na założeniu: model językowy nie musi przechowywać całej potrzebnej wiedzy w swoich parametrach – może najpierw pobrać pasujące fragmenty z zewnętrznego źródła, a dopiero potem wygenerować odpowiedź na ich podstawie. Komponent wyszukiwający (ang. *retriever*) lokalizuje użyteczne fragmenty, a komponent generujący (ang. *generator*) interpretuje je i formułuje odpowiedź. Podejście to jest szczególnie istotne wtedy, gdy poprawna odpowiedź zależy od wiedzy dziedzinowej, zamkniętego korpusu dokumentów lub informacji nieobecnych w danych treningowych modelu [4].

Dominującym wzorcem RAG stał się schemat pobierz-następnie-czytaj (ang. *retrieve-then-read*): najpierw pobiera się pasujące fragmenty, a następnie model generuje odpowiedź na ich podstawie. Wyszukiwarki skutecznie lokalizują informacje w dużych zbiorach tekstu, lecz same nie interpretują ich znaczenia. Modele generatywne syntetyzują i formułują odpowiedź, lecz bez dostępu do zewnętrznej bazy wiedzy wykazują tendencję do generowania halucynacji. RAG łączy mocne strony obu komponentów, ograniczając jednocześnie słabości każdego z nich z osobna [26], [27].

2.2.2 Problem halucynacji w modelach językowych

Halucynacja w kontekście modeli językowych oznacza generowanie treści pozornie spójnych i wiarygodnych, lecz niezgodnych z faktami lub niemożliwych do zweryfikowania na podstawie dostarczonych źródeł [28]. Ji i in. wyróżniają dwa główne typy: halucynacje wewnętrzne (ang. *intrinsic*), gdzie model zaprzecza informacjom zawartym w kontekście, oraz halucynacje zewnętrzne (ang. *extrinsic*), gdzie model dodaje twierdzenia niemające pokrycia w żadnym ze źródeł [13]. Oba typy są szczególnie szkodliwe w dziedzinach regulacyjnych – błędnie przytoczony limit sumy ubezpieczenia lub pominięte wyłączenie odpowiedzialności mogą prowadzić do realnych szkód dla użytkownika końcowego.

Źródłem halucynacji jest fundamentalna cecha modeli językowych: generują one tekst na podstawie statystycznych wzorców wyuczonych z danych treningowych, a nie na podstawie dostępu do weryfikowalnych faktów. Model nie odróżnia informacji pewnych od domysłów – produkuje oba rodzaje treści z podobnym poziomem pewności językowej [13]. Problem narasta przy pytaniach dziedzinowych, gdzie wiedza zakodowana w parametrach modelu jest niepełna

lub nieaktualna, a model zamiast przyznać brak wiedzy, uzupełnia luki pozornie wiarygodnymi szczegółami.

Generowanie wspomagane wyszukiwaniem stanowi bezpośrednią odpowiedź na ten problem: zamiast polegać wyłącznie na wiedzy zapisanej w wagach, system pobiera trafne fragmenty z zewnętrznego korpusu i dostarcza je modelowi jako kontekst do generowania odpowiedzi [4]. Samo dostarczenie kontekstu nie eliminuje jednak halucynacji całkowicie. Lewis i in. wskazują, że model może ignorować pobrane fragmenty na rzecz wiedzy parametrycznej, szczególnie gdy kontekst jest długi lub zawiera informacje sprzeczne z wewnętrzną wiedzą modelu [4]. Problem ten narasta w systemach wieloagentowych: każdy węzeł może wprowadzić własne nieścisłości, które propagują się przez kolejne etapy przetwarzania – zjawisko określane jako kaskadowa propagacja błędu (ang. *cascading error propagation*) [5].

Tabela 2.4 zestawia oba typy halucynacji wraz z podkategoriami i przykładami. Topologia agenta wpływa na profil halucynacyjny systemu w dwóch wymiarach: liczba kroków przetwarzania determinuje liczbę okazji do odejścia od oryginalnych dokumentów, a sposób zarządzania kontekstem między modułami – architektury przekazujące streszczenia zamiast pełnych fragmentów – zwiększa ryzyko uzupełniania luk domysłami modelu.

Tabela 2.4 Porównanie typów halucynacji w modelach językowych

Typ halucynacji	Podkategoria	Definicja	Przykład
Wewnętrzna	Sprzeczność logiczna	Wnioskowanie sprzeczne z przesłankami w tekście	Model twierdzi, że $A > B$, mimo że tekst mówi $A < B$
Wewnętrzna	Sprzeczność kontekstowa	Generowanie faktów sprzecznych z wcześniejszą częścią własnej wypowiedzi	W jednym akapicie model potwierdza pokrycie szkody, w kolejnym zaprzecza
Zewnętrzna	Fabrykacja faktów	Tworzenie nieistniejących bytów, cytatów lub danych	Model cytuje nieistniejący wyrok sądu [29]
Zewnętrzna	Niespójność instrukcji	Ignorowanie ograniczeń narzuconych w prompcie	Model generuje kod w Pythonie, mimo prośby o C++

Źródło: opracowanie własne na podstawie [28].

2.2.3 Problem okna kontekstowego w modelach językowych

Każdy potok RAG działa w granicach wyznaczonych przez rozmiar okna kontekstowego modelu językowego. Nowoczesne modele obsługują okna rzędu setek tysięcy tokenów, jednak w praktyce przestrzeń ta jest dzielona między instrukcję systemową, historię konwersacji, pobrane fragmenty dokumentów oraz pośrednie kroki rozumowania. Efektywna przestrzeń dostępna na pobrane fragmenty dokumentów jest zatem znacznie mniejsza niż nominalna pojemność okna [5].

Samo zwiększanie okna kontekstowego nie rozwiązuje jednak problemu jakości wykorzystania zawartych w nim informacji. Liu i in. wykazali, że modele językowe wykazują systematyczną tendencję do pomijania fragmentów umieszczonych w środkowej części kontekstu – efektywnie przetwarzając jedynie informacje z jego początku i końca [30]. Zjawisko to, określane jako *lost-in-the-middle*, sprawia, że kolejność wstawiania pobranych fragmentów do okna ma bezpośredni wpływ na jakość odpowiedzi, niezależnie od ich merytorycznej trafności. W dokumentach ubezpieczeniowych, gdzie kluczowe wyłączenia odpowiedzialności mogą znajdować się w środkowych sekcjach wielostronicowego OWU, zjawisko to znacząco zwiększa ryzyko błędnych odpowiedzi.

W systemach agentowych oba ograniczenia nakładają się na koszt orkiestracji. W architekturze ReAct każda iteracja pętli decyzyjnej dopisuje do kontekstu nową warstwę rozumowania i wynik narzędzia [6], w systemach hierarchicznych orkiestrator musi natomiast zmieścić w swoim oknie wyniki wszystkich agentów podrzędnych [8].

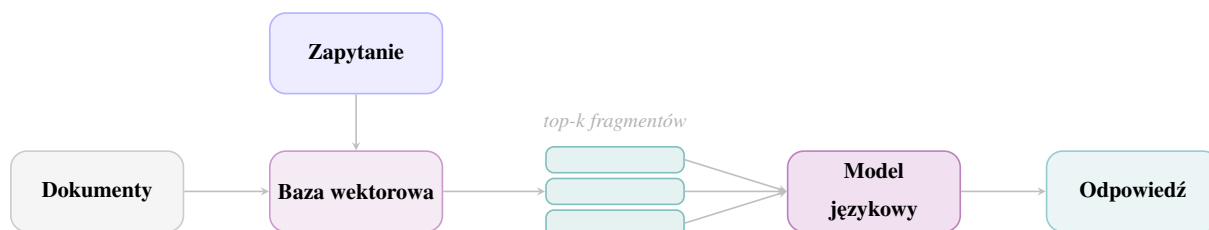
2.2.4 Architektury RAG

Gao i in. wyróżniają trzy kategorie implementacyjne systemów RAG różniące się stopniem złożoności potoku przetwarzania [31]. Podział ten odzwierciedla ewolucję podejścia do problemu – od najprostszego układu jednorazowego pobrania i generacji, przez rozbudowane potoki wzbogacone o etapy przetwarzania wstępnego i końcowego, aż po architektury modularne pozwalające na swobodną konfigurację komponentów. Każda kolejna kategoria uwzględnia ograniczenia poprzedniej, jednocześnie zwiększając złożoność implementacji i koszty operacyjne.

Prosty RAG (ang. *naive RAG*) wykonuje jednorazowe pobranie fragmentów na podstawie zapytania użytkownika i na tej podstawie generuje odpowiedź. Zapytanie przekształcane jest w wektor, wyszukiwarka zwraca stałą liczbę najbardziej podobnych fragmentów, a model językowy syntetyzuje odpowiedź. Układ jest odpowiedni dla pytań prostych, gdzie odpowiedź jest zawarta w jednym lub kilku sąsiadujących fragmentach. Ograniczenia ujawniają się przy zadaniach wymagających połączenia informacji z wielu źródeł – system nie weryfikuje trafności pobranych fragmentów i przekazuje do generatora stałą ich liczbę niezależnie od tego, czy faktycznie zawierają potrzebną informację [31].

Rysunek 2.3 przedstawia potok prostego RAG – zapytanie trafia bezpośrednio do bazy wektorowej, a pobrane fragmenty przekazywane są do modelu bez dodatkowego przetwarzania.

Rysunek 2.3 Potok prostego RAG

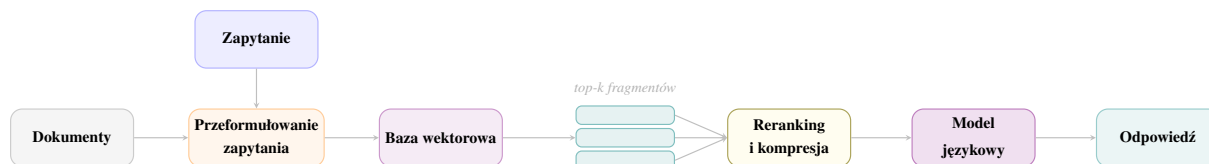


Źródło: opracowanie własne na podstawie [31].

Zaawansowany RAG (ang. *advanced RAG*) rozbudowuje potok o dodatkowe etapy zarówno przed wyszukiwaniem, jak i po nim. Przed wyszukiwaniem stosuje się przeformułowanie zapytania (ang. *query rewriting*) lub rozkład pytania złożonego na podzapytania. Po wyszukiwaniu pobrane fragmenty poddawane są rerankingowi oraz kompresji usuwającej informacje nieistotne dla pytania. Każdy z tych etapów poprawia jakość kontekstu trafiającego do generatora bez konieczności zmiany samego modelu językowego [31].

Rysunek 2.4 przedstawia potok zaawansowanego RAG. Zapytanie użytkownika przechodzi najpierw przez moduł przeformułowania, który może je uprościć, rozbudować lub rozbić na podzapytania. Pobrane fragmenty poddawane są rerankingowi i kompresji przed przekazaniem do modelu językowego.

Rysunek 2.4 Potok zaawansowanego RAG



Źródło: opracowanie własne na podstawie [31].

Modularny RAG (ang. *modular RAG*) traktuje wyszukiwarkę, reranker, kompresor kontekstu i generator jako wymienne komponenty, które można konfigurować i zestawiać w zależności od wymagań konkretnego zadania. Pozwala to zastępować poszczególne moduły nowszymi rozwiązaniami bez przebudowy całego systemu. Złożone warianty modularnego RAG wymagają kolejnych decyzji o tym, czego szukać i kiedy zakończyć wyszukiwanie – co zbliża je do zachowania agentowego [31].

Rysunek 2.5 przedstawia potok modularnego RAG. Poszczególne komponenty – wyszukiwarka, reranker, kompresor – są reprezentowane jako wymienne moduły oznaczone przerywaną ramką, które można zastępować i rekonfigurować niezależnie od pozostałych etapów potoku.

Rysunek 2.5 Potok modularnego RAG



Źródło: opracowanie własne na podstawie [31].

2.2.5 Metody wyszukiwania dokumentów

Podstawą wyszukiwania semantycznego w systemach RAG jest koncepcja osadzenia (ang. *embedding*) – reprezentacji tekstu w postaci wektora liczb rzeczywistych w przestrzeni wysokowymiarowej. Model osadzający przekształca fragment tekstu – zdanie, akapit lub dokument – w wektor o stałej długości, przy czym teksty o zbliżonym znaczeniu otrzymują wektory leżące blisko siebie w tej przestrzeni, niezależnie od użytych słów. Jakość osadzeń zależy bezpośrednio od modelu – modele trenowane na dużych korpusach wielojęzycznych precyzyjniej modelują znaczenie w specjalistycznych domenach, podczas gdy modele ogólne mogą niepoprawnie reprezentować terminologię dziedzinową [32].

Wygenerowane osadzenia przechowywane są w wyspecjalizowanych strukturach danych zwanych bazami wektorowymi (ang. *vector stores*) lub indeksami wektorowymi. Baza wektorowa umożliwia wydajne przeszukiwanie milionów wektorów w celu znalezienia tych najbliższych zapytaniu – operacja określana jako przeszukiwanie przybliżonych sąsiadów (ang. *approximate nearest neighbour search*, ANN). Do popularnych implementacji należą FAISS [33], Chroma, Pinecone oraz Weaviate. Wybór bazy wektorowej wpływa na kompromis między szybkością wyszukiwania a precyzją wyników – niektóre implementacje poświęcają część dokładności na rzecz niższego opóźnienia, co może mieć znaczenie w systemach obsługujących duże wolumeny zapytań.

Wyszukiwanie rzadkie (ang. *sparse retrieval*) opiera się na dopasowaniu leksykalnym – dokumenty reprezentowane są jako zbiory ważonych termów, a trafność mierzy się częstością ich wystąpień. Klasycznym przykładem jest BM25, który uwzględnia zarówno częstość termu w dokumencie, jak i jego rzadkość w całym korpusie [34]. Metoda jest obliczeniowo wydajna i skuteczna przy precyzyjnych zapytaniach terminologicznych. Jej ograniczeniem jest brak rozumienia synonimii – dwa semantycznie tożsame pojęcia zapisane różnymi słowami nie zostaną dopasowane, jeśli nie współwystępują w tych samych dokumentach.

Wyszukiwanie gęste (ang. *dense retrieval*) reprezentuje zarówno zapytania, jak i dokumenty jako wektory w przestrzeni osadzeń, gdzie teksty o podobnym znaczeniu leżą blisko siebie niezależnie od użytych słów. Do popularnych modeli osadzających należą Sentence-BERT [32] oraz gęste wyszukiwanie fragmentów (ang. *Dense Passage Retrieval*, DPR) [35]. Słabością jest wrażliwość na jakość modelu osadzającego oraz wyższy koszt obliczeniowy indeksowania względem metod leksykalnych.

Układy hybrydowe łączą oba sygnały – leksykalny i semantyczny – scalając wyniki przez algorytmy takie jak wzajemna fuzja rankingów (ang. *Reciprocal Rank Fusion*), które wyznaczają

wynik końcowy fragmentu na podstawie jego pozycji w obu rankingach jednocześnie [36].

Po pobraniu wyników stosuje się ponowne szeregowanie (ang. *reranking*), w którym osobny model ocenia trafność każdego fragmentu względem pytania dokładniej niż standardowa wyszukiwarka. Najbardziej użyteczne fragmenty trafiają dzięki temu na początek okna kontekstowego generatora – ma to znaczenie praktyczne, ponieważ model poświęca największą uwagę właśnie pierwszym elementom kontekstu [37].

2.2.6 Metody fragmentacji dokumentów

Podział dokumentów na fragmenty (ang. *chunking*) – jednostki możliwe do zaindeksowania i wyszukania – jest jedną z kluczowych decyzji projektowych każdego systemu RAG. Zbyt mały fragment może nie zawierać wystarczającego kontekstu, zbyt duży obniża precyzję dopasowania i zajmuje nieproporcjonalnie dużą część okna kontekstowego. Barnett i in. wskazują dobór strategii fragmentacji jako jedno z siedmiu głównych wyzwań projektowych systemów RAG [38].

Najprostsza strategią jest podział na nakładające się okna o ustalonej długości (ang. *sliding window chunking*). Sąsiednie fragmenty zachowują część wspólną, co ogranicza ryzyko przecięcia zdania na granicy fragmentu. Strategia ta ignoruje jednak granice semantyczne dokumentu i może umieszczać logicznie powiązane treści w różnych fragmentach.

Fragmentacja rekurencyjna (ang. *recursive chunking*) podąża za hierarchią separatorów obecnych w dokumencie: najpierw dzieli tekst na akapity, następnie na zdania, a w ostateczności na słowa. Zachowuje to naturalne granice semantyczne, lecz skuteczność spada przy słabo ustrukturyzowanych dokumentach pozbawionych wyraźnych separatorów [38].

Fragmentacja semantyczna (ang. *semantic chunking*) wyznacza granice na podstawie spójności tematycznej, a nie długości tekstu. Model osadzający ocenia podobieństwo kolejnych zdań i tworzy nowy fragment tam, gdzie spójność spada poniżej ustalonego progu. Koszt obliczeniowy jest wyższy niż w poprzednich strategiach, ponieważ model osadzający musi być uruchomiony na etapie indeksowania [38].

Fragmentacja strukturalna (ang. *document-aware chunking*) wykorzystuje metadane dokumentu – nagłówki, listy, tabele – do wyznaczenia granic zgodnych z jego logiczną organizacją. Każdy fragment odpowiada jednej spójnej jednostce informacyjnej, lecz jakość podziału zależy bezpośrednio od możliwości parsera ekstrakcji struktury.

Tabela 2.5 zestawia omówione strategie.

Tabela 2.5 Porównanie strategii fragmentacji dokumentów

Strategia	Zalety	Wady	Typowe zastosowanie
Nakładające się okna (ang. <i>sliding window</i>)	Prosta implementacja, nakładanie ogranicza przecięcia na granicy fragmentu	Ignoruje granice semantyczne, duże fragmenty rozmywają sygnał trafności	Dokumenty jednorodne, szybkie prototypowanie
Rekurencyjna (ang. <i>recursive chunking</i>)	Zachowuje naturalne granice semantyczne dokumentu	Nierówna długość fragmentów, spada skuteczność przy słabej strukturze	Dokumenty z wyraźną hierarchią sekcji
Semantyczna (ang. <i>semantic chunking</i>)	Granice zgodne z tematyką, wysoka jakość semantyczna fragmentów	Wysoki koszt obliczeniowy, wymaga modelu osadzającego przy indeksowaniu	Dokumenty o złożonej tematyce
Strukturalna (ang. <i>document-aware chunking</i>)	Fragmenty zgodne z logiczną organizacją dokumentu	Wymaga parsera struktury, wrażliwa na jakość ekstrakcji metadanych	Dokumenty ze złożoną strukturą hierarchiczną

Źródło: opracowanie własne na podstawie [38].

2.2.7 Ewaluacja RAG

Jakość systemu RAG należy oceniać odrębnie dla etapu wyszukiwania i etapu generacji, ponieważ błąd może wystąpić w każdym z nich niezależnie. System może pobierać trafne fragmenty i generować z nich niewierną odpowiedź, lub odwrotnie: generować poprawną odpowiedź na podstawie nietrafionych fragmentów. Bez osobnych metryk dla każdego etapu nie można ustalić, gdzie leży źródło problemu. Framework RAGAS (ang. *Retrieval-Augmented Generation Assessment*) proponuje zestaw metryk realizujący ten podział [39].

Etap generacji oceniany jest przez dwie metryki. Wierność (ang. *faithfulness*) weryfikuje, czy każde twierdzenie zawarte w odpowiedzi można wywieść z pobranego kontekstu. Operacyjnie realizuje się to przez dekompozycję odpowiedzi na atomiczne zdania, a następnie sprawdzenie, czy każde z nich wynika z co najmniej jednego pobranego fragmentu. Twierdzenie niewynikające z żadnego fragmentu traktowane jest jako halucynacja – metryka przyjmuje wartość od zera do jedności, gdzie jedność oznacza pełną wierność wobec źródeł [39]. Wierność jest szczególnie istotna w dziedzinach wymagających precyzji faktograficznej, gdzie każde nieuzasadnione twierdzenie może wprowadzić użytkownika w błąd. Trafność odpowiedzi (ang. *answer relevance*) mierzy stopień, w jakim odpowiedź adresuje zadane pytanie. Metryka penalizuje odpowiedzi

niekompletne – pomijające część pytania – oraz odpowiedzi zawierające informacje zbędne, niezwiązane z intencją zapytania. Operacyjnie obliczana jest przez generowanie kilku pytań z wygenerowanej odpowiedzi i pomiar ich podobieństwa semantycznego do oryginalnego pytania [39].

Etap wyszukiwania oceniany jest przez metryki wywodzące się z klasycznego wyszukiwania informacji (ang. *information retrieval*). RAGAS adaptuje je do kontekstu RAG, traktując pobrane fragmenty jako wyniki klasyfikacji binarnej – trafny lub nietrafny. Precyzja kontekstu (ang. *context precision*) określa, jaka część pobranych fragmentów zawierała informacje faktycznie przydatne do udzielenia odpowiedzi. Wysoka precyzja oznacza, że system nie zużywa przestrzeni okna kontekstowego na fragmenty nieistotne – co jest szczególnie ważne przy ograniczonej pojemności kontekstu modelu. Pełność kontekstu (ang. *context recall*) ocenia natomiast, czy pobrane materiały pokrywają wszystkie informacje niezbędne do poprawnej odpowiedzi. Niska pełność oznacza, że system pominął fragmenty kluczowe dla pytania – nawet jeśli model językowy doskonale syntetyzuje dostarczone informacje, nie będzie w stanie udzielić poprawnej odpowiedzi bez właściwych danych wejściowych [39]. Formalnie obie metryki wyrażają się wzorami:

$$\text{Precyzja} = \frac{TP}{TP + FP} \quad (2.1)$$

$$\text{Pełność} = \frac{TP}{TP + FN} \quad (2.2)$$

$$F_1 = 2 \cdot \frac{\text{Precyzja} \cdot \text{Pełność}}{\text{Precyzja} + \text{Pełność}} \quad (2.3)$$

gdzie TP oznacza fragmenty trafnie zaklasyfikowane jako relewantne, FP fragmenty błędnie uznane za relewantne, a FN fragmenty relewantne pominięte przez system wyszukiwania. Miara F_1 łączy obie perspektywy w jeden wynik, co jest przydatne przy porównywaniu systemów o różnych profilach precyzja–pełność.

Uzupełnieniem powyższych metryk jest zwięzłość odpowiedzi (ang. *answer conciseness*), oceniająca czy odpowiedź nie zawiera treści nadmiarowych względem pytania. Metryka ta jest szczególnie istotna w systemach odpowiadających na pytania na podstawie dokumentów – odpowiedź przytaczająca wiele fragmentów bez wskazania właściwego może być równie problematyczna jak odpowiedź błędna, ponieważ przenosi ciężar interpretacji na użytkownika końcowego [39].

Tabela 2.6 zestawia omówione metryki wraz z opisem mierzonego wymiaru i metodą pomiaru.

Tabela 2.6 Porównanie metryk ewaluacji systemów RAG

Etap	Metryka	Co mierzy	Metoda pomiaru	Źródło
Generacja	Wierność (ang. <i>faithfulness</i>)	Czy każde twierdzenie w odpowiedzi wynika z pobranych fragmentów; twierdzenie bez pokrycia w źródłach traktowane jako halucynacja	Dekompozycja odpowiedzi na zdania atomiczne i weryfikacja wyników z kontekstu przez model językowy	[39]
	Trafność odpowiedzi (ang. <i>answer relevance</i>)	Stopień w jakim odpowiedź adresuje zadane pytanie; penalizuje niekompletność i treści zbędne	Generowanie pytań z odpowiedzi i pomiar podobieństwa semantycznego do oryginalnego pytania	[39]
Wyszukiwanie	Precyzja kontekstu (ang. <i>context precision</i>)	Udział fragmentów relevantnych wśród wszystkich pobranych; wysoka precyzja minimalizuje szum informacyjny w kontekście	$TP / (TP + FP)$ na podstawie oceny relewantności każdego pobranego fragmentu	[39]
	Pełność kontekstu (ang. <i>context recall</i>)	Pokrycie wszystkich informacji niezbędnych do odpowiedzi; niska pełność oznacza pominięcie kluczowych fragmentów	$TP / (TP + FN)$ na podstawie oceny relewantności każdego pobranego fragmentu	[39]
	Zwiężłość odpowiedzi (ang. <i>answer conciseness</i>)	Brak treści nadmiarowych względem pytania; nadmiar informacji przenosi ciężar interpretacji na użytkownika	Ocena przez LLM-sędziego względem pytania i odpowiedzi	[39]

Źródło: opracowanie własne na podstawie [39].

2.2.8 Agentowy RAG

Naturalną ewolucją zaawansowanego RAG jest jego integracja z architekturą agentową – podejście określane w literaturze jako *agentic RAG* [5]. W klasycznym potoku RAG sekwencja kroków jest z góry ustalona: pobranie fragmentów, a następnie generacja odpowiedzi. W *agentic RAG* decyzja o tym, kiedy i czego szukać, jest podejmowana dynamicznie przez model w trakcie wnioskowania. Agent może pominąć wyszukiwanie dla prostych pytań faktograficznych, powtórzyć je z przeformułowanym zapytaniem po ocenie pierwszych wyników lub równolegle odpytać kilka specjalistycznych indeksów i dokonać syntezy uzyskanych fragmentów.

Różnica między zaawansowanym RAG a *agentic RAG* ma charakter jakościowy, nie jedynie ilościowy. Zaawansowany RAG rozbudowuje stały potok o dodatkowe operacje – reranking, przeformułowanie zapytań, iteracyjne wyszukiwanie. *Agentic RAG* zastępuje stały potok pętlą decyzyjną, w której każdy krok stanowi odpowiedź na bieżący stan zadania, a nie z góry zaplanowaną operację [9].

W literaturze wyróżnia się kilka konkretnych podejść do sterowania procesem wyszukiwania, od prostych po wysoce autonomiczne. Pierwszym jest wyszukiwanie iteracyjne (ang. *iterative retrieval*), w którym agent nie poprzestaje na jednym zapytaniu, lecz formułuje kolejne na podstawie wcześniejszych wyników. Podejście to jest szczególnie użyteczne w pytaniach wieloskokowych (ang. *multi-hop*), gdzie informacja X jest potrzebna po to, aby w ogóle sformułować pytanie o informację Y . Trivedi i in. zaproponowali IRCot (Interleaving Retrieval with Chain-of-Thought), które przeplata kroki rozumowania z kolejnymi wyszukiwaniami – każdy krok wnioskowania może uruchomić nowe zapytanie do bazy wektorowej [40].

Podejściem pośrednim jest HyDE (ang. *Hypothetical Document Embeddings*), które wykorzystuje model językowy do wygenerowania hipotetycznej odpowiedzi na pytanie, a następnie używa tej odpowiedzi jako zapytania do bazy wektorowej [27]. Zakłada się, że hipotetyczna odpowiedź wykazuje większe podobieństwo stylistyczne i terminologiczne do fragmentów źródłowych niż samo pytanie sformułowane przez użytkownika – co może istotnie poprawić trafność pobieranych fragmentów, szczególnie gdy pytanie i dokumenty operują różnymi rejestrami językowymi.

Bardziej autonomiczne podejście reprezentuje FLARE (ang. *Forward-Looking Active REtrieval*), w którym model sam decyduje, kiedy uruchomić dodatkowe wyszukiwanie [41]. Agent monitoruje pewność własnych generacji i inicjuje wyszukiwanie tylko wtedy, gdy rozpoznaje brak wystarczającego wsparcia dowodowego dla formułowanego fragmentu odpowiedzi. Eliminuje to niepotrzebne wywołania wyszukiwarki przy prostych pytaniach, jednocześnie zapewniając dostęp do zewnętrznych źródeł tam, gdzie jest to konieczne.

Najdalej posuniętą autonomię zapewnia Self-RAG, który wprowadza do procesu generacji specjalne tokeny refleksji (ang. *reflection tokens*) [42]. Model uczy się samodzielnie oceniać, czy pobrane fragmenty są relewantne, czy wygenerowana odpowiedź jest wierna źródłom i czy odpowiedź jest kompletna – wszystko to bez zewnętrznego modułu ewaluacyjnego. Pozwala to systemowi na adaptacyjne sterowanie zarówno wyszukiwaniem, jak i procesem generacji

w ramach jednego modelu.

Kluczową zdolnością wszystkich wariantów agentic RAG jest przeformułowanie zapytania (ang. *query rewriting*). Gdy pierwsze wyszukiwanie nie przynosi wystarczających wyników, agent może rozłożyć złożone zapytanie na podzapytania lub zastąpić termin potoczny jego odpowiednikiem terminologicznym. Zdolność do translacji między różnymi rejestrami językowymi stanowi jedną z istotnych przewag architektur z dynamicznym sterowaniem wyszukiwaniem [27].

Integracja RAG z architekturą agentową rodzi jednak nowe wyzwania. Wielokrotne wywołania modułu wyszukiwania zwiększają opóźnienie i zużycie tokenów, a każda kolejna reformulacja zapytania niesie ryzyko dryftu semantycznego – agent może stopniowo odchodzić się od oryginalnego pytania, poszukując informacji coraz bardziej od niego odległych [5]. Kontrola długości ścieżki wyszukiwania oraz mechanizmy zatrzymania stanowią zatem równie istotny element projektu architektury agentowej jak jakość samego indeksu wektorowego.

2.3. Środowisko dokumentów ubezpieczeniowych

Skuteczność systemu odpowiadania na pytania jest uwarunkowana zarówno architekturą agentową, jak i specyfiką korpusu, na którym system działa. Dokumenty prawne cechuje długość, złożona składnia i gęsta sieć odwołań krzyżowych, które systematycznie obniżają skuteczność ogólnych modeli językowych. Cechy te omówiono na przykładzie polskich dokumentów ubezpieczeniowych.

2.3.1 Przetwarzanie języka naturalnego dla dokumentów ubezpieczeniowych

Dokumenty prawne stanowią jeden z najtrudniejszych obszarów zastosowań przetwarzania języka naturalnego. Badania nad prawniczym przetwarzaniem języka naturalnego wyodrębniły kilka cech strukturalnych tekstów prawnych, które systematycznie obniżają skuteczność ogólnych modeli językowych [43], [44].

Pierwszą z nich jest długość. Dokumenty prawne często liczą dziesiątki lub setki stron, co wymusza fragmentację tekstu i wiąże się z ryzykiem utraty istotnych zależności semantycznych między odległymi fragmentami. Zbyt drobna fragmentacja pozbawia model potrzebnego kontekstu, zbyt gruba obniża precyzję dopasowania podczas wyszukiwania [38].

Drugą cechą jest złożoność składniowa i specjalistyczne słownictwo. Zdania warunkowe, wielokrotne wyłączenia i odesłania do innych artykułów tworzą sieć zależności wymagającą wieloetapowego rozumowania. Modele trenowane na ogólnych tekstach mają trudność z poprawnym rozpoznawaniem koreferencji i ról semantycznych w takim środowisku [45]. Trudności te motywowały rozwój wyspecjalizowanych modeli i zbiorów danych, takich jak LegalBERT, CUAD i ContractNLI [43], [44], [45].

Trzecią cechą są odwołania krzyżowe między sekcjami. Odpowiedź na pytanie o zakres ochrony może wymagać sięgnięcia do definicji z jednego rozdziału, klauzuli ogólnej z innego i wyłączeń z kolejnego jednocześnie. Dla systemów opartych na wyszukiwaniu oznacza to, że pojedyncze zapytanie rzadko przynosi kompletną odpowiedź – potrzeba kilku skoordynowanych

zapytań lub mechanizmu iteracyjnego wyszukiwania [43], [46].

Czwartą cechą są typowe tryby niepowodzeń systemów RAG pracujących na tekstach prawnych. Należą do nich fałszywe cytowania (ang. *spurious citations*), gdzie model przypisuje twierdzenie do fragmentu, który go nie uzasadnia, oraz pominięte cytowania (ang. *missing citations*), gdzie poprawne twierdzenia pozbawione są odniesienia do źródła. Osobnym problemem jest niezgodność granularności (ang. *granularity mismatch*) – system cytuje cały artykuł zamiast konkretnego ustępu, zmuszając użytkownika do samodzielnego przeszukania długiego fragmentu [43].

Piątą cechą jest brak publicznych zbiorów danych dla wielu języków i jurysdykcji. Większość zasobów do prawniczego przetwarzania języka naturalnego dotyczy prawa anglosaskiego, co ogranicza możliwość bezpośredniego transferu wyników na teksty polskiego prawa kontynentalnego [44].

Typowy dokument prawny – umowa, regulamin lub kodeks – składa się z hierarchicznie zagnieżdżonych jednostek redakcyjnych. Na poziomie najwyższym znajdują się artykuły lub paragrafy, które dzielą się na ustępy, te zaś na punkty i litery. Taka struktura umożliwia precyzyjne odsyłanie do konkretnych postanowień i ogranicza powtarzanie tych samych zapisów w różnych miejscach dokumentu. Jednocześnie stwarza trudności dla systemów opartych na wyszukiwaniu, ponieważ sens danej jednostki często zależy od kontekstu zdefiniowanego kilka poziomów wyżej [38].

Tabela 2.7 zestawia te jednostki wraz z typowymi kontekstami użycia i wyzwaniem, jakie stawiają systemom fragmentacji i wyszukiwania.

Tabela 2.7 Zestawienie struktur polskich tekstów prawnych

Jednostka	Skrót	Kontekst użycia	Wyzwanie dla RAG
Artykuł	art.	Ustawy, akty normatywne	Wysoki poziom ogólności; podział na artykuły jest naturalny, lecz pojedynczy artykuł może być zbyt obszerny jako fragment
Paragraf	§	Rozporządzenia, OWU	Nosi kluczową treść normatywną; powinien być zachowany jako jednostka nadrzędna przy fragmentacji
Ustęp	ust.	Podział paragrafu lub artykułu	Naturalna granica fragmentu; wyrwany z kontekstu paragrafu traci część znaczenia normatywnego
Punkt	pkt	Wyliczenia przesłanek i wyłączeń	Sens zależny od ustępu nadrzędnego; izolowany fragment może być interpretowany błędnie
Litera	lit.	Zagnieżdżone wyliczenia	Wymaga kontekstu z co najmniej dwóch poziomów wyżej; samodzielnie jest niejednoznaczna
Tiret	—	Najdrobniejsze doprecyzowania	Najwyższe ryzyko utraty znaczenia po wyizolowaniu; niemal zawsze wymaga kontekstu nadrzędnego

Źródło: opracowanie własne.

Najszerzej stosowanym punktem odniesienia dla ogólnego rozumienia tekstów prawnych jest LexGLUE, agregujący siedem zbiorów danych obejmujących orzeczenia Europejskiego Trybunału Praw Człowieka, decyzje amerykańskiego Sądu Najwyższego i regulaminy serwisów internetowych [47]. Benchmark ten ustanowił wspólny protokół ewaluacyjny, jednak jego zakres tematyczny i językowy jest wąski – wszystkie dane są anglojęzyczne, a dominuje w nich prawo precedensowe. Do analizy umów przeznaczony jest CUAD, zawierający 510 kontraktów opatrzonych adnotacjami ekspertów dla 41 kategorii klauzul [45]. ContractNLI przyjmuje inną formułę: modeluje rozumienie umów jako zadanie wnioskowania naturalnego – dla każdej hipotezy system ma określić, czy umowa ją potwierdza, wyklucza czy pozostawia nierozstrzygniętą [43]. Oba zbiory dotyczą wyłącznie anglosaskich umów handlowych i nie obejmują dokumentów konsumenckich ani produktów ubezpieczeniowych.

Wspólną cechą tych zbiorów jest koncentracja na prawie anglosaskim i systemie common law. Bezpośredni transfer modeli wytrenowanych na tych zasobach na teksty polskiego prawa

kontynentalnego jest metodologicznie wątpliwy – ze względu na odmienną strukturę przepisów, inne konwencje redakcyjne i brak pojęciowej odpowiedniości między instytucjami prawnymi obu systemów [44]. Dostępne zasoby otwartych dokumentów prawnych w języku polskim są ograniczone. Prace nad polskim NLP zaowocowały zasobami ogólnymi – PolQA [48] dla otwartego odpowiadania na pytania oraz PIRB [49] dla ewaluacji metod wyszukiwania – jednak żaden z nich nie obejmuje dokumentów prawnych o charakterze umownym.

2.3.2 Ogólne Warunki Ubezpieczenia

Ogólne Warunki Ubezpieczenia (OWU) są podstawowym dokumentem umownym określającym relację między ubezpieczycielem a ubezpieczającym. Polskie przepisy wskazują minimalne elementy, które taki dokument musi zawierać, a szersze tło regulacyjne wyznaczają ramy europejskie, w tym krajowa ustawa o działalności ubezpieczeniowej i reasekuracyjnej [50] oraz dyrektywa Solvency II [51]. Należą do nich między innymi przedmiot i zakres ochrony, wyłączenia odpowiedzialności, procedura zgłaszania szkody oraz zasady ustalania składki. W praktyce rozbudowane OWU dla produktu komunikacyjnego może liczyć od 30 do 80 stron i zawierać kilkaset numerowanych postanowień [52].

Specyfika języka OWU wykracza poza ogólną trudność tekstów prawnych. Polskie dokumenty ubezpieczeniowe łączą formalny rejestr prawny z terminologią ubezpieczeniową. Istotną cechą OWU są odwołania krzyżowe: artykuł o zakresie ochrony często odsyła jednocześnie do definicji, wyłączeń, zasad naliczania składki i procedury zgłoszenia szkody. Udzielenie odpowiedzi wymaga więc zintegrowania informacji z kilku oddzielnych części dokumentu – praktyka ta ogranicza powtarzanie tych samych zapisów, co z perspektywy systemu odpowiadania na pytania oznacza typowy problem wnioskowania wieloskokowego [43], [46].

2.3.3 Informacyjny Dokument Produktu Ubezpieczeniowego

Informacyjny Dokument Produktu Ubezpieczeniowego (IPID) jest krótszym i silnie ustandaryzowanym dokumentem wynikającym z wymogów dyrektywy o dystrybucji ubezpieczeń (ang. *Insurance Distribution Directive*, IDD) [53] oraz krajowej ustawy o dystrybucji ubezpieczeń [54]. W przeciwieństwie do OWU nie jest pełnym tekstem umowy – ma służyć konsumentowi jako krótkie podsumowanie produktu przed zakupem, obejmujące rodzaj ubezpieczenia, zakres ochrony, główne wyłączenia, zasięg terytorialny, obowiązki klienta, składkę oraz zasady zakończenia umowy. Dokument celowo ograniczono do dwóch stron, co sprawia, że stanowi zwięzłe streszczenie, a nie pełną specyfikację prawną.

IPID odpowiada głównie na pytania ogólne – o podstawowy zakres ochrony albo główne wyłączenia. OWU służy natomiast do rozstrzygania konkretnych przypadków i zawiera pełną podstawę prawną odpowiedzi. Oba dokumenty pełnią zatem różne funkcje: IPID umożliwia wstępne zapoznanie z produktem, a OWU dostarcza rozstrzygających szczegółów [53], [54].

Tabela 2.8 zestawia oba dokumenty z perspektywy ich długości, funkcji informacyjnej i implikacji dla systemu odpowiadania na pytania.

Tabela 2.8 Porównanie dokumentów OWU i IPID

Cecha		OWU	IPID	Konsekwencja dla systemu
Długość dokumentu	doku-	30–80 stron, kilka-set postanowień	2 strony, ustandaryzowany układ	OWU wymaga fragmentacji na dziesiątki segmentów; IPID może być indeksowany jako jeden fragment
Poziom szczegółowości	szczegóło-	Pełna podstawa prawna, definicje, wyjątki i procedury	Skrótowe podsumowanie kluczowych informacji	Pytania szczegółowe wymagają OWU; IPID nie zawiera wystarczającej podstawy do rozstrzygnięcia sporów
Dominujące typy pytań	typy	Złożone, wieloskokowe – zakres ochrony, wyłączenia, definicje pojęć	Ogólne – zakres ochrony, zasięg terytorialny	Zestaw testowy musi obejmować oba typy; pytania złożone testują głównie OWU
Problemy wyszukiwania	wyszuki-	Odwołania krzyżowe, ryzyko pominięcia wyłączeń	Ryzyko mylenia z podobnymi fragmentami OWU	Architektura musi łączyć fragmenty z wielu sekcji; dla IPID kluczowa precyzja identyfikacji dokumentu
Sposób fragmentacji	fragmenta-	Podział na artykuły lub okna z nakładaniem się	Jeden lub dwa fragmenty na dokument	Strategia musi być dostosowana do typu dokumentu; mieszanie klas zwiększa ryzyko błędnego pobrania

Źródło: opracowanie własne.

2.3.4 Klasyfikacja pytań

Pytania zadawane przez użytkowników dokumentów prawnych różnią się zasadniczo pod względem wymaganych procesów poznawczych – od prostego odczytu wartości po wieloetapową syntezę informacji rozproszonych w korpusie. W literaturze klasyfikuje się je według stopnia złożoności wnioskowania niezbędnego do udzielenia odpowiedzi [43], [46].

1. **Pytania faktograficzne** (ang. *factoid questions*): Wymagają zlokalizowania i odczytania konkretnej wartości lub parametru [55]. Odpowiedź pochodzi z jednego miejsca w dokumencie i nie wymaga wnioskowania wykraczającego poza ekstrakcję – przykładem jest pytanie o wysokość sumy ubezpieczenia lub długość okresu karencji.
2. **Pytania listowe** (ang. *list questions*): Polegają na identyfikacji i wyliczeniu zbioru elementów spełniających określone kryterium [55]. Wymagają przeszukania wielu fragmentów dokumentu i agregacji wyników – typowym przykładem jest pytanie o wszystkie wyłączenia odpowiedzialności dotyczące danego rodzaju szkody.
3. **Pytania definicyjne** (ang. *definition questions*): Wymagają zidentyfikowania i wyjaśnienia znaczenia pojęcia stosowanego w dokumencie [55]. Odpowiedź ma charakter opisowy i jest abstrahowana od konkretnych przypadków – przykładem jest pytanie o znaczenie terminu użytego w klauzuli ochronnej.
4. **Pytania decyzyjne i weryfikacyjne** (ang. *confirmation questions*): Wymagają głębszego wnioskowania logicznego. Nie wystarczy odnaleźć właściwy fragment – trzeba zweryfikować warunki, uwzględnić ewentualne wyłączenia i dokonać dedukcji co do tego, czy opisany stan faktyczny mieści się w zakresie ochrony [43].
5. **Pytania przyczynowo-skutkowe i proceduralne** (ang. *causal and how-to questions*): Wymagają zrozumienia sekwencji zdarzeń lub kroków proceduralnych [55]. Odpowiedź nie jest pojedynczą wartością, lecz opisem procesu – przykładem jest pytanie o sposób postępowania w celu uzyskania świadczenia.
6. **Pytania złożone wymagające wielokrotnego wnioskowania** (ang. *multi-hop questions*): Stanowią najtrudniejszą kategorię. Odpowiedź nie znajduje się w jednym miejscu – wymaga połączenia przesłanek z odrębnych sekcji lub dokumentów, przy zachowaniu porządku między źródłami [46].

Współczesne badania nad odpowiadaniem na pytania w dziedzinach specjalistycznych koncentrują się coraz bardziej na pytaniach złożonych i weryfikacyjnych, ponieważ w najszerszym stopniu odpowiadają rzeczywistym potrzebom użytkowników analizujących dokumenty prawne [43], [46].

2.4. Przegląd prac pokrewnych

Rozdział przedstawia przegląd istniejących rozwiązań z obszaru systemów agentowych i odpowiadania na pytania opartego na dokumentach, identyfikuje luki w dotychczasowych badaniach oraz wskazuje, w jaki sposób niniejsza praca je uzupełnia. Na styku tych dwóch dziedzin pojawiły się liczne prace, lecz ich ograniczeniem jest brak kontrolowanego porównania. Utrudnia to formułowanie wniosków o tym, czy złożoność orkiestracji bezpośrednio przekłada się na wyższą jakość odpowiedzi.

2.4.1 Istniejące rozwiązania

Punktem wyjścia dla większości współczesnych architektur agentowych jest ReAct, który jako pierwszy połączył ślad rozumowania z wywoływaniem narzędzi w jednym modelu, wykazując przewagę nad łańcuchem myśli na zbiorach wymagających wyszukiwania [6]. Praca ta ustanowiła wzorzec naprzemiennego rozumowania i wywoływania narzędzi, który stał się punktem odniesienia dla kolejnych badań – w tym niniejszego. Istotne ograniczenie ReAct polega na tym, że weryfikacja odbywała się na zbiorach ogólnych: Fever, HotPotQA, AlfWorld. Pytania w tych zbiorach są zazwyczaj krótkie i jednorodne strukturalnie, a dokumenty źródłowe nie zawierają sieci wzajemnych odesłań charakterystycznej dla tekstów prawnych. Nie wiadomo zatem, jak architektura zachowuje się w warunkach długich dokumentów ze złożoną hierarchią warunkowych wyłączeń.

Badania architektur wieloagentowych rozwinęły się w dwóch kierunkach. Pierwszy – reprezentowany przez AutoGen i MetaGPT – skupił się na demonstracji możliwości podziału pracy między agentami: AutoGen przez wzorce konwersacyjne [7], MetaGPT przez jawne role organizacyjne wzorowane na strukturach projektowych [8]. Oba podejścia przyniosły obiecujące wyniki w zadaniach programistycznych, lecz w obu przypadkach weryfikacja odbywała się w domenie, gdzie zadania są naturalnie dekomponowalne, a poprawność odpowiedzi można obiektywnie ocenić. Pytania o zakres ochrony ubezpieczeniowej mają inny charakter – poprawna odpowiedź wymaga połączenia wielu częściowych przesłanek, nie wykonania sekwencji autonomicznych kroków. Drugi kierunek – refleksja i samoweryfikacja – reprezentują Reflexion i Self-RAG. Reflexion wprowadza pętlę samokrytyki [17], Self-RAG uczy model rozpoznawania momentów wymagających dodatkowego wyszukiwania [42]. Oba przynoszą poprawę wierności względem prostego RAG, lecz żadne nie bada, jak ich mechanizmy samoweryfikacji wypadają wobec innych topologii w kontrolowanych warunkach. Wzorzec generator-krytyk jest zbliżony do architektury hierarchicznej – krytyk pełni rolę zbliżoną do orkiestratora weryfikującego wyniki pracowników.

Deбата wieloagentowa opisana przez Du i in. wskazuje alternatywne źródło poprawy jakości – kontrolowany spór między agentami jako mechanizm weryfikacji odpowiedzi, niezależny od jakości wyszukiwania [16]. Wariant debatowy nie został w tej pracy wdrożony ze względu na wysoki koszt tokenowy nieadekwatny do budżetu badania, ale stanowi naturalny punkt

porównania dla architektur współpracujących.

Osobną linię badań reprezentują prace wzmacniające etap wyszukiwania bez modyfikacji topologii agentowej. GraphRAG buduje graf wiedzy i zagregowane streszczenia fragmentów, poprawiając jakość syntezy dla długich dokumentów [56]. Self-RAG rozszerza to podejście, ucząc model samodzielnego rozpoznawania momentów, w których odpowiedź wymaga dodatkowego wyszukiwania lub weryfikacji [42]. Badania tego typu często raportują poprawę względem prostych systemów RAG, ale nie pozwalają jednoznacznie stwierdzić, czy wynika ona z samej topologii czy z lepszego wyszukiwania.

Jednym z najbliższych obszarów zastosowań są dokumenty finansowe. FinAgent łączy analizę tekstu, danych liczbowych i sygnałów rynkowych przez architekturę hierarchiczną [57]. Praca ta jest tematycznie najbliższa niniejszemu badaniu – dokumenty finansowe i ubezpieczeniowe wykazują podobną specjalistyczną terminologię i złożoną strukturę zależności. FinAgent nie oferuje jednak kontrolowanego porównania z układami jednoagentowymi ani nie analizuje, jak architektura zachowuje się przy różnych poziomach trudności pytań.

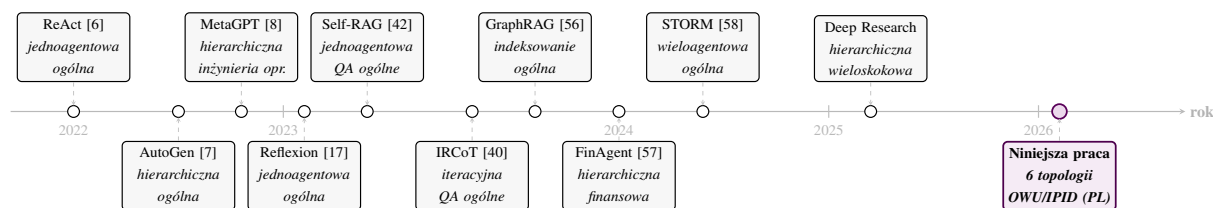
IRCoT przeplata kroki rozumowania z kolejnymi wyszukiwaniami, wykazując przewagę nad jednorazowym pobieraniem na zbiorach wieloskokowych [40]. Wyniki obu badań wskazują, że jakość etapu wyszukiwania ma decydujące znaczenie dla jakości odpowiedzi. STORM realizuje wieloagentowe generowanie artykułów przez symulację rozmów między wyspecjalizowanymi agentami, lecz jest zaprojektowany dla zadań generatywnych, nie dla precyzyjnego odpowiadania na pytania z podaniem podstawy faktycznej [58].

Innym punktem odniesienia jest Deep Research – system OpenAI zaprojektowany do wieloetapowego wyszukiwania i syntezy informacji z wielu źródeł. Architektura ta realizuje pętlę agentową zbliżoną do ReAct, uzupełnioną o mechanizmy planowania i iteracyjnej weryfikacji pobranych materiałów. Deep Research nie jest jednak systemem otwartym – nie ujawniono szczegółów implementacyjnych ani wyników na kontrolowanych zbiorach testowych, co uniemożliwia bezpośrednie porównanie z architekturami opisywanymi w literaturze akademickiej. Fakt jego wdrożenia wskazuje na rosnące zainteresowanie architekturami zdolnymi do wieloskokowego wnioskowania nad dokumentami.

Większość badań nad systemami wieloagentowymi korzysta z ogólnodomenowych zbiorów anglojęzycznych, takich jak HotPotQA [46], budowanych poza domeną regulacyjną [59]. Zbiory te dobrze testują wnioskowanie wieloskokowe, ale nie obejmują problemów typowych dla dokumentów ubezpieczeniowych – zależności między definicjami, wyłączeniami i klauzulami ochrony ani pytań porównawczych między polisami. Brak polskich zasobów ewaluacyjnych dla tej dziedziny jest jednym z bezpośrednich uzasadnień niniejszej pracy.

Rysunek 2.6 porządkuje omówione prace chronologicznie, pokazując kierunek ewolucji od ogólnych architektur jednoagentowych do coraz bardziej wyspecjalizowanych systemów dokumentowych.

Rysunek 2.6 Chronologia rozwiązań



Źródło: opracowanie własne na podstawie literatury.

2.4.2 Luki badawcze i wkład niniejszej pracy

Przegląd literatury ujawnia trzy wyraźne luki, które bezpośrednio motywują projekt niniejszego badania. Żadna z omówionych prac nie uwzględnia ich wszystkich jednocześnie – co oznacza, że mimo rozbudowanego dorobku w obszarze architektur agentowych i systemów RAG, pytanie o rzeczywisty wpływ topologii na jakość odpowiedzi w domenie regulacyjnej pozostaje otwarte.

Najistotniejszy jest brak kontrolowanych porównań wielu topologii przy stałym modelu, tych samych narzędziach i tym samym korpusie. W każdej z omówionych prac trudno oddzielić wpływ topologii od wpływu lepszego wyszukiwania, bardziej dopracowanych instrukcji systemowych lub innego modelu bazowego. Wang i in. oraz Xi i in. wskazują ten brak jako jedną z głównych przeszkód w budowaniu kumulatywnej wiedzy o architekturach agentowych [5], [9]. Nowa architektura zestawiana jest zazwyczaj z prostym modelem bazowym – nie z innymi architekturami w identycznych warunkach – co uniemożliwia odpowiedź na pytanie, czy złożoność orkiestracji bezpośrednio wpływa na wyższą jakość odpowiedzi, czy jedynie na wyższe koszty operacyjne. Problem ten jest szczególnie widoczny, gdy wyniki poszczególnych prac są ze sobą zestawiane: różnice w modelu bazowym, zestawie narzędzi czy zbiorze testowym sprawiają, że porównanie między architekturami opisywanymi w różnych publikacjach jest obarczone dużą niepewnością metodologiczną.

Drugą luką jest brak zestawu testowego dla dokumentów ubezpieczeniowych w języku polskim. Dostępne zbiory – HotPotQA [46], Wikipedia, zadania programistyczne – testują wnioskowanie wieloskokowe na tekstach ogólnych i nie obejmują problemów typowych dla tej domeny: sieci wzajemnych odesłań między definicjami a wyłączeniami, konieczności uzgadniania sprzecznych postanowień czy porównywania zapisów z wielu polis jednocześnie. Polskojęzyczne zasoby ewaluacyjne dla tej dziedziny nie istnieją, co oznacza, że wyniki uzyskane na zbiorach anglojęzycznych nie mogą być bezpośrednio przeniesione na systemy operujące na polskich dokumentach regulacyjnych – zarówno ze względu na różnice językowe, jak i odmienną strukturę prawną.

Trzecia luka dotyczy głębokości analizy wyników. Dominujące podejście polega na raportowaniu średnich wartości metryk jakościowych i operacyjnych, co pozwala co najwyżej uszeregować architektury według łącznego wyniku, lecz nie wyjaśnia mechanizmów stojących za obserwowanymi różnicami. Wiedza o tym, kiedy i dlaczego dany system zawodzi – czy traci

kontekst przy długich zadaniach, czy błędnie klasyfikuje zapytania, czy propaguje błędy między modułami – ma bezpośrednią wartość przy wyborze architektury do zastosowania produkcyjnego. Tymczasem analiza trybów niepowodzeń pozostaje w literaturze nieobecna lub ograniczona do kilku przykładów anegdotycznych.

Tabela 2.9 zestawia omówione prace względem tych trzech wymiarów. Każda ze zidentyfikowanych luk przekłada się bezpośrednio na jeden z wkładów badania: brak kontrolowanego porównania – na projekt eksperymentalny z izolowaną topologią jako jedyną zmienną niezależną, przy stałym modelu, narzędziach, korpusie i zestawie 90 pytań dla wszystkich sześciu architektur; brak polskiego zestawu testowego – na oryginalny zestaw pytań o czterech poziomach trudności oparty wyłącznie na polskich dokumentach OWU i IPID, obejmujący zarówno pytania proste jak i wymagające wieloetapowego wnioskowania między sekcjami; brak analizy błędów – na jakościowe badanie śladów wykonania, które wykracza poza agregowane metryki i pozwala zidentyfikować wzorce niepowodzeń charakterystyczne dla każdej topologii wraz z rekomendacjami praktycznymi dla projektantów systemów produkcyjnych.

Tabela 2.9 Zestawienie prac pokrewnych

System	Topologia	Dziedzina	Kontrolowane porównanie	Główna metryka	Luka względem niniejszej pracy
ReAct [6]	Jednoagentowa	Ogólna (Wikipedia)	Nie – porównanie z CoT	Dokładność zadań	Brak domeny regulacyjnej, jedna topologia
AutoGen [7]	Hierarchiczna, grupowa	Kodowanie, ogólna	Nie – brak wspólnego korpusu	Wskaźnik sukcesu zadania	Brak kontrolowanego środowiska, brak QA na dokumentach
MetaGPT [8]	Hierarchiczna	Inżynieria oprogramowania	Nie – porównanie z bazowym LLM	Poprawność kodu	Brak domeny dokumentowej, jedna topologia
FinAgent [57]	Hierarchiczna	Dokumenty finansowe	Nie – brak linii bazowej jednoagentowej	Zwrot z inwestycji	Brak kontrolowanego porównania topologii, domena finansowa
Self-RAG [42]	Jednoagentowa	Ogólna (QA)	Nie – porównanie z RAG	Wierność, dokładność	Jedna topologia, brak domeny regulacyjnej
Reflexion [17]	Jednoagentowa	Ogólna	Nie – jedna architektura	Wskaźnik sukcesu	Brak porównania topologii, brak dokumentów prawnych
GraphRAG [56]	Wzbogacone indeksowanie	Ogólna (teksty długie)	Nie – porównanie z RAG	Jakość odpowiedzi	Brak porównania topologii agentowych, brak domeny ubezpieczeniowej
Niniejsza praca	Sześć topologii	Polskie OWU/IPID	Tak – stały model, korpus, metryki	Wierność, poprawność, opóźnienie, tokeny	Kontrolowane porównanie, polska domena ubezpieczeniowa, analiza trybów niepowodzeń, rekomendacje produkcyjne

Źródło: opracowanie własne na podstawie literatury.

3. Metodologia

W rozdziale opisano podejście badawcze przyjęte w niniejszej pracy. Opis obejmuje wybór metody badawczej, strukturę eksperymentu, materiał badawczy, kryteria oceny, procedurę analizy wyników oraz zasady odtwarzalności. Rozdział przedstawia sposób przełożenia założeń teoretycznych na kontrolowany plan badawczy.

3.1. Metoda badawcza

Główny problem badawczy dotyczy wpływu topologii systemu agentowego na jakość odpowiedzi generowanych na podstawie dokumentów regulacyjnych. Jako podstawową metodę badawczą przyjęto kontrolowany eksperyment porównawczy, w którym warianty systemu przetwarzają ten sam korpus, odpowiadają na identyczny zestaw pytań i podlegały ujednoliconej procedurze oceny.

Wybór metody pozwala na izolację sposobu orkiestracji agentów jako jedynej badanej zmiennej. Zastosowanie odmiennych modeli, indeksów dokumentów lub zestawów narzędzi uniemożliwiłoby przypisanie różnic w wynikach wyłącznie testowanej architekturze. Układ kontrolowany eliminuje wpływ czynników pobocznych.

Alternatywne metody badawcze przedstawiono w tabeli 3.1. Studium przypadku pojedynczej architektury uniemożliwia ocenę porównawczą. Analiza teoretyczna pomija wpływ specyfiki korpusu i parametrów sterowania na zachowanie modeli językowych. Z kolei ankieta ekspercka mierzy subiektywne oczekiwania, a nie rzeczywiste parametry działania systemów w ustandaryzowanych warunkach.

Tabela 3.1 Porównanie metod badawczych

Podejście	Przydatność	Ograniczenie
Studium przypadku	Pozwala szczegółowo opisać jedną architekturę	Nie daje podstaw do porównania topologii
Analiza teoretyczna	Ułatwia uporządkowanie pojęć i oczekiwanych zależności	Nie uwzględnia zachowania modelu na konkretnym korpusie
Ankieta ekspercka	Może zebrać opinie praktyków lub użytkowników domenowych	Mierzy ocenę subiektywną, a nie przebieg systemu
Eksperyment porównawczy	Umożliwia pomiar jakości i kosztu w tych samych warunkach	Wymaga ścisłej kontroli konfiguracji i jednolitego protokołu oceny

Źródło: opracowanie własne.

3.2. Projekt eksperymentu

Eksperyment polega na porównaniu sześciu topologii agentowych w identycznym środowisku zadaniowym. Zmienną różnicującą jest wyłącznie organizacja pracy systemu, w tym kolejność kroków, zakres autonomii, podział zadań oraz liczba wywołań narzędzi.

3.2.1 Zmienna niezależna

Zmienną niezależną jest topologia orkiestracji systemu agentowego. Pojęcie to oznacza sposób organizacji przepływu informacji, obejmujący m.in. przetwarzanie sekwencyjne, dekompozycję problemu na podzadania, planowanie działań, wybór specjalisty na podstawie klasyfikacji pytania oraz iteracyjną modyfikację kroków w zależności od wyników pośrednich.

Do eksperymentu wybrano sześć topologii: deterministyczną, planer–wykonawca, router–specjalista, tablicową, hierarchiczną oraz ReAct. Zestaw dobrano tak, aby obejmował zarówno architektury jednoagentowe, jak i wieloagentowe, a także różne poziomy autonomii sterowania. Każda architektura musiała pozwalać na implementację w jednolitym środowisku bez wprowadzania dodatkowych, specyficznych narzędzi. Umożliwiło to analizę wpływu orkiestracji na jakość odpowiedzi.

Tabela 3.2 porównuje oba typy architektur w kluczowych wymiarach projektowych, stanowiąc punkt odniesienia dla interpretacji wyników eksperymentu w rozdziale 5.

Tabela 3.2 Porównanie architektur agentowych

Wymiar	Jednoagentowe	Wieloagentowe
Złożoność wdrożenia	Niska – jeden kontekst, jedna instrukcja systemowa, jeden punkt awarii	Wysoka – wiele instrukcji systemowych, mechanizmy koordynacji, wiele potencjalnych punktów awarii
Koszt operacyjny	Niski – jedno lub kilka wywołań LLM na pytanie	Wysoki – każdy agent generował osobne wywołania; narzut koordynacji zwiększał zużycie tokenów [5]
Zarządzanie kontekstem	Scentralizowane – wszystkie informacje w jednym buforze kontekstu; ryzyko przekroczenia limitu przy złożonych zadaniach [6]	Rozproszone – agenci wykorzystywali rozdzielone podzbiory informacji; ryzyko utraty danych podczas ich przekazywania [7]
Odporność na błąd	Błąd w jednym kroku mógł być skorygowany w kolejnej iteracji pętli	Błąd w węźle nadrzędnym propagował się kaskadowo do kolejnych agentów [5]
Specjalizacja ról	Brak – jeden agent obsługiwał całe zadanie	Możliwa – każdy agent mógł mieć wyspecjalizowaną instrukcję, narzędzia i filtry metadanych [8]
Skalowalność zadań	Ograniczona rozmiarem okna kontekstowego i pojemnością pojedynczego modelu	Wyższa – dekompozycja zadania pozwalała przetwarzać fragmenty równolegle [7]
Przewidywalność	Wysoka – deterministyczny przepływ sterowania zapewnia powtarzalność wyników	Niższa – złożoność koordynacji obniża powtarzalność odpowiedzi dla identycznych zapytań [5]
Typowe zastosowanie	Pytania wymagające jednego źródła wiedzy, systemy z wymogiem niskich opóźnień	Zadania wymagające specjalizacji dziedzinowej, złożone przepływy z wieloma krokami

Źródło: opracowanie własne na podstawie literatury.

3.2.2 Zmienne zależne

Zmienne zależne obejmują dwa wymiary: jakość odpowiedzi oraz koszt operacyjny. Ich rozdzielenie wynika z faktu, że jakość osiągnięta przy dużej liczbie wywołań modelu stanowi odmienną wartość aplikacyjną niż jakość uzyskana w prostszym przebiegu.

Do wymiaru jakości należały: wierność względem pobranych źródeł, poprawność względem odpowiedzi referencyjnej, zwięzłość oraz trafność pobranych dokumentów. Do wymiaru operacyjnego należały: opóźnienie, zużycie tokenów, liczba wywołań narzędzi, liczba iteracji oraz wskaźnik błędów wykonania. Dobór tych zmiennych wynikał z metryk omówionych w części teoretycznej, zwłaszcza w kontekście ewaluacji systemów agentowych i RAG.

3.2.3 Warunki kontrolowane

Utrzymano stałe: model językowy, parametry generacji, korpus dokumentów, indeks wektorowy, zestaw narzędzi, zbiór pytań, format odpowiedzi oraz procedurę ewaluacji. Jest to warunek konieczny do zapewnienia porównywalności wyników i eliminacji wpływu środowiska testowego na działanie systemu.

Temperaturę generacji ustawiono na zero. Zabieg ten minimalizuje losowość odpowiedzi modelu i wariancję między przebiegami, co pozwala przypisać zaobserwowane różnice bezpośrednio badanym mechanizmom sterowania, a nie zmienności LLM.

Zestawienie warunków kontrolowanych przedstawia tabela 3.3.

Tabela 3.3 Zestawienie warunków kontrolowanych

Element	Znaczenie dla porównania
Model i parametry	Eliminowały wpływ różnej jakości modeli oraz różnej losowości odpowiedzi
Korpus dokumentów	Zapewniał wspólną podstawę wiedzy dla wszystkich architektur
Indeks wektorowy	Utrzymywał jednakowy punkt wyjścia dla etapu wyszukiwania
Zestaw narzędzi	Dawał każdej topologii ten sam zakres możliwych działań
Zestaw pytań	Pozwalał porównywać odpowiedzi na identycznych przypadkach testowych
Protokół ewaluacji	Ujednolicał sposób oceny odpowiedzi i śladów wykonania

Źródło: opracowanie własne.

3.2.4 Plan porównawczy

Przyjęto pełny plan porównawczy w układzie zapytanie–architektura. Każde z 90 pytań zostało obsłużone przez wszystkie sześć topologii, generując łącznie 540 uruchomień. Taki model wewnątrzgrupowy eliminuje wpływ zróżnicowanej trudności zapytań na wyniki poszczególnych topologii.

Analiza zagregowana definiuje ogólny profil architektury, natomiast analiza przekrojowa (według poziomu trudności, ubezpieczyciela i kategorii produktu) weryfikuje stabilność wyników poszczególnych wariantów przy zmianie warunków wejściowych.

3.3. Materiał badawczy

Materiał badawczy składał się z dwóch części: korpusu dokumentów oraz zestawu pytań testowych. Korpus stanowił bazę wiedzy dla testowanych systemów, natomiast zbiór pytań służył do ilościowego pomiaru jakości generowanych odpowiedzi. Z uwagi na brak publicznego benchmarku dla polskojęzycznych dokumentów ubezpieczeniowych, oba komponenty opracowano autorsko.

Korpus obejmował 22 dokumenty ubezpieczeniowe pochodzące od trzech głównych towarzystw ubezpieczeniowych: STU ERGO Hestia S.A., Powszechnego Zakładu Ubezpieczeń S.A. oraz TUiR Warta S.A. Dla każdego ubezpieczyciela uwzględniono trzy kategorie produktowe: ubezpieczenia komunikacyjne, mieszkaniowe i podróżne. W korpusie znalazły się dwa typy dokumentów: OWU oraz IPID.

Dokumenty ubezpieczeniowe charakteryzują się cechami stanowiącymi wyzwanie dla architektur RAG i systemów agentowych: dużą objętością, sformalizowaną strukturą, obecnością odniesień krzyżowych oraz wymogiem ścisłego rozróżniania zakresu ochrony, warunków i wyłączeń. Złożoność korpusu pozwala na wyraźniejsze uwidocznienie różnic w skuteczności poszczególnych topologii względem prostych zbiorów testowych.

Tabela 3.4 przedstawia skład korpusu według ubezpieczycieli.

Tabela 3.4 Zestawienie korpusu dokumentów

Ubezpieczyciel	Kategorie	OWU	IPID
Hestia	3	3	3
PZU	3	3	3
Warta	3	5	5
Razem	9	11	11

Źródło: opracowanie własne.

Zestaw testowy opracowany na podstawie analizy manualnej dokumentów obejmuje 90 pytań. Do każdego przypisano odpowiedź referencyjną oraz uzasadniający fragment źródłowy. Umożliwia to zautomatyzowaną ewaluację zgodności wyników oraz ręczną weryfikację przypadków brzegowych.

Zdefiniowano cztery poziomy trudności. Poziom bardzo prosty testuje podstawową orientację w specyfikacji produktu. Poziom prosty wymaga zlokalizowania pojedynczej informacji bazowej. Poziom złożony narzuca konieczność integracji warunku, wyjątku lub uzupełniającego fragmentu

kontekstu. Z kolei pytania bardzo złożone wymuszają syntezę danych z wielu nieciągłych fragmentów dokumentu w celu zbudowania pełnej odpowiedzi.

Zestaw zbalansowano według ubezpieczycieli (po 30 pytań). Trzy główne poziomy trudności zawierają po 27 pytań, a poziom bardzo prosty – 9 pytań. Taka dystrybucja eliminuje ryzyko obciążenia wyników specyfiką pojedynczego dostawcy lub typu produktu. Szczegółowy rozkład pytań przedstawia tabela 3.5.

Tabela 3.5 Zestawienie rozkładu pytań

Poziom trudności	Hestia	PZU	Warta	Razem
Bardzo proste	3	3	3	9
Proste	9	9	9	27
Złożone	9	9	9	27
Bardzo złożone	9	9	9	27
Razem	30	30	30	90

Źródło: opracowanie własne.

Przykłady pytań z każdego poziomu przedstawia tabela 3.6. Ilustruje ona różnice w oczekiwanym nakładzie pracy systemu dla każdej kategorii pytań: od krótkiej odpowiedzi informacyjnej po odpowiedź wymagającą uwzględnienia warunku i wyjątku.

Tabela 3.6 Zestawienie przykładowych pytań

Poziom	Pytanie	Odpowiedź referencyjna
Bardzo prosty	Na jaki okres zawierana jest standardowo umowa Autocasco z WARTA?	Standardowo umowy AC zawiera się na 12 miesięcy.
Prosty	Na jaki okres może zostać zawarte ubezpieczenie PZU Wojażer?	Umowa może zostać zawarta na czas oznaczony od 1 dnia do 1 roku.
Złożony	Czy PZU Wojażer obejmuje koszty leczenia spowodowane epidemią?	Co do zasady nie, z wyjątkiem kosztów dotyczących nagłego zachorowania na COVID-19, kwarantanny lub izolacji.
Bardzo złożony	Czy w ubezpieczeniu PZU Wojażer objęte są ochroną koszty leczenia chorób przewlekłych?	Z ochrony wyłączone są koszty leczenia chorób przewlekłych, z wyjątkiem zaostrzeń albo powikłań, a za opłatą dodatkowej składki ochrona może zostać rozszerzona do 100% sumy ubezpieczenia.

Źródło: opracowanie własne.

3.4. Kryteria oceny

Kryteria oceny zdefiniowano tak, aby uwzględniały zarówno odpowiedź końcową, jak i ślad jej generowania. W systemach agentowych analiza samego wyjścia jest niewystarczająca do diagnostyki działania. Przykładowo, dwie architektury mogą wygenerować poprawną odpowiedź, jednak jedna z nich uzyska ją optymalną ścieżką wywołań, a druga wykona operacje nadmiarowe. Z kolei niska jakość odpowiedzi może wynikać m.in. z błędów wyszukiwania, logiki sterującej lub wadliwej syntezy informacji.

3.4.1 Miary ilościowe

Miary ilościowe dobrano spośród metryk omówionych w rozdziale teoretycznym. Objęły one dwa obszary: jakość odpowiedzi i koszt operacyjny. W obszarze jakości zastosowano wierność, poprawność względem odpowiedzi referencyjnej, zwięzłość oraz trafność dokumentów. W obszarze kosztu uwzględniono opóźnienie, zużycie tokenów, liczbę wywołań narzędzi, liczbę iteracji oraz błędy wykonania.

Wierność określa stopień pokrycia odpowiedzi pobranymi dokumentami, co jest kluczowe dla wykrywania halucynacji w dziedzinie ubezpieczeniowej. Poprawność względem referencji mierzy trafność merytoryczną niezależnie od formalnej zgodności z kontekstem. Zwięzłość

weryfikuje użyteczność formatu wyjściowego, a trafność dokumentów diagnozuje skuteczność etapu wyszukiwania.

Miary operacyjne charakteryzowały praktyczny koszt działania architektury. Zużycie tokenów przekładało się bezpośrednio na koszt wywołań modelu, opóźnienie na czas obsługi zapytania, a liczba narzędzi i iteracji na złożoność przebiegu. Wskaźnik błędów wykonania określa niezawodność: wysoka średnia jakość odpowiedzi traci znaczenie praktyczne w przypadku wysokiej awaryjności działania.

Tabela 3.7 przedstawia zestawienie wszystkich zastosowanych miar, ich przedmiot oceny oraz uzasadnienie doboru.

Tabela 3.7 Zestawienie metryk oceny

Wymiar	Miara	Co oceniała	Dlaczego istotna
Jakość	Wierność	Czy twierdzenia w odpowiedzi miały oparcie w dokumentach	Wykrywała halucynacje – kluczowe w domenę regulacyjnej
	Poprawność	Zgodność z odpowiedzią referencyjną	Oceniała trafność merytoryczną niezależnie od stylu
	Zwięzłość	Adekwatność długości do pytania	Zbyt rozbudowane odpowiedzi obniżały użyteczność
	Trafność dokumentów	Czy pobrane fragmenty były istotne	Lokalizowała błąd na etapie wyszukiwania
Koszt	Zużycie tokenów	Łączna liczba tokenów wejściowych i wyjściowych	Bezpośredni wskaźnik kosztu wywołań API
	Liczba wywołań narzędzi	Ile razy architektura odwoływała się do narzędzi	Różnicowała architektury o stałej i zmiennej liczbie kroków
	Opóźnienie	Czas od pytania do odpowiedzi końcowej	Istotne dla zastosowań produkcyjnych
	Wskaźnik błędów	Odsetek uruchomień zakończonych błędem wykonania	Mierzył niezawodność architektury

Źródło: opracowanie własne.

3.4.2 Miary jakościowe

Ocenę ilościową uzupełniono analizą jakościową przebiegów. Jej celem było ustalenie, w jaki sposób dana architektura dochodziła do odpowiedzi i w którym miejscu najczęściej dochodziło do spadku jakości. Analizie podlegały pytanie, pobrane fragmenty, kolejność wywołań narzędzi, ewentualne decyzje sterujące, odpowiedź końcowa oraz odpowiedź referencyjna.

Analiza jakościowa identyfikuje mocne strony i ograniczenia topologii niewidoczne w wartościach uśrednionych. Przykładowo, architektura o wysokich wynikach średnich może generować stały narzut obliczeniowy dla prostych pytań, a topologia szybka – wykazywać niestabilność w obsłudze zapytań złożonych.

Niepowodzenia klasyfikowano według etapu, na którym powstał problem. Tabela 3.8 przedstawia przyjętą taksonomię. Kategorie miały charakter analityczny: pojedynczy przebieg mógł zawierać więcej niż jeden problem, jednak klasyfikacja według dominującej przyczyny ułatwiała porównanie architektur.

Tabela 3.8 Zestawienie kategorii błędów

Kategoria	Opis
Błąd wyszukiwania	Pobrane fragmenty nie zawierały informacji potrzebnych do odpowiedzi
Błąd rozumowania	Dowody były dostępne, ale system wyciągał z nich niepoprawny wniosek
Błąd sterowania	Topologia wybrała niewłaściwą ścieżkę, rolę, plan albo moment zakończenia
Błąd syntezy	Odpowiedź końcowa zniekształcała poprawnie pobrane informacje
Nadmiarowy przebieg	System wykonywał zbędne kroki, które podnosiły koszt bez poprawy jakości

Źródło: opracowanie własne.

3.4.3 Protokół ewaluacji

Ewaluację odpowiedzi prowadzono dwutorowo. Pierwszy tor stanowiła ocena automatyczna z wykorzystaniem modelu językowego w roli sędziego. Model oceniał odpowiedzi według zdefiniowanych kryteriów i zwracał wyniki dla metryk jakości. Drugi tor stanowiła kontrola autorska, obejmująca przegląd wyników, analizę przypadków trudnych oraz klasyfikację typów niepowodzeń.

Ewaluacja LLM-as-a-judge automatyzuje pomiar i skaluje go do 540 uruchomień, zapewniając jednocześnie determinizm stosowania kryteriów wobec badanych architektur. Wyniki te służą wyłącznie jako względne porównanie systemów w ramach oceny algorytmicznej, nie zaś absolutna weryfikacja poprawności interpretacji prawnej dokumentów.

W toku analizy uwzględniono ograniczenia ewaluacji automatycznej: preferencję wobec długości tekstu, podatność na styl bazowy modelu oraz niedoskonałą detekcję niuansów dziedzi-

nowych. Wobec tego metryki ilościowe interpretowano łącznie z jakościową analizą konkretnych przebiegów wykonania systemu.

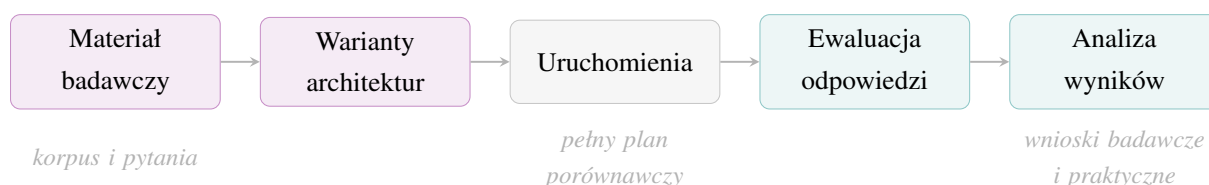
3.5. Procedura badawcza

Procedura badawcza definiuje strukturę eksperymentu od preparacji danych do syntezy wniosków. Kluczowym założeniem jest izolacja etapów generacji, oceny oraz analizy w celu wyeliminowania sprzężeń zwrotnych.

3.5.1 Etapy eksperymentu

Proces eksperymentalny obejmuje pięć faz. W pierwszej przygotowano materiał badawczy. W drugiej zdefiniowano warianty architektur. Trzecia faza to uruchomienie pełnego planu zapytanie–architektura. Faza czwarta obejmuje ewaluację. Piąta faza to wielowymiarowa analiza wyników (ilościowa, jakościowa oraz produkcyjna).

Rysunek 3.1 Etapy eksperymentu



Źródło: opracowanie własne.

Izolacja etapu ewaluacji zapobiega przekazywaniu informacji zwrotnej do architektur generujących. Gwarantuje to, że każdy zarejestrowany przebieg jest niezależnym i obiektywnym wynikiem działania badanej topologii na przypisanym pytaniu.

3.5.2 Analiza ilościowa

Analiza ilościowa realizowana jest dwupoziomowo. Poziom zagregowany obejmuje wskaźniki uśrednione: jakość, koszt, opóźnienie, zużycie tokenów i liczbę błędów, definiujące ogólny profil efektywności każdej konfiguracji.

Poziom przekrojowy analizuje wyniki m.in. w zależności od stopnia trudności zapytania. Pozwala to zweryfikować korelację między złożonością topologii a jej skutecznością w zadaniach wieloetapowych. Analiza przekrojowa eliminuje ponadto ryzyko zaburzenia wyników przez lokalne ekstrema trudności w poszczególnych fragmentach korpusu.

Wnioski ilościowe są ograniczone do testowanej specyfiki i nie stanowią uniwersalnego benchmarku systemów agentowych. Ich zasadniczym celem jest zmapowanie relacji między jakością a kosztem na poziomie poszczególnych topologii.

3.5.3 Analiza jakościowa

Analiza jakościowa koncentruje się na wybranych, brzegowych śladach wykonania: przypadkach o skrajnie niskiej ocenie, przerwaniach z błędem oraz anomaliach kosztowych. W celu identyfikacji przyczyn niepowodzenia odtwarza się pełny ciąg zdarzeń: od rozpoznania intencji do syntezy końcowej.

Badanie to ma na celu identyfikację wzorców błędów. Przykładowo, w architekturze router-specjalista weryfikuje się trafność klasyfikacji zapytania; dla wariantów iteracyjnych – celowość powtarzania kroków; dla topologii zdecentralizowanych – stabilność utrzymania danych w toku przepływu informacji.

Wyniki oceny jakościowej stanowią niezbędny kontekst dla metryk ilościowych, umożliwiając odróżnienie architektur o zbliżonej średniej, lecz odmiennych klasach dominujących awarii.

3.5.4 Perspektywa produkcyjna

Ostatni wymiar analizy przekłada wyniki z warunków kontrolowanych na środowisko produkcyjne. Zdefiniowano w nim granice uzasadnionej złożoności orkiestracji oraz wskaźniki stabilności wdrożeniowej.

Wdrożenie komercyjne wymaga uwzględnienia przewidywalności, kosztu obsługi i ciągłości działania. Wobec tego wnioski z eksperymentu przyjmują formę rekomendacji wielokryterialnych, dostosowujących wybór architektury do wymagań wierności, budżetu operacyjnego oraz przewidywanego stopnia złożoności systemu produkcyjnego.

3.6. Odtwarzalność

Odtwarzalność jest jednym z warunków wiarygodności eksperymentu. W niniejszej pracy miała ona istotne znaczenie, ponieważ badanie dotyczyło systemów opartych na dużych modelach językowych, których zachowanie zależy od konfiguracji modelu, wersji usług, sposobu przygotowania danych oraz treści promptów.

Odtwarzalność środowiska badawczego zabezpieczono przez pełną archiwizację kodu, konfiguracji LLM, korpusów oraz logów ewaluacyjnych w postaci zwartego zestawu artefaktów. Takie ujęcie ustanawia referencyjny punkt odniesienia do powtórnej weryfikacji obliczeń.

Z racji zależności od zewnętrznych platform udostępniających modele przez API, absolutna niezmiennosc wyników na poziomie generacji tokenów nie jest gwarantowana. Jako środki zaradcze wprowadzono logowanie parametrów i wersji modeli w repozytorium oraz udostępniono pełny zapis przebiegów, na których bezpośrednio oparto wnioski badawcze.

4. Implementacja

Rozdział opisuje platformę eksperymentalną zbudowaną na potrzeby porównania sześciu architektur agentowych w zadaniu odpowiadania na pytania dotyczące polskich dokumentów ubezpieczeniowych. W rozdziale omówiono stos technologiczny, podział kodu na moduły, komponenty współdzielone przez wszystkie warianty, implementację promptów, narzędzi agentowych i grafów sterowania, potok przetwarzania dokumentów, infrastrukturę ewaluacyjną oraz wybrane decyzje inżynierskie.

Kompletny kod źródłowy systemu, w tym konfiguracja środowiska i skrypty eksperymentów, udostępniono w publicznym repozytorium GitHub pod adresem <https://github.com/mchojna/master-thesis-pjatk>.

Implementację oparto na założeniu metodologicznym, w którym architektury różnią się wyłącznie sposobem sterowania przebiegiem zadania. Dokumenty, indeks wektorowy, zestaw narzędzi, konfiguracja modeli, format odpowiedzi i procedura ewaluacji są wspólne dla wszystkich wariantów. Podejście to determinuje strukturę kodu, wymuszając wyraźną granicę między warstwą sterowania a warstwą operacyjną.

4.1. Stos technologiczny

System zaimplementowano w języku Python w wersji 3.13. Warstwę orkiestracji oparto na bibliotece LangGraph, umożliwiającej definiowanie grafów stanowych z węzłami asynchronicznymi, krawędziami warunkowymi i kontrolowanymi pętlami. Właściwości te są niezbędne do odwzorowania różnic między topologiami. Dostęp do modeli językowych i modelu osadzeń zapewnia LangChain wraz z pakietem langchain-openai. Fragmenty dokumentów przechowywane są w bazie wektorowej Qdrant, a warstwa ewaluacji i rejestracji przebiegów korzysta z Arize Phoenix, PostgreSQL oraz OpenTelemetry.

Tabela 4.1 pokazuje komponenty stosu wraz z wersjami i rolami w systemie.

Tabela 4.1 Stos technologiczny

Warstwa	Komponent	Wersja	Rola
Środowisko uruchomieniowe	Python	≥ 3.13	Język implementacji
Orkiestracja agentów	LangGraph	$\geq 0.2.0$	Grafy stanowe architektur agentowych
Warstwa modeli językowych	LangChain, langchain-openai	$\geq 1.2.10$	Klienty modeli i osadzeń
	OpenAI API	$\geq 2.21.0$	Wywołania gpt-5, text-embedding-3-large
	Pydantic	$\geq 2.12.5$	Schematy wyjść strukturalnych
Warstwa wyszukiwania	Qdrant	1.15	Baza wektorowa i filtry metadanych
Warstwa ewaluacji	Arize Phoenix	13	Ewaluatory LLM-as-a-judge i panel śledzenia
	PostgreSQL	14	Baza operacyjna usługi Phoenix
Przetwarzanie dokumentów	pypdfium2, Pillow	$\geq 5.4.0$	Renderowanie stron PDF do obrazów PNG
	langchain-text-splitters	$\geq 1.1.0$	Fragmentacja Markdown po nagłówkach i rekurencyjna
Infrastruktura pomocnicza	tenacity, structlog	$\geq 9.1.4$	Ponowienia wywołań i logowanie strukturalne

Źródło: opracowanie własne na podstawie pliku `pyproject.toml`.

W systemie wyróżniono trzy klasy artefaktów. Danymi wejściowymi są pliki PDF z dokumentami OWU i IPID oraz plik CSV z pytaniami testowymi. Artefaktami pośrednimi są pliki Markdown powstałe po ekstrakcji i kolekcja Qdrant. Artefaktami końcowymi są odpowiedzi agentów, metadane wykonania oraz pliki ewaluacyjne. Tabela 4.2 przedstawia wszystkie klasy artefaktów z ich lokalizacją i zastosowaniem.

Tabela 4.2 Artefakty systemu

Etap	Artefakt	Zastosowanie
Wejście	data/documents/raw_documents	Surowe dokumenty OWU i IPID w formacie PDF
	data/evaluation/questions/*.csv	Pytania, odpowiedzi referencyjne i metadane próbek
	data/documents/extracted_documents	Dokumenty po ekstrakcji do Markdown
Pośredni	data/documents/insurance_documents	Kolekcja Qdrant z wektorami i metadanymi fragmentów
	data/diagrams	Diagramy architektur generowane z definicji Mermaid
Wyjście	data/evaluation/results/runner_results_*.jsonl/csv/json	Surowe odpowiedzi, cytowania i metadane wykonania
	data/evaluation/results/overall_*.csv	Artefakty ewaluacji odpowiedzi i trafności dokumentów

Źródło: opracowanie własne.

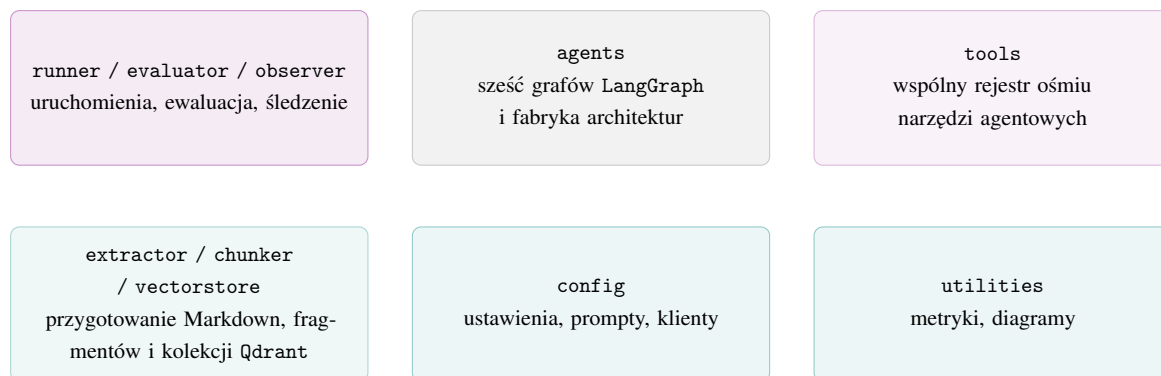
4.2. Architektura modułowa systemu

Główna implementacja systemu mieści się w module `sources` i dzieli się na sześć warstw: konfigurację, architektury agentowe, narzędzia, potok dokumentów, wykonanie eksperymentu oraz narzędzia pomocnicze. Podział ten ogranicza zależności między komponentami – wymiana jednej architektury nie wymaga modyfikacji narzędzi wyszukiwania, syntezy ani ewaluacji.

Istotny podział wprowadzono między katalogami `agents` i `tools`. Pierwszy z nich zawiera wyłącznie logikę sterowania: kolejność węzłów, krawędzie warunkowe, pętle i warunki zakończenia. Drugi z kolei zawiera operacje dziedzinowe wykonywane przez wszystkie architektury: parsowanie pytania, przepisywanie zapytania, wyszukiwanie, selekcję dowodów, cytowania, dobór promptu i syntezę odpowiedzi. Żadna z architektur nie implementuje własnej logiki wyszukiwania ani generowania odpowiedzi.

Zależności między warstwami przedstawiono na rysunku 4.1, a odpowiedzialności poszczególnych modułów opisano w tabeli 4.3.

Rysunek 4.1 Warstwy systemu



Źródło: opracowanie własne.

Tabela 4.3 Moduły systemu

Moduł	Zakres	Główne pliki
Konfiguracja	Konfiguracja ścieżek, modeli, Qdrant, ewaluacji i promptów	app.py, graph.py, prompts.py
Agenty	Implementacje sześciu topologii i wspólny stan	state.py, factory.py, pliki architektur
Narzędzia	Rejestr i implementacja narzędzi agentowych	osiem modułów narzędziowych.
Potok dokumentów	Ekstrakcja PDF, fragmentacja i indeksowanie	extractor.py, chunker.py, vectorstore.py
Wykonanie	Uruchamianie macierzy pytań i architektur	runner.py
Ewaluacja	Przygotowanie danych dla Phoenix i zapis metryk	evaluator.py, observer.py
Pomocnicze	Śledzenie kosztów, tokenów i renderowanie diagramów	tracker.py, drawer.py
Notatniki	Sterowany przebieg przygotowania indeksu, uruchomień i ewaluacji	NB_00 – NB_04

Źródło: opracowanie własne.

4.3. Elementy architektury współdzielone

Komponenty współdzielone stanowią podstawę wszystkich architektur. Obejmują wspólny obiekt stanu, konfigurację uruchomieniową, zamknięte schematy wyjść strukturalnych, rejestr narzędzi oraz fabrykę grafów.

4.3.1 Stan grafu

Węzły grafów komunikują się za pomocą typowanego słownika `ExperimentState`, zdefiniowanego w `sources/agents/state.py`. Każdy węzeł otrzymuje pełny stan, ale zwraca wyłącznie pola zmodyfikowane – `LangGraph` scala je automatycznie. Zapobiega to nadpisywaniu wyników poprzednich kroków i upraszcza diagnostykę.

Listing 4.1 Inicjalizacja stanu grafu

```
1 def make_initial_state(  
2     question: str,  
3     pattern_name: str,  
4     *,  
5     company: str | None = None,  
6     product: str | None = None,  
7 ) -> dict:  
8     return {  
9         "question":      question,  
10        "company":       company,  
11        "product":       product,  
12        "intent":        None,  
13        "entities":      [],  
14        # ... (pola pomocnicze pominięte dla zwięzłości)  
15        "rewritten_queries": [],  
16        "retrieved_chunks": [],  
17        "evidence_chunks": [],  
18        "citations":     [],  
19        "answer":        None,  
20        "error":         None,  
21        "pattern_name":  pattern_name,  
22        "metadata":      {},  
23    }
```

Pola stanu pogrupowano według funkcji w tabeli 4.4. Pola pomocnicze, specyficzne dla danej architektury (np. `_plan`, `_worker_index`, `_bb_iter`, `_react_observations`), przechowywane są w słowniku `metadata`. Prefiks w postaci podkreślenia oznacza, że pole służy sterowaniu lokalnemu i nie należy do kontraktu danych eksperymentalnych.

Tabela 4.4 Stan grafu

Grupa	Pola
Wejście	question, pattern_name, opcjonalnie company, product
Interpretacja pytania	intent, entities, is_multi_part, language
Wyszukiwanie	rewritten_queries, retrieved_chunks, evidence_chunks
Generowanie	selected_prompt_template, system_prompt, citations
Odpowiedź	answer_body, answer_with_references, answer
Diagnostyka	error, metadata

Źródło: opracowanie własne.

4.3.2 Konfiguracja uruchomieniowa

Konfiguracja składa się z dwóch poziomów. AppConfig z `sources/config/app.py` to obiekt Pydantic agregujący grupy ustawień: ścieżki danych, konfigurację modeli z temperaturą 0,0, parametry osadzeń, profile fragmentacji dla OWU i IPID, parametry Qdrant i współbieżność. Mapa `role_models` przypisuje modele do ról pomocniczych i głównej syntezy odpowiedzi.

GraphConfig z `sources/config/graph.py` to stała klasa danych, tworzona dla każdego przebiegu. Pełni rolę kontenera zależności: udostępnia klienty modeli i bazy wektorowej, inicjowane na żądanie i buforowane. Zapewnia to współdzielenie puli połączeń HTTP w ramach jednego przebiegu. Metoda `aclose` zamyka połączenia po jego zakończeniu, co zapobiega wyczerpaniu puli przy 540 równoległych wywołaniach.

4.3.3 Schematy wyjść strukturalnych

W `sources/config/schemas.py` zdefiniowano trzy schematy Pydantic dziedziczące po `_StrictBaseModel` z atrybutem `extra="forbid"`. Są one przekazywane do API OpenAI za pomocą interfejsu `with_structured_output`, co blokuje zwracanie niezadeklarowanych pól.

`QuestionParseResult` opisuje wynik parsowania pytania: intencję (dziewięć literałów), listę encji, flagę wieloczęściowości i język. `RouterDecision` ogranicza wyjście routera do ośmiu intencji (z pominięciem `unknown`), wymuszając jednoznaczną decyzję. `ReactDecision` zawiera uzasadnienie, nazwę akcji i parametry wyszukiwania. Zamknięte schematy zapobiegają wystąpieniu nieoczekiwanych kluczy w zmiennych sterujących.

4.3.4 Rejestr narzędzi i fabryka grafów

Rejestr narzędzi w `sources/tools/__init__.py` mapuje nazwy używane przez agentów na asynchroniczne funkcje narzędziowe. Z tego samego rejestru korzystają architektura planer-wykonawca i ReAct.

Listing 4.2 Rejestr narzędzi agentowych

```
1 TOOL_REGISTRY = {  
2     "question_parser":    parse_question,  
3     "query_rewriter":     rewrite_query,  
4     "retriever":         retrieve,  
5     "reranker":          rerank,  
6     "evidence_selector": select_evidence,  
7     "citation_maker":     make_citations,  
8     "prompt_selector":    select_prompt,  
9     "answer_synthesizer": synthesize_answer,  
10 }
```

Fabryka grafów w `sources/agents/factory.py` powiązuje nazwę architektury z modulem udostępniającym funkcję `build_graph`. Lista `ALL_PATTERNS` wykorzystywana jest przez komponent runner do budowy macierzy uruchomień. Funkcja `build_graph` wykorzystuje `importlib` do importu odroczonego (ładowane są tylko moduły używane w uruchomieniu). Przekazanie nieznanej nazwy zgłasza `ValueError`, zapobiegając błędom braku obsługi danej topologii.

4.4. Implementacja promptów

Wszystkie prompty zdefiniowano w jednym module `sources/config/prompts.py`. Taka centralizacja ułatwia kontrolę nad systemem: instrukcje nie są rozproszone, a każdy prompt ma jednoznaczną nazwę i miejsce użycia. Wyróżniono cztery grupy promptów: ekstrakcyjne, strukturalne, sterujące i odpowiedziowe. Dodatkową grupę stanowią prompty ewaluacyjne, wykorzystywane przez usługę Phoenix po zakończeniu przebiegów. Pełne treści promptów zamieszczono w dodatku A.

4.4.1 Prompt ekstrakcyjny

`EXTRACTION_PROMPT` wykorzystywany jest podczas konwersji strony PDF do formatu Markdown. Instrukcja wymusza zachowanie polskich znaków, hierarchii nagłówków, numeracji, tabel i list. W przypadku dokumentów IPID narzuca zestaw nagłówków drugiego poziomu, zgodny z obowiązkowym układem formularza. Dla dokumentów OWU wymaga zachowania struktury rozdziałów i paragrafów oraz odwzorowania tabel składek i limitów. W obu przypadkach dodawanie treści nieobecnych w źródle jest zabronione, co ogranicza ryzyko halucynacji na etapie ekstrakcji.

4.4.2 Prompty strukturalne

Prompty strukturalne zwracają dane w formacie JSON, przetwarzane następnie przez kod sterujący. Parser pytania i router wykorzystują `with_structured_output` oraz schematy Pydantic. Pozostałe prompty zwracają tablice JSON, parsowane ręcznie z wykorzystaniem mechanizmu awaryjnego.

QUERY_REWRITER_SYSTEM_PROMPT generuje trzy warianty zapytania: bezpośredni, uogólniony i hipotetyczny (implementujący strategię HyDE). RERANKER_SYSTEM_PROMPT ocenia trafność fragmentów w partiach. ROUTER_SYSTEM_PROMPT klasyfikuje pytanie do jednej z ośmiu intencji. PLANNER_SYSTEM_PROMPT buduje plan z maksymalnie ośmioma krokami. REACT_SYSTEM_PROMPT prowadzi pojedynczą iterację cyklu myśl–akcja–obserwacja. DECOMPOSER_SYSTEM_PROMPT dzieli pytanie złożone na maksymalnie trzy podpytania.

4.4.3 Prompty odpowiedziowe

Słownik PROMPT_TEMPLATES przypisuje szablony odpowiedzi do jedenastu intencji pytań. Wszystkie szablony wymuszają strukturę złożoną z sekcji **Odpowiedź** (krótka konkluzja), separatora oraz sekcji **Szczegóły** (uzasadnienie z obowiązkiem cytowania każdego faktu znacznikiem [N]). Szablony różnią się strategią wyboru fragmentów: EXCLUSIONS_PROMPT dotyczy wyłączeń, LIMITS_PROMPT – sum ubezpieczenia, CLAIMS_PROMPT – procedury zgłoszenia szkody, a COMPARISON_PROMPT narzuca układ tabelaryczny. We wszystkich wariantach zalecono zwiększyć i neutralny ton w języku polskim.

4.4.4 Prompty ewaluacyjne

Komponent ewaluacyjny Phoenix wykorzystuje instrukcje REFERENCE_CORRECTNESS_PROMPT i CONCISENESS_PROMPT po zakończeniu przebiegów. Każdy prompt definiuje pięciostopniową skalę ocen (od 1 do 5). Oddzielenie promptów generujących od oceniających zapobiega przeciekowi informacji między warstwami.

Tabela 4.5 podsumowuje wszystkie grupy promptów z ich zastosowaniem.

Tabela 4.5 Prompty systemu

Grupa	Prompty	Zastosowanie
Ekstrakcja	EXTRACTION_PROMPT	Konwersja stron PDF do Markdown
Strukturalne	QUESTION_PARSER_SYSTEM_PROMPT, QUERY_REWRITER_SYSTEM_PROMPT, RERANKER_SYSTEM_PROMPT	Dane dla filtrów, zapytań i oceny fragmentów
Sterujące	PLANNER_SYSTEM_PROMPT, ROUTER_SYSTEM_PROMPT, REACT_SYSTEM_PROMPT, DECOMPOSER_SYSTEM_PROMPT, MERGE_ANSWERS_SYSTEM_PROMPT	Decyzje grafów i łączenie odpowiedzi częściowych
Odpowiedziowe	FAQ_PROMPT, COVERAGE_PROMPT, EXCLUSIONS_PROMPT, LIMITS_PROMPT, COMPARISON_PROMPT, CLAIMS_PROMPT i pięć pozostałych	Dobór stylu odpowiedzi do intencji pytania
Ewaluacyjne	REFERENCE_CORRECTNESS_PROMPT, CONCISENESS_PROMPT	Ocena odpowiedzi po zakończeniu przebiegu

Źródło: opracowanie własne.

4.5. Implementacja narzędzi

Narzędzia agentowe zrealizowano jako funkcje asynchroniczne o wspólnym kontrakcie: przyjmują bieżący stan `ExperimentState` oraz konfigurację `GraphConfig`, a zwracają słownik częściowej aktualizacji stanu. Umożliwia to wywoływanie tych samych narzędzi z poziomu grafu liniowego, pętli wykonawcy, dyspozytora tablicowego oraz kontrolera `ReAct`.

Listing 4.3 Kontrakt narzędzia agentowego

```

1 async def tool_name(
2     state: ExperimentState,
3     *,
4     config: GraphConfig,
5     **kwargs,
6 ) -> dict:
7     ...

```

Każde narzędzie aktualizuje słownik metadata: `record_tool_call` rejestruje liczbę i kolejność wywołań, natomiast `record_model_usage` dopisuje szacowaną liczbę tokenów i koszt operacji. Metadane te służą analizie przebiegu i nie wpływają na zwracaną odpowiedź.

4.5.1 Parser pytania

Narzędzie `question_parser` wyodrębnia z pytania nazwę ubezpieczyciela, produkt, intencję (spośród dziewięciu literałów), listę encji, flagę `is_multi_part` i język. Wywołanie zrealizowano za pomocą `with_structured_output` oraz schematu `QuestionParseResult`. W przypadku błędu zwracany jest stan awaryjny z intencją `unknown`, bez przerywania przebiegu. Jeżeli zidentyfikowano produkt bez nazwy firmy, jest ona uzupełniana na podstawie wewnętrznego mapowania.

4.5.2 Przepisywanie zapytania

Narzędzie `query_rewriter` generuje trzy warianty zapytania: bezpośredni (zachowujący oryginalne słownictwo), uogólniony (wykorzystujący nazwy sekcji dokumentów) i hipotetyczny (realizujący strategię HyDE). W przypadku błędu parsowania zapytanie powielane jest w trzech kopiach jako bezpieczny wariant domyślny.

4.5.3 Wyszukiwanie

Narzędzie `retriever` wektoryzuje zapytanie i odpytuje kolekcję `Qdrant`. Przed konstrukcją filtra normalizowane są metadane: usuwane są znaki diakrytyczne oraz rozwiązywane są aliasy. Wyszukiwanie wykorzystuje trójstopniową relaksację filtrów: najpierw uwzględnia firmę i produkt, następnie wyłącznie firmę, a na końcu przebiega bez filtra.

Listing 4.4 Relaksacja filtrów wyszukiwania

```
1 for filter_mode, query_filter in _build_filter_candidates(  
2     parsed_company, parsed_product  
3 ):  
4     hits = await config.qdrant.query_points(  
5         collection_name=config.collection_name,  
6         query=query_vector,  
7         query_filter=query_filter,  
8         limit=retrieval_limit,  
9     )  
10    for point in hits.points:  
11        collected_points[str(point.id)] = point  
12    if len(collected_points) >= minimum_hits:  
13        break
```

Funkcja `retrieve_multi` wykonuje `retriever` równoległe (`asyncio.gather`) dla trzech wariantów zapytania. Wyniki podlegają deduplikacji względem klucza (`source_file`, `header_2`, `header_3`), a zachowywany jest fragment o najwyższym podobieństwie.

4.5.4 Ponowne szeregowanie

Narzędzie `reranker` ocenia trafność fragmentów z wykorzystaniem modelu językowego (funkcja domyślnie wyłączona). W wariantcie aktywnym dzieli fragmenty na partie z uwzględ-

nieniem limitu tokenów, ocenia je współbieżnie i odrzuca wyniki poniżej progu 0,3. Węzeł zaimplementowano w każdym grafie, co umożliwia parametryzację eksperymentu.

4.5.5 Selektor dowodów

Narzędzie `evidence_selector` to deterministyczna funkcja operująca bez modelu językowego. Realizuje deduplikację względem pary (`source_file`, `header_2`), sortowanie (na podstawie wyniku, głębokości nagłówka i długości fragmentu), wybór najlepszych fragmentów (`evidence_top_k`) przy zapewnieniu pokrycia co najmniej dwóch plików źródłowych oraz utrzymanie stabilnej kolejności wyników.

4.5.6 Generator cytowań

Narzędzie `citation_maker` konstruuje listę cytowań z wykorzystaniem fragmentów dowodowych. Pojedyncze cytowanie obejmuje identyfikator, plik źródłowy, typ dokumentu, metadane oraz cytat do 320 znaków. Numeracja cytowań jest zachowywana podczas całego przebiegu, co gwarantuje spójność odnośników [N] ze stopką **Źródła**.

4.5.7 Selektor promptu

Narzędzie `prompt_selector` określa szablon odpowiedzi na podstawie zmiennej `intent` (wariant awaryjny to `unknown`). Decyzja podejmowana jest w sposób deterministyczny, co eliminuje wywołania modelu językowego i zużycie tokenów.

4.5.8 Syntezator odpowiedzi

Narzędzie `answer_synthesizer` buduje blok kontekstu na podstawie pytania, metadanych oraz cytowań, a następnie wywołuje model z wybranym szablonem. Otrzymana odpowiedź podlega trójstopniowej normalizacji: wymuszeniu formatu **Odpowiedź/Szczegóły**, standaryzacji znaczników oraz dołączeniu stopki **Źródła**. Błąd API logowany jest w zmiennej `error`, a zwracany komunikat informuje o niewystarczającym kontekście.

Tabela 4.6 przedstawia wszystkie narzędzia.

Tabela 4.6 Narzędzia agentowe

Narzędzie	Wejście	Wyjście	Model
question_parser	pytanie, opcjonalnie firma i produkt	intencja, encje, firma, produkt, język	tak
query_rewriter	pytanie i metadane parsera	trzy warianty zapytania	tak
retriever	zapytanie, firma, produkt, top_k	fragmenty z Qdrant	osadzenia
reranker	pytanie i fragmenty	przefiltrowane fragmenty z wynikiem	tak
evidence_selector	fragmenty pobrane lub ocenione	zwięzły zestaw dowodów	nie
citation_maker	fragmenty dowodowe	obiekty cytowań	nie
prompt_selector	intencja	nazwa szablonu i prompt systemowy	nie
answer_synthesizer	pytanie, dowody, cytowania, prompt	odpowiedź i źródła	tak

Źródło: opracowanie własne.

4.6. Implementacja architektur agentowych

Każda architektura stanowi osobny moduł w `sources/agents`, eksportujący funkcję `build_graph`. Funkcja ta tworzy graf `StateGraph(ExperimentState)`, rejestruje węzły, definiuje krawędzie i zwraca skompilowany obiekt `LangGraph`. Diagramy architektur wygenerowano automatycznie na podstawie definicji Mermaid i zamieszczono w dodatku B.

4.6.1 Architektura deterministyczna

Architektura deterministyczna to graf liniowy pozbawiony węzłów decyzyjnych. Przetwarzanie pytania obejmuje sekwencję ośmiu kroków: parsowanie, przepisywanie, wyszukiwanie wielozapytaniowe, ponowne szeregowanie, selekcję dowodów, wybór promptu, budowę cytowań i syntezę. Brak pętli i warunków sprawia, że wariant ten stanowi punkt odniesienia: profil kosztów jest deterministyczny, a wszelkie różnice względem pozostałych architektur wynikają wyłącznie z logiki sterującej.

Listing 4.5 Krawędzie grafu deterministycznego

```

1 graph.add_edge(START, "parse_question")
2 graph.add_edge("parse_question", "rewrite_query")
3 graph.add_edge("rewrite_query", "retrieve_all")
4 graph.add_edge("retrieve_all", "rerank")

```



```

5 graph.add_edge("rerank", "select_evidence")
6 graph.add_edge("select_evidence", "select_prompt")
7 graph.add_edge("select_prompt", "make_citations")
8 graph.add_edge("make_citations", "generate_answer")
9 graph.add_edge("generate_answer", END)

```

4.6.2 Architektura planer–wykonawca

Architektura planer–wykonawca wdraża węzeł planera poprzedzający wykonanie narzędzi. Planer przetwarza pytanie i definicje narzędzi z TOOL_REGISTRY, zwracając listę kroków w formacie JSON ({tool_name, rationale}). Walidacja planu wymaga maksymalnie ośmiu kroków z rejestru, zakończonych wywołaniem answer_synthesizer. Błąd parsowania lub niespełnienie warunków skutkuje przyjęciem planu awaryjnego (zgodnego z potokiem deterministycznym).

Listing 4.6 Warunek pętli wykonawcy

```

1 def should_continue(state: ExperimentState) -> str:
2     metadata = state.get("metadata", {})
3     plan = metadata.get("_plan", [])
4     idx = metadata.get("_plan_index", 0)
5     if (idx < len(plan)
6         and plan[idx].get("tool_name") != "answer_synthesizer"):
7         return "executor"
8     return "synthesizer"

```

4.6.3 Architektura router–specjalista

Architektura router–specjalista klasyfikuje pytanie do jednej z ośmiu intencji (schemat RouterDecision), a następnie deleguje wykonanie do jednego z czterech podpotoków (FAQ, opis, roszczenia, porównania). Podpotoki współdzielą zestaw kroków, różniąc się strategią wyszukiwania: wariant roszczeniowy zwiększa parametr top_k, wariant porównawczy omija filtr firmy, a wariant FAQ wykorzystuje pojedyncze zapytanie.

Listing 4.7 Mapowanie intencji na podpotoki

```

1 # app_config.router.category_to_pipeline
2 CATEGORY_TO_PIPELINE = {
3     "faq": "faq",
4     "limits": "faq",
5     "description": "description",
6     "coverage": "description",
7     "conditions": "description",
8     "claims": "claims",
9     "exclusions": "claims",
10    "comparison": "comparison",

```

```
11 }
```

4.6.4 Architektura tablicowa

W architekturze tablicowej model językowy zastąpiono deterministyczną funkcją regułową (dyspozytorem). W kolejnych iteracjach dyspozytor weryfikuje uzupełnienie pól stanu i inicjuje odpowiednie narzędzie (przy stałej kolejności weryfikacji). Po powrocie do dyspozytora inkrementowany jest licznik iteracji. Parametr `max_blackboard_iterations` zapobiega wystąpieniu nieskończonej pętli.

Listing 4.8 Reguły dyspozytora tablicowego

```
1 if metadata.get("_bb_iter", 0) >= max_iter:
2     return "answer_agent"
3 if state.get("intent") is None:
4     return "parser_agent"
5 if not state.get("rewritten_queries"):
6     return "rewriter_agent"
7 if not state.get("retrieved_chunks"):
8     return "retriever_agent"
9 if chunks and not state.get("evidence_chunks"):
10    return "evidence_selector_agent"
```

4.6.5 Architektura hierarchiczna

Architektura hierarchiczna obejmuje cztery komponenty: parser, dekompozytor, pętlę pracownika i syntezytor. Dekompozytor dzieli zapytanie (na maksymalnie trzy podpytania). Pracownik przetwarza je w uproszczonym potoku (przepisanie, wyszukiwanie, selekcja, odpowiedź częściowa). Syntezytor integruje wyniki, wykonując globalną deduplikację cytowań i wymuszając dwusekcyjny format. Deduplikacja fragmentów zapobiega liniowemu przyrostowi rozmiaru kontekstu.

Listing 4.9 Pętla pracownika w architekturze hierarchicznej

```
1 def should_continue_worker(state: ExperimentState) -> str:
2     metadata = state.get("metadata", {})
3     idx = metadata.get("_worker_index", 0)
4     sub = metadata.get("sub_questions", [])
5     if idx < len(sub):
6         return "worker"
7     return "synthesizer"
```

4.6.6 Architektura ReAct

Architektura ReAct integruje trzy węzły: agenta, wykonawcę narzędzi oraz węzeł wymuszonego zakończenia. Agent iteracyjnie zwraca obiekt `ReactDecision` (myśl, akcja, param-

try). Wykonawca inicjuje narzędzie i loguje skróconą obserwację w `_react_observations` (pomijając pełną treść odpowiedzi). Zakończenie następuje po akcji `finish`, wygenerowaniu odpowiedzi lub przekroczeniu limitu iteracji. Mechanizm skróconych obserwacji zapobiega rozrostowi kontekstu.

Listing 4.10 Warunek kontynuacji pętli ReAct

```

1 def should_continue(state: ExperimentState) -> str:
2     metadata = state.get("metadata", {})
3     decision = metadata.get("_react_decision", {})
4     iterations = len(metadata.get("_react_observations", []))
5     if decision.get("action") == "finish" or state.get("answer") is not None:
6         return "finish"
7     if iterations >= max_iter:
8         return "force_finish"
9     return "agent"

```

Tabela 4.7 zestawia implementacyjne różnice między architekturami.

Tabela 4.7 Zestawienie topologii grafów

Architektura	Sterowanie	Cechy grafu	Zakończenie
Deterministyczna	Stała sekwencja	Liniowy, skierowany acykliczny graf, osiem węzłów	Ostatni węzeł
Planer–wykonawca	Plan JSON z modelu	Planer, pętla wykonawcy, synteza	Wyczerpanie planu
Router–specjalista	Klasyfikacja intencji	Router i cztery podpotoki	Koniec podpotoku
Tablicowa	Reguły na stanie	Dyspozytor i pula agentów	Odpowiedź lub limit
Hierarchiczna	Dekompozycja pytania	Dekompozytor, pętla, scalanie	Wszystkie podpytania
ReAct	Decyzja modelu w pętli	Agent, wykonawca, zakończenie	<code>finish</code> lub limit

Źródło: opracowanie własne.

4.7. Protokół przetwarzania dokumentów

Potok dokumentów przygotowuje zamknięty korpus do wyszukiwania wektorowego. Proces ten uruchamiany jest jednorazowo przed eksperymentem (architektury nie modyfikują go w trakcie odpowiadania). Obejmuje cztery etapy: ekstrakcję PDF do Markdown, fragmentację tekstu, wektoryzację oraz zapis w Qdrant. Kolejność operacji zilustrowano na rysunku 4.2.

Rysunek 4.2 Potok przetwarzania dokumentów



Źródło: opracowanie własne.

4.7.1 Ekstrakcja

Ekstrakcję realizuje klasa `Extractor` (`sources/extractor.py`). Biblioteka `pypdfium2` renderuje obraz strony (skala 2), który – po zakodowaniu jako Base64 PNG – trafia do modelu wizyjnego wraz z promptem. Kontekst międzystroniczny pozwala dołączyć 1024 znaki z poprzedniej strony, co zachowuje ciągłość sekcji rozbitych między arkuszami. Wywołanie powtarzane jest maksymalnie pięciokrotnie z wykładniczym opóźnieniem. Wynik stanowi plik Markdown zapisany w ustalonej strukturze katalogów.

4.7.2 Fragmentacja

Fragmentację tekstu wykonuje klasa `Chuncker`. Dokument dzielony jest według nagłówków (poziomy 1–3), a odpowiednie metadane propagowane są do poszczególnych fragmentów. Większe sekcje podlegają fragmentacji rekurencyjnej z wykorzystaniem parametrów właściwych dla danego typu dokumentu (tabela 5.20).

Tabela 4.8 Parametry fragmentacji

Typ dokumentu	Rozmiar fragmentu	Nakładanie	Minimum
IPID	384	48	48
OWU	512	64	64

Źródło: opracowanie własne.

Rozmiar fragmentów dla dokumentów IPID zredukowano ze względu na zwięzłość formularzy, natomiast dla dokumentów OWU wartości zwiększono w celu zachowania kontekstu długich klauzul. Fragmentom przypisywany jest deterministyczny identyfikator UUIDv5 (na bazie nazwy pliku, nagłówków i treści).

4.7.3 Indeksowanie

Tworzenie indeksu realizuje klasa `VectorStore` (`sources/vectorstore.py`). Inicjalizuje ona kolekcję `insurance_documents` z miarą cosinusową, indeksem HNSW oraz indeksami pól (payload). Przed wektoryzacją do treści fragmentu doklejane są metadane: firma, produkt, kategoria, typ i ścieżka nagłówków. Zabieg ten poprawia dopasowanie wektorowe dla pytań

uwzględniających ubezpieczyciela. Zapis w bazie Qdrant przebiega partiami. Pola strukturalne indeksowane są jako *keyword*, co gwarantuje stały czas działania filtrów metadanych.

4.8. Infrastruktura ewaluacyjna

Warstwę ewaluacyjną podzielono na trzy moduły. Komponent Runner uruchamia architektury i zapisuje odpowiedzi. Evaluator oblicza metryki, korzystając z usługi Phoenix. Observer inicjalizuje śledzenie OpenTelemetry i rejestruje przebiegi. Podział ten uniezależnia generowanie odpowiedzi od oceny (architektury nie uzyskują sprzężenia zwrotnego z ewaluacji).

4.8.1 Uruchamianie architektur

Komponent Runner wczytuje pliki CSV z pytaniami i generuje iloczyn kartezjański par pytanie–architektura (540 przebiegów dla 90 pytań). Wykonanie zrównoleglono w puli wątków, wykorzystując `graph.ai.invoke`. Wyniki logowane są w sposób ciągły do plików JSONL i CSV, co zapobiega utracie danych. Pojedynczy rekord zawiera pytanie, wariant architektury, odpowiedź, metryki wywołań, koszty oraz pełne metadane wykonania.

4.8.2 Ocena odpowiedzi

Komponent Evaluator implementuje bibliotekę `phoenix.evals`. Normalizuje wyniki i wykonuje weryfikację pod kątem: wierności (*faithfulness*), zwięzłości (*conciseness*) oraz poprawności (*correctness*). Trafność dokumentów (*document relevance*) oceniana jest rozdzielnie (dla cytowanych fragmentów), co izoluje analizę wyszukiwania od syntezy. Klasyfikatory poprawności i zwięzłości wykorzystują pięciostopniowe skale rzutowane na wartości numeryczne.

4.8.3 Śledzenie przebiegów

Komponent Observer konfiguruje usługę Phoenix (`auto_instrument=True`), co zapewnia automatyczne śledzenie wywołań bibliotek LangChain i OpenAI. Ponadto zdefiniowano własne zakresy monitorowania dla przebiegów architektur i obliczeń metryk.

Tabela 4.9 przedstawia artefakty wyjściowe fazy ewaluacji.

Tabela 4.9 Artefakty ewaluacji

Plik	Zawartość
runner_results_*.jsonl	Strumieniowy zapis pełnych rekordów uruchomień
runner_results_*.csv	Spłaszczona tabela wyników wykonawczych
runner_results_*.json	Pełny zapis wyników po zakończeniu biegu
runner_summary_*.csv	Zbiorcze metryki operacyjne według architektury
overall_answer_metrics_*.csv	Metryki odpowiedzi na poziomie pojedynczego przebiegu
overall_document_relevance_*.csv	Ocena trafności cytowanych fragmentów
overall_summary_*.csv	Zbiorcze podsumowanie ewaluacji Phoenix

Źródło: opracowanie własne.

4.9. Wybrane zagadnienia inżynieryjne

4.9.1 Zarządzanie rozmiarem okna kontekstowego

W systemach RAG nadmierny rozrost kontekstu stanowi istotne ograniczenie techniczne. Moduł rerankera przycina fragmenty do 1000 znaków i dzieli dane na partie (limit 12000 tokenów). Selektor dowodów ogranicza kontekst syntezy do `evidence_top_k` fragmentów. W architekturze ReAct zamiast pełnej treści dokumentów logowane są krótkie podsumowania działań narzędzi, co uniezależnia rozmiar historii iteracji od liczby pobranych fragmentów.

4.9.2 Agregacja metadanych wykonania

Metadane wykonania są agregowane przez funkcję `merge_tracking_metadata`. Sumuje ona liczniki narzędzi, tokenów i kosztów przy jednoczesnym zachowaniu lokalnych zmiennych sterujących. Mechanizm ten znajduje zastosowanie w architekturze hierarchicznej oraz w funkcji `retrieve_multi`, gdzie pojedyncze narzędzie wywoływane jest wielokrotnie w ramach jednego węzła.

4.9.3 Obsługa niepoprawnych wyjść modeli

Komponenty zależne od formatu JSON wyposażono w ścieżki awaryjne. Parser zwraca metadane neutralne, rewriter powiela zapytanie bazowe, planer stosuje wariant deterministyczny, a dekompozytor nie dzieli pytania. Architektura ReAct loguje nierozpoznaną akcję jako obserwację. Błędy są rejestrowane, jednak system gwarantuje zwrócenie kontrolowanego wyniku końcowego.

4.9.4 Strategia wielojęzycznych promptów

Instrukcje systemowe sformułowano w języku angielskim (z uwagi na zwieżłość i zgodność z formatami danych), natomiast pytania, dokumenty i odpowiedzi pozostają polskojęzyczne.

Prompty odpowiedziowe wymuszają generowanie tekstu w języku polskim oraz zakazują korzystania z wiedzy zewnętrznej.

4.9.5 Stabilność identyfikatorów fragmentów

Identyfikatory fragmentów generowane są deterministycznie (algorytm UUIDv5) na podstawie metadanych i treści. Gwarantuje to identyczność kluczy w bazie Qdrant przy kolejnych uruchomieniach potoku dla niezmiennego korpusu, co pozwala na przyrostową aktualizację i porównywanie wyników.

4.9.6 Izolacja warstwy ewaluacyjnej

Ewaluator operuje wyłącznie na utrwalonych artefaktach. Architektury nie modyfikują odpowiedzi na podstawie wyników z Phoenix. Separacja ta zapewnia odtwarzalność procesu: surowe pliki wynikowe mogą być poddawane ponownej ocenie (z użyciem innych ewaluatorów lub rubryk) bez konieczności powtarzania przebiegów.

5. Eksperyment i wyniki

Rozdział dokumentuje przebieg eksperymentu porównawczego oraz surowe wyniki liczbowe. Ramy metodyczne i kryteria oceny opisano w rozdziale 3, a szczegóły implementacji – w rozdziale 4. Niniejszy rozdział ogranicza się do prezentacji danych i wyników – interpretację i wnioski omówiono w rozdziale następnym.

5.1. Przebieg wykonania

Eksperyment zrealizowano w ramach jednego uruchomienia obejmującego pełną macierz $90 \times 6 = 540$ par pytanie–architektura. Poniższe podsekcje dokumentują czas trwania, kompletność wyników oraz zestaw wygenerowanych artefaktów.

5.1.1 Czas trwania

Uruchomienie przeprowadzono 13 marca 2026 r. pod identyfikatorem 20260313_200511. Komponent wykonawczy przekazał 540 par do puli 32 wątków roboczych; ostatni rekord zapisano o 21:52 UTC, co odpowiada rzeczywistemu czasowi wykonania 1 h 47 min. Łączny czas sekwencyjny – rozumiany jako suma opóźnień wszystkich uruchomień – wynosi 62 698 s ($\approx 17,4$ h). Faza ewaluacji jakościowej w systemie Phoenix zakończyła się o 22:55 UTC.

Tabela 5.1 przedstawia rozkład czasu sekwencyjnego według architektur wraz z udziałem procentowym oraz średnim czasem pojedynczego uruchomienia (z odchyleniem standardowym). Architektura ReAct wykazała największy udział w łącznym czasie sekwencyjnym (26,1%), natomiast planer–wykonawca – najmniejszy (12,1%). Stanowi to interesujący paradoks wydajnościowy: mimo dodatkowego narzutu czasowego na generowanie planu, agent ten okazał się najszybszy w ogólnej egzekucji (średnio 84,3 s), wyprzedzając pod tym względem nawet najprostszą architekturę deterministyczną (93,0 s). Wynika to z faktu, że trafnie ułożony plan pozwolił na redukcję liczby zbędnych wywołań narzędzi i kosztownych operacji pobierania danych.

Tabela 5.1 Czas wykonania według architektury

Architektura	Uruchomienia	Suma opóźnień [s]	Udział [%]	Średnia $\pm$ odchylenie [s]
Deterministyczna	90	8 372	13,4	93,0 ($\pm$ 17,5)
Planer–wykonawca	90	7 583	12,1	84,3 ($\pm$ 17,1)
Router–specjalista	90	8 902	14,2	98,9 ($\pm$ 18,3)
Tablicowa	90	8 229	13,1	91,4 ($\pm$ 19,5)
Hierarchiczna	90	13 239	21,1	147,1 ($\pm$ 68,2)
ReAct	90	16 374	26,1	181,9 ($\pm$ 44,9)
Łącznie	540	62 698	100,0	–

Źródło: opracowanie własne.

5.1.2 Kompletność i błędy wykonania

Spośród 540 uruchomień 539 zakończyło się pomyślnie. Jedyne błędy odnotowano w architekturze hierarchicznej dla pytania dotyczącego zakresu autocasco na terytorium Białorusi. Bezpośrednią przyczyną był wyjątek `openai.BadRequestError 400` zwrócony przez interfejs `OpenAI API`: węzeł dekompozytora wygenerował podpytanie w mieszanym formacie polsko-angielskim, co uniemożliwiło poprawną serializację odpowiedzi. Wyjątek przechwycono i zapisano jako rekord z flagą błędu; zgodnie z przyjętą procedurą nie ponowiono wykonania zadania. Wskaźnik błędów wynosi $1/540 \approx 0,19\%$ dla całego eksperymentu i $1,11\%$ dla architektury hierarchicznej.

Żadne uruchomienie nie przekroczyło limitu czasowego 120 s. Nie odnotowano również osiągnięcia limitu iteracji w architekturach iteracyjnych; maksymalna liczba iteracji architektury ReAct wyniosła 11 (przy progu równym 15).

5.1.3 Artefakty wynikowe

Wyniki uruchomienia zapisano w katalogu `code/data/evaluation/results/`. Tabela 5.2 zestawia pliki wyjściowe z ich rozmiarami i zawartością; gwiazdka zastępuje pełny identyfikator uruchomienia `20260313_200511`. Największy rozmiar ma plik `.csv` z surowymi wynikami, natomiast plik zbiorczego podsumowania zajmuje poniżej 1 KB.

Tabela 5.2 Pliki wynikowe eksperymentu

Plik	Rozmiar	Zawartość
runner_results_*.jsonl	16 006 KB	540 rekordów surowych w formacie strumieniowym
runner_results_*.csv	17 131 KB	spłaszczona tabela 540 wierszy i 32 kolumn
runner_summary_*.csv	2 KB	wartości zagregowane według architektury
overall_answer_metrics_*.csv	1 988 KB	539 ocen jakości odpowiedzi z systemu Phoenix
overall_document_relevance_*.csv	2 216 KB	1 786 ocen trafności pojedynczych cytowań
overall_summary_*.csv	<1 KB	zbiorne miary jakości według architektury

Źródło: opracowanie własne.

5.2. Wyniki operacyjne

W sekcji zestawiono średnie wartości miar opisujących zasobochłonność uruchomień: zużycie tokenów i koszt, liczbę wywołań narzędzi i iteracji oraz liczbę pobranych fragmentów i cytowań. Wszystkie wartości w tabelach podano w formacie $\text{średnia} \pm \text{odchylenie standardowe}$; podstawę obliczeń stanowi 90 uruchomień danej architektury, z wyjątkiem hierarchicznej (89 uruchomień).

5.2.1 Zużycie tokenów i koszt

Tabela 5.3 podaje średnie zużycie tokenów wejściowych i wyjściowych oraz średni koszt pojedynczego uruchomienia, oszacowany na podstawie cennika OpenAI API obowiązującego w marcu 2026 r. Architektury jednoagentowe wykazują niewielki rozrzut: średnie zużycie tokenów wejściowych mieści się w przedziale 2 988–3 219, a odchylenie standardowe nie przekracza 260 tokenów. Architektury iteracyjne wykazują znaczną zmienność – średnia hierarchicznej wynosi 450 017 tokenów wejściowych przy odchyleniu standardowym 1 080 756, a architektury ReAct 391 935 przy odchyleniu 419 373. Wartości odchylenia standardowego tak drastycznie przekraczające średnią wynikają z silnie prawostronnie skośnego rozkładu zużycia tokenów: w większości uruchomień agenci zużywają stosunkowo niewielką liczbę tokenów, jednak pojedyncze przebiegi, w których agent ulega zapętleniu lub wielokrotnie ponawia błędne wywołania, konsumują ekstremalnie duże zasoby, co diametralnie zawyża średnią i wariancję. Średni koszt uruchomienia wynosi od 0,00037 USD (architektura tablicowa) do 0,04320 USD (ReAct).

Tabela 5.3 Tokeny i koszt na uruchomienie

Architektura	Tokeny wejściowe	Tokeny wyjściowe	Koszt [USD]
Deterministyczna	2 988 ($\pm$ 230)	528 ($\pm$ 160)	0,00038 ($\pm$ 0,00007)
Planer–wykonawca	3 219 ($\pm$ 260)	580 ($\pm$ 159)	0,00040 ($\pm$ 0,00007)
Router–specjalista	3 199 ($\pm$ 246)	519 ($\pm$ 151)	0,00039 ($\pm$ 0,00007)
Tablicowa	3 004 ($\pm$ 215)	498 ($\pm$ 144)	0,00037 ($\pm$ 0,00006)
Hierarchiczna	450 017 ($\pm$ 1 080 756)	115 761 ($\pm$ 303 962)	0,07109 ($\pm$ 0,18099)
ReAct	391 935 ($\pm$ 419 373)	58 548 ($\pm$ 66 709)	0,04320 ($\pm$ 0,04758)

Źródło: opracowanie własne.

5.2.2 Wywołania narzędzi i iteracje

Tabela 5.4 podaje średnią liczbę wywołań narzędzi i iteracji na uruchomienie wraz z odchyleniem standardowym. Pojęcie iteracji zależy od topologii: dla planera–wykonawcy oznacza długość wygenerowanego planu, dla tablicowej – liczbę cykli dyspozytora, dla hierarchicznej – liczbę przetworzonych podpytań, dla ReAct – liczbę zarejestrowanych obserwacji. Architektury deterministyczna i router–specjalista nie posługują się pojęciem iteracji. Architektura hierarchiczna wykonuje średnio 1 066 wywołań narzędzi na uruchomienie przy odchyleniu 2 568, a architektura ReAct 295,7 przy odchyleniu 294,1. Planer–wykonawca wykazuje najniższą liczbę wywołań narzędzi przy najmniejszym odchyleniu (średnio 7,2 przy odchyleniu 0,8).

Tabela 5.4 Liczba wywołań narzędzi i iteracji na uruchomienie

Architektura	Wywołania narzędzi	Iteracje
Deterministyczna	10,0 ($\pm$ 0,0)	–
Planer–wykonawca	7,2 ($\pm$ 0,8)	4,2 ($\pm$ 0,8)
Router–specjalista	9,8 ($\pm$ 0,6)	–
Tablicowa	10,0 ($\pm$ 0,0)	8,0 ($\pm$ 0,0)
Hierarchiczna	1 066,0 ($\pm$ 2 568,0)	10,6 ($\pm$ 4,8)
ReAct	295,7 ($\pm$ 294,1)	7,7 ($\pm$ 1,4)

Źródło: opracowanie własne.

5.2.3 Fragmenty z wyszukiwania i cytowania

Tabela 5.5 przedstawia średnie liczby fragmentów pobranych z bazy Qdrant oraz cytowań w odpowiedzi końcowej wraz z odchyleniami standardowymi. Kolumna *Suma* podaje łączną liczbę fragmentów pobranych w obrębie jednego uruchomienia – w architekturach iteracyjnych

obejmuje wszystkie wywołania pętli roboczej lub obserwacyjnej. Średnia łączna liczba fragmentów dla architektury hierarchicznej wynosi 1 772 przy odchyleniu 4 300, a dla ReAct 154,5 przy odchyleniu 131,1. Architektury jednoagentowe pobierają 5 lub 15 fragmentów na uruchomienie przy zerowym lub minimalnym odchyleniu standardowym. W architekturze ReAct liczba cytowań w odpowiedzi końcowej jest niższa niż w pozostałych wariantach (średnio w przedziale 3,29–3,82).

Szczególną anomalię odnotowano w przypadku architektury planera–wykonawcy. Zgodnie z tabelą 5.5, po etapie oceny trafności w stanie agenta pozostaje średnio zaledwie 0,31 fragmentu, co świadczy o niezwykle rygorystycznym odrzucaniu pobranego kontekstu przez sędziego. Paradoksalnie, odpowiedź końcowa generowana przez tę architekturę posiada średnio aż 3,38 cytowania. Wskazuje to na iluzję filtrowania dowodów: moduł syntezy w przypadku planera zignorował wyniki oceny trafności i samodzielnie uwzględnił fragmenty (bądź wyhalucynował ich znaczniki), które na etapie oceny zostały z kontekstu usunięte.

Tabela 5.5 Liczba fragmentów i cytowań

Architektura	Na zapytanie	Suma	Po ocenie trafności	Cytowania
Deterministyczna	4,28 ($\pm 1,00$)	15,0 ($\pm 0,0$)	4,28 ($\pm 1,00$)	3,72 ($\pm 0,67$)
Planer–wykonawca	4,98 ($\pm 0,15$)	5,0 ($\pm 0,0$)	0,31 ($\pm 1,17$)	3,38 ($\pm 0,77$)
Router–specjalista	4,33 ($\pm 0,99$)	14,4 ($\pm 2,0$)	4,33 ($\pm 0,99$)	3,67 ($\pm 0,65$)
Tablicowa	4,30 ($\pm 0,90$)	15,0 ($\pm 0,0$)	4,30 ($\pm 0,90$)	3,74 ($\pm 0,53$)
Hierarchiczna	4,70 ($\pm 1,48$)	1 771,7 ($\pm 4 299,8$)	4,70 ($\pm 1,48$)	3,74 ($\pm 0,71$)
ReAct	4,39 ($\pm 1,07$)	154,5 ($\pm 131,1$)	4,22 ($\pm 1,32$)	1,59 ($\pm 1,69$)

Źródło: opracowanie własne.

5.3. Wyniki jakościowe

Poniższe podsekcje przedstawiają wartości czterech miar jakości w rozbiciu na architekturę, poziom trudności pytania, kategorię produktową i ubezpieczyciela. Liczebności podzbiorów wynoszą: 9 pytań bardzo prostych, 27 prostych, 27 złożonych i 27 bardzo złożonych; 30 pytań na każdą kategorię produktową i 30 na każdego ubezpieczyciela. W architekturze hierarchicznej poziom złożony obejmuje 26 pytań ze względu na pojedynczy błąd wykonania.

5.3.1 Wierność

Tabele 5.6–5.8 przedstawiają średnią wierność odpowiedzi wraz z odchyleniem standardowym w rozbiciu na poziom trudności, kategorię produktową i ubezpieczyciela. Najwyższą wartość łączną osiąga router–specjalista (0,811), najniższą ReAct (0,611). Dla wszystkich architektur wierność spada wraz ze wzrostem poziomu trudności pytania. W rozbiciu według kategorii produktowej i ubezpieczyciela rozpiętość wyników między architekturami pozostaje zbliżona

do rozpiętości w ujęciu łącznym, z jednym silnym wyjątkiem. Architektura ReAct odnotowała katastrofalny spadek wierności w kategorii ubezpieczeń mieszkaniowych, uzyskując zaledwie 0,433 (wobec 0,667 dla ubezpieczeń komunikacyjnych i 0,733 dla turystycznych). Wynik bliski losowości wskazuje na wybitną podatność pętli obserwacyjnej ReAct na dryf semantyczny w kategoriach o szerszym i bardziej podatnym na wieloznaczność zakresie pojęciowym.

Tabela 5.6 Wierność według poziomu trudności

Architektura	B. proste	Proste	Złożone	B. złożone	Łącznie
Deterministyczna	0,889 ($\pm 0,105$)	0,852 ($\pm 0,068$)	0,741 ($\pm 0,084$)	0,704 ($\pm 0,088$)	0,778 ($\pm 0,044$)
Planer–wykonawca	0,889 ($\pm 0,105$)	0,741 ($\pm 0,084$)	0,852 ($\pm 0,068$)	0,741 ($\pm 0,084$)	0,789 ($\pm 0,043$)
Router–specjalista	0,889 ($\pm 0,105$)	0,852 ($\pm 0,068$)	0,741 ($\pm 0,084$)	0,815 ($\pm 0,075$)	0,811 ($\pm 0,041$)
Tablicowa	0,778 ($\pm 0,139$)	0,852 ($\pm 0,068$)	0,704 ($\pm 0,088$)	0,815 ($\pm 0,075$)	0,789 ($\pm 0,043$)
Hierarchiczna	0,556 ($\pm 0,166$)	0,815 ($\pm 0,075$)	0,731 ($\pm 0,087$)	0,593 ($\pm 0,095$)	0,697 ($\pm 0,049$)
ReAct	0,556 ($\pm 0,166$)	0,667 ($\pm 0,091$)	0,519 ($\pm 0,096$)	0,667 ($\pm 0,091$)	0,611 ($\pm 0,051$)

Źródło: opracowanie własne.

Tabela 5.7 Wierność według kategorii produktowej

Architektura	Auto	Mieszkanie	Podróż	Łącznie
Deterministyczna	0,800 ($\pm 0,073$)	0,800 ($\pm 0,073$)	0,733 ($\pm 0,081$)	0,778 ($\pm 0,044$)
Planer–wykonawca	0,800 ($\pm 0,073$)	0,833 ($\pm 0,068$)	0,733 ($\pm 0,081$)	0,789 ($\pm 0,043$)
Router–specjalista	0,800 ($\pm 0,073$)	0,833 ($\pm 0,068$)	0,800 ($\pm 0,073$)	0,811 ($\pm 0,041$)
Tablicowa	0,833 ($\pm 0,068$)	0,800 ($\pm 0,073$)	0,733 ($\pm 0,081$)	0,789 ($\pm 0,043$)
Hierarchiczna	0,586 ($\pm 0,091$)	0,700 ($\pm 0,084$)	0,800 ($\pm 0,073$)	0,697 ($\pm 0,049$)
ReAct	0,667 ($\pm 0,086$)	0,433 ($\pm 0,090$)	0,733 ($\pm 0,081$)	0,611 ($\pm 0,051$)

Źródło: opracowanie własne.

Tabela 5.8 Wierność według ubezpieczyciela

Architektura	Hestia	PZU	Warta	Łącznie
Deterministyczna	0,700 ($\pm$ 0,084)	0,800 ($\pm$ 0,073)	0,833 ($\pm$ 0,068)	0,778 ($\pm$ 0,044)
Planer–wykonawca	0,733 ($\pm$ 0,081)	0,800 ($\pm$ 0,073)	0,833 ($\pm$ 0,068)	0,789 ($\pm$ 0,043)
Router–specjalista	0,733 ($\pm$ 0,081)	0,800 ($\pm$ 0,073)	0,900 ($\pm$ 0,055)	0,811 ($\pm$ 0,041)
Tablicowa	0,833 ($\pm$ 0,068)	0,767 ($\pm$ 0,077)	0,767 ($\pm$ 0,077)	0,789 ($\pm$ 0,043)
Hierarchiczna	0,552 ($\pm$ 0,092)	0,833 ($\pm$ 0,068)	0,700 ($\pm$ 0,084)	0,697 ($\pm$ 0,049)
ReAct	0,633 ($\pm$ 0,088)	0,733 ($\pm$ 0,081)	0,467 ($\pm$ 0,091)	0,611 ($\pm$ 0,051)

Źródło: opracowanie własne.

5.3.2 Poprawność

Tabele 5.9–5.11 przedstawiają średnią poprawność odpowiedzi wraz z odchyleniem standardowym w tym samym rozbiściu. Najwyższą wartość łączną odnotowano dla architektury deterministycznej (0,716), a najniższą dla hierarchicznej (0,588). W rozbiściu według poziomu trudności poprawność przyjmuje najwyższe wartości dla pytań prostych we wszystkich wariantach, natomiast w przypadku pytań bardzo złożonych ulega obniżeniu. W rozbiściu według kategorii produktowej architektura deterministyczna osiągnęła najwyższe wartości we wszystkich trzech grupach. W rozbiściu według ubezpieczyciela nie zaobserwowano jednoznacznej tendencji w wynikach poszczególnych architektur.

Tabela 5.9 Poprawność według poziomu trudności

Architektura	B. proste	Proste	Złożone	B. złożone	Łącznie
Deterministyczna	0,644 ($\pm$ 0,132)	0,841 ($\pm$ 0,055)	0,685 ($\pm$ 0,077)	0,644 ($\pm$ 0,067)	0,716 ($\pm$ 0,038)
Planer–wykonawca	0,578 ($\pm$ 0,114)	0,681 ($\pm$ 0,070)	0,596 ($\pm$ 0,084)	0,596 ($\pm$ 0,077)	0,620 ($\pm$ 0,042)
Router–specjalista	0,611 ($\pm$ 0,116)	0,830 ($\pm$ 0,053)	0,633 ($\pm$ 0,078)	0,544 ($\pm$ 0,073)	0,663 ($\pm$ 0,039)
Tablicowa	0,544 ($\pm$ 0,136)	0,807 ($\pm$ 0,058)	0,633 ($\pm$ 0,079)	0,489 ($\pm$ 0,076)	0,633 ($\pm$ 0,042)
Hierarchiczna	0,567 ($\pm$ 0,119)	0,678 ($\pm$ 0,076)	0,608 ($\pm$ 0,077)	0,485 ($\pm$ 0,079)	0,588 ($\pm$ 0,043)
ReAct	0,633 ($\pm$ 0,105)	0,704 ($\pm$ 0,077)	0,567 ($\pm$ 0,081)	0,515 ($\pm$ 0,082)	0,599 ($\pm$ 0,044)

Źródło: opracowanie własne.

Tabela 5.10 Poprawność według kategorii produktowej

Architektura	Auto	Mieszkanie	Podróż	Łącznie
Deterministyczna	0,700 ($\pm 0,068$)	0,683 ($\pm 0,076$)	0,763 ($\pm 0,052$)	0,716 ($\pm 0,038$)
Planer–wykonawca	0,547 ($\pm 0,079$)	0,660 ($\pm 0,064$)	0,653 ($\pm 0,072$)	0,620 ($\pm 0,042$)
Router–specjalista	0,593 ($\pm 0,071$)	0,690 ($\pm 0,070$)	0,707 ($\pm 0,062$)	0,663 ($\pm 0,039$)
Tablicowa	0,667 ($\pm 0,066$)	0,603 ($\pm 0,077$)	0,630 ($\pm 0,074$)	0,633 ($\pm 0,042$)
Hierarchiczna	0,562 ($\pm 0,077$)	0,570 ($\pm 0,071$)	0,630 ($\pm 0,074$)	0,588 ($\pm 0,043$)
ReAct	0,630 ($\pm 0,078$)	0,463 ($\pm 0,077$)	0,703 ($\pm 0,065$)	0,599 ($\pm 0,044$)

Źródło: opracowanie własne.

Tabela 5.11 Poprawność według ubezpieczyciela

Architektura	Hestia	PZU	Warta	Łącznie
Deterministyczna	0,693 ($\pm 0,072$)	0,793 ($\pm 0,057$)	0,660 ($\pm 0,066$)	0,716 ($\pm 0,038$)
Planer–wykonawca	0,697 ($\pm 0,069$)	0,640 ($\pm 0,066$)	0,523 ($\pm 0,078$)	0,620 ($\pm 0,042$)
Router–specjalista	0,673 ($\pm 0,070$)	0,750 ($\pm 0,060$)	0,567 ($\pm 0,070$)	0,663 ($\pm 0,039$)
Tablicowa	0,683 ($\pm 0,076$)	0,623 ($\pm 0,073$)	0,593 ($\pm 0,068$)	0,633 ($\pm 0,042$)
Hierarchiczna	0,603 ($\pm 0,078$)	0,707 ($\pm 0,065$)	0,453 ($\pm 0,072$)	0,588 ($\pm 0,043$)
ReAct	0,720 ($\pm 0,072$)	0,567 ($\pm 0,080$)	0,510 ($\pm 0,070$)	0,599 ($\pm 0,044$)

Źródło: opracowanie własne.

Szczegółowa analiza wyników poprawności w zestawieniu z wynikami trafności dokumentów (tabela 5.17) uwidacznia interesujący paradoks. Trafność pobierania dla dokumentów ubezpieczyciela Hestia jest systematycznie wyższa (od 0,406 do 0,480) niż dla dokumentów PZU (od 0,270 do 0,342). Jednakże w przypadku wiodących architektur to właśnie pytania dotyczące PZU osiągają najwyższą poprawność w fazie syntezy (0,793 w architekturze deterministycznej), podczas gdy dla Hestii wartości te są niższe (0,693). Sugeruje to, że dokumenty Hestii charakteryzują się formą sprzyjającą wyszukiwaniu wektorowemu (być może lepszą strukturą pojęciową), lecz trudniejszą do interpretacji przez model LLM, podczas gdy dokumenty PZU, mimo trudniejszego pozycjonowania w przestrzeni osadzeń, posiadają zapisy, z których łatwiej wydedukować jednoznaczną i poprawną odpowiedź.

5.3.3 Zwięzłość

Tabele 5.12–5.14 przedstawiają średnią zwięzłość odpowiedzi wraz z odchyleniem standardowym. Wartości są bardziej zbliżone do siebie pomiędzy architekturami niż w przypadku

wierności i poprawności – mieszczą się w przedziale 0,631–0,682. Najwyższą wartość łączną odnotowano dla wariantu ReAct (0,682), a najniższą dla architektury deterministycznej (0,631). W rozbiu według poziomu trudności wartości zwężłości nie wykazują jednoznacznej tendencji. W rozbiu według kategorii i ubezpieczyciela różnice między architekturami pozostają niewielkie.

Tabela 5.12 Zwężłość według poziomu trudności

Architektura	B. proste	Proste	Złożone	B. złożone	Łącznie
Deterministyczna	0,544 ($\pm$ 0,050)	0,630 ($\pm$ 0,033)	0,648 ($\pm$ 0,039)	0,644 ($\pm$ 0,033)	0,631 ($\pm$ 0,019)
Planer–wykonawca	0,667 ($\pm$ 0,050)	0,637 ($\pm$ 0,039)	0,630 ($\pm$ 0,037)	0,689 ($\pm$ 0,038)	0,653 ($\pm$ 0,021)
Router–specjalista	0,700 ($\pm$ 0,047)	0,659 ($\pm$ 0,031)	0,674 ($\pm$ 0,035)	0,581 ($\pm$ 0,037)	0,644 ($\pm$ 0,019)
Tablicowa	0,678 ($\pm$ 0,060)	0,674 ($\pm$ 0,035)	0,626 ($\pm$ 0,039)	0,652 ($\pm$ 0,039)	0,653 ($\pm$ 0,021)
Hierarchiczna	0,700 ($\pm$ 0,047)	0,700 ($\pm$ 0,038)	0,700 ($\pm$ 0,037)	0,622 ($\pm$ 0,040)	0,676 ($\pm$ 0,021)
ReAct	0,611 ($\pm$ 0,060)	0,685 ($\pm$ 0,035)	0,689 ($\pm$ 0,034)	0,696 ($\pm$ 0,041)	0,682 ($\pm$ 0,020)

Źródło: opracowanie własne.

Tabela 5.13 Zwężłość według kategorii produktowej

Architektura	Auto	Mieszkanie	Podróż	Łącznie
Deterministyczna	0,610 ($\pm$ 0,035)	0,630 ($\pm$ 0,034)	0,653 ($\pm$ 0,029)	0,631 ($\pm$ 0,019)
Planer–wykonawca	0,700 ($\pm$ 0,038)	0,613 ($\pm$ 0,032)	0,647 ($\pm$ 0,035)	0,653 ($\pm$ 0,021)
Router–specjalista	0,617 ($\pm$ 0,034)	0,673 ($\pm$ 0,031)	0,643 ($\pm$ 0,033)	0,644 ($\pm$ 0,019)
Tablicowa	0,657 ($\pm$ 0,035)	0,623 ($\pm$ 0,037)	0,680 ($\pm$ 0,034)	0,653 ($\pm$ 0,021)
Hierarchiczna	0,679 ($\pm$ 0,031)	0,640 ($\pm$ 0,038)	0,710 ($\pm$ 0,038)	0,676 ($\pm$ 0,021)
ReAct	0,737 ($\pm$ 0,031)	0,660 ($\pm$ 0,039)	0,650 ($\pm$ 0,033)	0,682 ($\pm$ 0,020)

Źródło: opracowanie własne.

Tabela 5.14 Zwięzłość według ubezpieczyciela

Architektura	Hestia	PZU	Warta	Łącznie
Deterministyczna	0,647 ($\pm$ 0,031)	0,597 ($\pm$ 0,033)	0,650 ($\pm$ 0,035)	0,631 ($\pm$ 0,019)
Planer–wykonawca	0,633 ($\pm$ 0,033)	0,673 ($\pm$ 0,039)	0,653 ($\pm$ 0,035)	0,653 ($\pm$ 0,021)
Router–specjalista	0,660 ($\pm$ 0,027)	0,640 ($\pm$ 0,031)	0,633 ($\pm$ 0,039)	0,644 ($\pm$ 0,019)
Tablicowa	0,697 ($\pm$ 0,035)	0,653 ($\pm$ 0,036)	0,610 ($\pm$ 0,034)	0,653 ($\pm$ 0,021)
Hierarchiczna	0,690 ($\pm$ 0,035)	0,670 ($\pm$ 0,033)	0,670 ($\pm$ 0,040)	0,676 ($\pm$ 0,021)
ReAct	0,647 ($\pm$ 0,031)	0,697 ($\pm$ 0,040)	0,703 ($\pm$ 0,033)	0,682 ($\pm$ 0,020)

Źródło: opracowanie własne.

5.3.4 Trafność dokumentów

Trafność dokumentów oceniano na poziomie pojedynczego cytowania – łącznie 1 786 par pytanie–fragment. Tabele 5.15–5.17 podają średnie arytmetyczne ocen binarnych wraz z odchyleniem standardowym w obrębie odpowiednich podzbiorów. Zaobserwowane wartości trafności są niższe niż w przypadku pozostałych miar jakościowych i mieszczą się w przedziale 0,332–0,364 dla wyników łącznych. Architektura deterministyczna i ReAct osiągnęły tę samą najwyższą wartość (0,364), a tablicowa najniższą (0,332). We wszystkich architekturach trafność przyjmuje wyższe wartości dla pytań bardzo prostych i maleje wraz ze wzrostem poziomu trudności. W rozbiu według ubezpieczyciela najwyższe wartości we wszystkich architekturach odnotowano w grupie Hestia, a najniższe – w grupie Warta.

Tabela 5.15 Trafność dokumentów według poziomu trudności

Architektura	B. proste	Proste	Złożone	B. złożone	Łącznie
Deterministyczna	0,545 ($\pm$ 0,087)	0,394 ($\pm$ 0,048)	0,330 ($\pm$ 0,046)	0,305 ($\pm$ 0,047)	0,364 ($\pm$ 0,026)
Planer–wykonawca	0,500 ($\pm$ 0,091)	0,323 ($\pm$ 0,048)	0,359 ($\pm$ 0,050)	0,315 ($\pm$ 0,049)	0,349 ($\pm$ 0,027)
Router–specjalista	0,441 ($\pm$ 0,085)	0,392 ($\pm$ 0,050)	0,283 ($\pm$ 0,045)	0,370 ($\pm$ 0,048)	0,358 ($\pm$ 0,026)
Tablicowa	0,500 ($\pm$ 0,086)	0,307 ($\pm$ 0,046)	0,337 ($\pm$ 0,048)	0,298 ($\pm$ 0,045)	0,332 ($\pm$ 0,026)
Hierarchiczna	0,515 ($\pm$ 0,087)	0,330 ($\pm$ 0,046)	0,333 ($\pm$ 0,047)	0,314 ($\pm$ 0,046)	0,344 ($\pm$ 0,026)
ReAct	0,500 ($\pm$ 0,134)	0,370 ($\pm$ 0,066)	0,419 ($\pm$ 0,075)	0,219 ($\pm$ 0,073)	0,364 ($\pm$ 0,040)

Źródło: opracowanie własne.

Tabela 5.16 Trafność dokumentów według kategorii produktowej

Architektura	Auto	Mieszkanie	Podróż	Łącznie
Deterministyczna	0,315 ($\pm$ 0,044)	0,386 ($\pm$ 0,046)	0,391 ($\pm$ 0,047)	0,364 ($\pm$ 0,026)
Planer–wykonawca	0,323 ($\pm$ 0,048)	0,383 ($\pm$ 0,047)	0,337 ($\pm$ 0,047)	0,349 ($\pm$ 0,027)
Router–specjalista	0,352 ($\pm$ 0,046)	0,372 ($\pm$ 0,045)	0,349 ($\pm$ 0,046)	0,358 ($\pm$ 0,026)
Tablicowa	0,315 ($\pm$ 0,044)	0,342 ($\pm$ 0,044)	0,339 ($\pm$ 0,045)	0,332 ($\pm$ 0,026)
Hierarchiczna	0,309 ($\pm$ 0,044)	0,331 ($\pm$ 0,043)	0,394 ($\pm$ 0,047)	0,344 ($\pm$ 0,026)
ReAct	0,318 ($\pm$ 0,070)	0,372 ($\pm$ 0,074)	0,393 ($\pm$ 0,065)	0,364 ($\pm$ 0,040)

Źródło: opracowanie własne.

Tabela 5.17 Trafność dokumentów według ubezpieczyciela

Architektura	Hestia	PZU	Warta	Łącznie
Deterministyczna	0,460 ($\pm$ 0,047)	0,336 ($\pm$ 0,044)	0,294 ($\pm$ 0,044)	0,364 ($\pm$ 0,026)
Planer–wykonawca	0,406 ($\pm$ 0,048)	0,333 ($\pm$ 0,048)	0,304 ($\pm$ 0,046)	0,349 ($\pm$ 0,027)
Router–specjalista	0,436 ($\pm$ 0,047)	0,324 ($\pm$ 0,046)	0,313 ($\pm$ 0,043)	0,358 ($\pm$ 0,026)
Tablicowa	0,425 ($\pm$ 0,047)	0,270 ($\pm$ 0,042)	0,301 ($\pm$ 0,043)	0,332 ($\pm$ 0,026)
Hierarchiczna	0,416 ($\pm$ 0,046)	0,342 ($\pm$ 0,045)	0,274 ($\pm$ 0,042)	0,344 ($\pm$ 0,026)
ReAct	0,480 ($\pm$ 0,071)	0,302 ($\pm$ 0,063)	0,300 ($\pm$ 0,072)	0,364 ($\pm$ 0,040)

Źródło: opracowanie własne.

5.4. Parametry konfiguracyjne eksperymentu

Sekcja dokumentuje wartości parametrów sterujących przebiegiem uruchomienia. Pełną konfigurację zawierają moduły `sources/config/app.py` i `sources/config/graph.py`; poniższe zestawienie obejmuje wyłącznie ustawienia użyte w uruchomieniu 20260313_200511.

5.4.1 Parametry modeli językowych

Wszystkie role pomocnicze – parser, router, planer, kontroler ReAct, rewriter, dekompozytor i ewaluator – zrealizowano z wykorzystaniem modelu gpt-5. Ten sam model zastosowano do syntezy odpowiedzi końcowej. Każdą rolę skonfigurowano z temperaturą równą zero, limitem 120 s na pojedyncze wywołanie i mechanizmem ponowień z wykładniczym opóźnieniem od 1 do 60 s. Tabela 5.18 zestawia pełną konfigurację.

Tabela 5.18 Parametry modeli językowych

Parametr	Wartość	Rola
model_name	gpt-5	model syntezy odpowiedzi
parser_model_name	gpt-5	parser pytań
router_model_name	gpt-5	router specjalisty
planner_model_name	gpt-5	planer kroków
controller_model_name	gpt-5	kontroler ReAct
rewrite_model_name	gpt-5	przepisywanie zapytań
decomposer_model_name	gpt-5	dekompozycja pytań
evaluation_model_name	gpt-5	sędzia jakości
temperature	0,0	determinizm wyjścia
retry_max_attempts	5	liczba ponowień
retry_min_wait	1,0 s	minimalna pauza między ponowieniami
retry_max_wait	60,0 s	maksymalna pauza między ponowieniami
llm_request_timeout	120,0 s	limit pojedynczego wywołania
max_concurrent_api_calls	16	limit równoległości wywołań API

Źródło: opracowanie własne na podstawie kodu źródłowego.

5.4.2 Parametry wyszukiwania

Wyszukiwanie wektorowe wykonano na kolekcji `insurance_documents` w bazie Qdrant, z wykorzystaniem modelu osadzeń `text-embedding-3-large` o wymiarze 3072. Etap re-rankingu wyłączono we wszystkich architekturach; węzeł rerankera pozostał jednak obecny w każdym grafie. Tabela 5.19 prezentuje pełną konfigurację warstwy wyszukiwania wraz z nominalnymi parametrami rerankera.

Tabela 5.19 Parametry wyszukiwania

Parametr	Wartość	Opis
embedding.model_name	text-embedding-3-large	model osadzeń
embedding.dimension	3072	wymiar wektora
qdrant.collection_name	insurance_documents	nazwa kolekcji
top_k	5	liczba pobieranych fragmentów
evidence_top_k	4	liczba dowodów przekazywanych do syntezy
reranking.enabled	False	reranking wyłączony w eksperymencie
reranking.score_threshold	0,3	próg odrzucenia fragmentu
reranking.top_k_before	20	wejście do rerankera
reranking.top_k_after	5	wyjście z rerankera

Źródło: opracowanie własne na podstawie kodu źródłowego.

5.4.3 Parametry fragmentacji

Dokumenty OWU i IPID przetwarzano przy użyciu odrębnych profili fragmentacji. W obu przypadkach zastosowano podział po nagłówkach Markdown poziomów 1–3, a następnie rekurencyjną fragmentację większych sekcji. Parametry wyrażone w liczbie znaków zestawiono w tabeli 5.20; zastosowanie krótszych fragmentów dla IPID wynikało z większej gęstości informacyjnej tego formatu.

Tabela 5.20 Parametry fragmentacji dokumentów

Parametr	OWU	IPID	Opis
chunk_size	512	384	docelowy rozmiar fragmentu
chunk_overlap	64	48	nakładanie sąsiednich fragmentów
min_chunk_size	64	48	minimalny dopuszczalny rozmiar

Źródło: opracowanie własne na podstawie kodu źródłowego.

5.4.4 Parametry architektury

Architektury iteracyjne wykorzystują jawne ograniczenia liczby kroków; architektura hierarchiczna ogranicza dekompozycję do trzech podpytań bezpośrednio w kodzie węzła dekomponera. Architektury deterministyczna i router-specjalista nie mają parametrów sterujących właściwych

dla danej topologii. Tabela 5.21 zestawia wartości dla wariantów, w których takie parametry występują.

Tabela 5.21 Parametry sterujące architektur

Architektura	Parametr	Wartość	Znaczenie
ReAct	max_react_iterations	15	limit iteracji pętli obserwacyjnej
Tablicowa	max_blackboard_iterations	15	limit cykli dyspozytora
Hierarchiczna	max. liczba podpytań	3	przycięcie listy sub_questions

Źródło: opracowanie własne na podstawie kodu źródłowego.

5.4.5 Parametry wykonania

Komponent wykonawczy pobierał parametry współbieżności z `ConcurrencyConfig` i inicjował pulę 32 wątków obsługujących pełną macierz par pytanie–architektura. Tabela 5.22 przedstawia liczebności i ustawienia dla uruchomienia 20260313_200511.

Tabela 5.22 Parametry wykonania eksperymentu

Parametr	Wartość	Opis
max_workers	32	rozmiar puli wątków roboczych
Liczba pytań testowych	90	zestaw pytania_20251130.csv
Liczba architektur	6	komplet ALL_PATTERNS
Łączna liczba uruchomień	540	macierz 90×6

Źródło: opracowanie własne na podstawie kodu źródłowego.

6. Wnioski i zakończenie

Rozdział podsumowuje pracę na podstawie wyników z rozdziale 5. Kolejne sekcje obejmują analizę ilościową czterech miar jakości i zestawu miar operacyjnych, analizę jakościową typowych wzorców niepowodzeń, rozstrzygnięcie głównego problemu badawczego, odpowiedzi na pytania badawcze oraz ocenę realizacji celów badawczych sformułowanych w rozdziale 1, kierunki dalszych badań, rekomendacje dla praktyków oraz podsumowanie pracy.

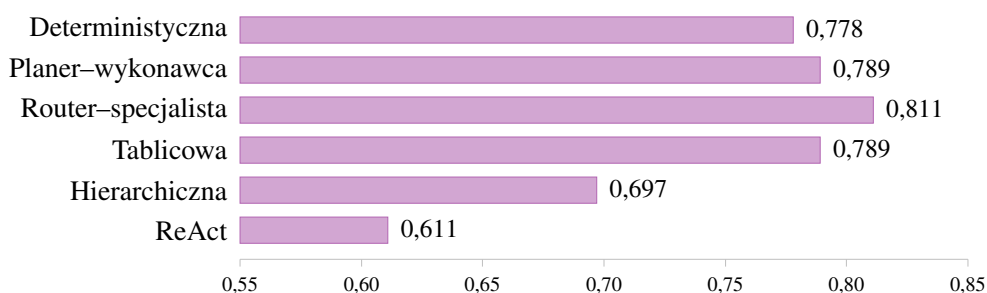
6.1. Wnioski ilościowe

Analiza wszystkich miar wskazuje na podział sześciu architektur na dwie grupy. Pierwszą tworzą architektury: deterministyczna, planer–wykonawca, router–specjalista i tablicowa, których wyniki różnią się nieznacznie. Drugą stanowią dwa warianty iteracyjne o wysokim narzucie operacyjnym: hierarchiczna i ReAct, które osiągają niższe wyniki w większości miar i generują wyższe koszty.

6.1.1 Wierność

Wartości wierności wynoszą od 0,611 do 0,811. Cztery architektury z pierwszej grupy osiągają wyniki w przedziale 0,778–0,811, podczas gdy architektura hierarchiczna uzyskuje 0,697, a ReAct 0,611 (tabela 5.6). Maksymalna różnica wynosi 0,200 punktu i wynika z odstępstwa wariantów iteracyjnych od pierwszej grupy architektur.

Wykres 6.1 Wierność według architektury



Źródło: opracowanie własne.

Najwyższy wynik routera–specjalisty (0,811) wynika z mechanizmu klasyfikacji pytania do jednej z ośmiu kategorii semantycznych przed pobraniem fragmentów. Klasyfikator zawężył obszar wyszukiwania do fragmentów właściwych dla danego typu pytania, ograniczając liczbę kontekstów semantycznie zbliżonych, lecz produktowo nieadekwatnych. Różnicę tę zaobserwowano dla pytań bardzo złożonych (0,815 wobec 0,704 architektury deterministycznej), natomiast nie występuje ona dla pytań prostszych, dla których przestrzeń wyszukiwania jest jednoznaczna bez trasowania.

Niższa wierność w architekturze hierarchicznej (0,697) wynika z dwóch czynników. Pierwszym jest bezwarunkowe uruchomienie dekompozytora: pytania jednoaspektowe są rozkładane na podpytania, co skutkuje wzrostem liczby kontekstów dostarczanych do syntezy. Drugim jest węzeł scalający, który uzupełnia odpowiedzi z podpytań elementami z wiedzy parametrycznej modelu zamiast wyłącznie z pobranych fragmentów.

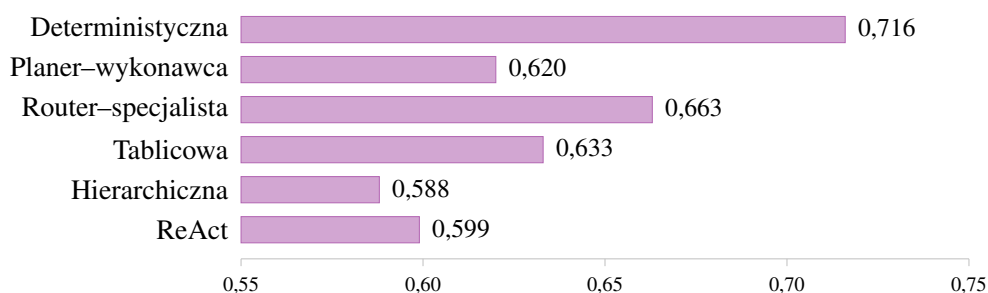
Architektura ReAct uzyskuje najniższą wierność (0,611) z powodu dryfu kontrolera w pętli myśl–działanie. Po kilku iteracjach przepisane zapytania stopniowo poszerzają zakres tematyczny: zapytania o konkretny produkt są zastępowane zapytaniami ogólniejszymi, zwracającymi fragmenty z innych ubezpieczycieli i innych produktów. Wygenerowane odpowiedzi są spójne lokalnie, lecz odnoszą się do produktu innego niż przedmiot pytania.

Cztery architektury z pierwszej grupy różnią się między sobą o 0,033 punktu. Przy próbie 90 pytań różnica ta nie jest statystycznie istotna; jedyną parą wykazującą istotną różnicę jest router–specjalista i ReAct.

6.1.2 Poprawność

Najwyższą poprawność osiąga architektura deterministyczna (0,716). Drugą pozycję zajmuje router–specjalista (0,663), natomiast architektura hierarchiczna uzyskuje najniższy wynik: 0,588 (tabela 5.9).

Wykres 6.2 Poprawność według architektury



Źródło: opracowanie własne.

Wyższy wynik architektury deterministycznej jest następstwem braku węzłów decyzyjnych: pojedyncza ścieżka wykonania nie wprowadza ryzyka błędnej klasyfikacji, dekompozycji ani trawowania. Każdy taki węzeł w pozostałych architekturach jest potencjalnym źródłem rozbieżności z odpowiedzią referencyjną. W przypadku routera–specjalisty część pytań kierowana jest do podpotoku niezgodnego z referencyjną klasyfikacją; wybrany podpotok pomija klauzule istotne dla pytania, mimo że zwracane fragmenty są wewnętrznie spójne.

Rozbieżność między wiernością a poprawnością wynika z definicji obu miar. Odpowiedź może być wierna wobec pobranych fragmentów, a jednocześnie niezgodna z odpowiedzią referencyjną, jeśli pobrane fragmenty opisują niekompletny stan faktyczny. Router–specjalista osiąga wyższą wierność, lecz niższą poprawność niż architektura deterministyczna: wąskie wyszukiwanie ogranicza ryzyko halucynacji, lecz zwiększa ryzyko pominięcia kontekstu z innej

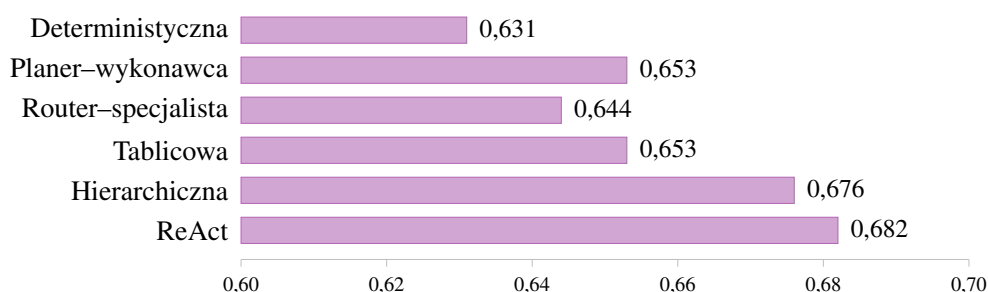
części dokumentu.

W obu wariantach iteracyjnych zaobserwowano spadek poprawności dla pytań bardzo złożonych (hierarchiczna 0,485, ReAct 0,515). Przy tym poziomie trudności pytanie wymaga zestawienia kilku klauzul z różnych części dokumentu – oba kontrolery pobierają te fragmenty, lecz węzeł syntezy nie scala ich poprawnie przy nadmiarze kontekstu.

6.1.3 Zwięźłość

Wyniki w zakresie zwięźłości wykazują najmniejszą wariancję. Wszystkie architektury osiągają wartości w przedziale 0,631–0,682, a maksymalna różnica wynosi 0,051 punktu (tabela 5.12). Wskazuje to, że dominującym czynnikiem długości i gęstości informacyjnej odpowiedzi jest wspólny prompt syntezy, a nie topologia sterowania.

Wykres 6.3 Zwięźłość według architektury



Źródło: opracowanie własne.

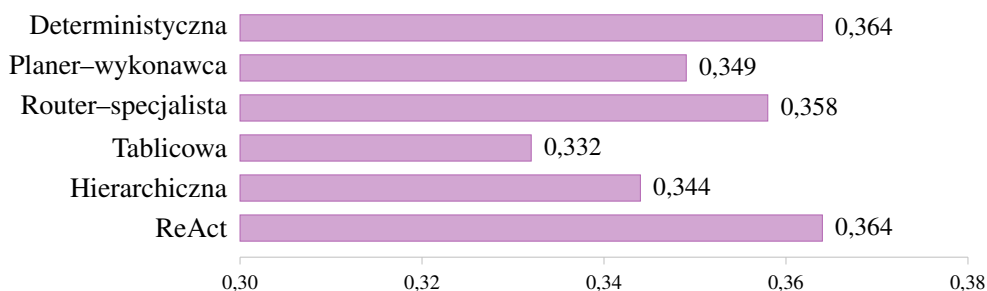
Najwyższy wynik ReAct (0,682) wynika z mechanizmu obserwacji: kontroler filtruje fragmenty zwrócone przez wcześniejsze wywołania narzędzi i przekazuje do syntezy mniejszy zbiór dowodów (średnio 1,59 cytowania w odpowiedzi końcowej, tabela 5.5). Najniższy wynik architektury deterministycznej (0,631) wynika z odwrotnego zjawiska: synteza otrzymuje pełny zestaw 4,28 fragmentów po reranku i uwzględnia każdy z nich w odpowiedzi wraz z cytowaniem.

Praktyczne znaczenie tej miary jest jednak ograniczone w kontekście dokumentów regulacyjnych. Zbyt zwięzła odpowiedź może pomijać warunek dodatkowy, stanowiący integralną część prawidłowej odpowiedzi. Wzrost zwięźłości nie powinien być traktowany jako poprawa jakości bez równoczesnej weryfikacji poprawności i kompletności.

6.1.4 Trafność dokumentów

Wartości trafności dokumentów są niskie we wszystkich architekturach – wartości łączne wynoszą od 0,332 do 0,364 (tabela 5.15). Rozpiętość między architekturami wynosi 0,032 punktu, co wskazuje, że ograniczenie leży poza warstwą orkiestracji.

Wykres 6.4 Trafność dokumentów według architektury



Źródło: opracowanie własne.

Główną przyczyną tego zjawiska we wszystkich wariantach jest sposób reprezentacji wektorowej dokumentów ubezpieczeniowych. Klauzule ochrony i klauzule wyłączeń zajmują w przesłaniu osadzeń różne pozycje względem typowego pytania. Pytania klientów są formułowane afirmatywnie („czy ubezpieczenie obejmuje X?”), a wyszukiwanie kosinusowe faworyzuje fragmenty o zbliżonej polaryzacji. Klauzule wyłączeń, formułowane przez negację, są systematycznie przesuwane w dół rankingu, mimo że niejednokrotnie zawierają kluczową odpowiedź. Trafność wykazuje ponadto tendencję spadkową wraz z poziomem trudności pytania: z zakresu 0,441–0,545 dla pytań bardzo prostych do 0,219–0,370 dla bardzo złożonych.

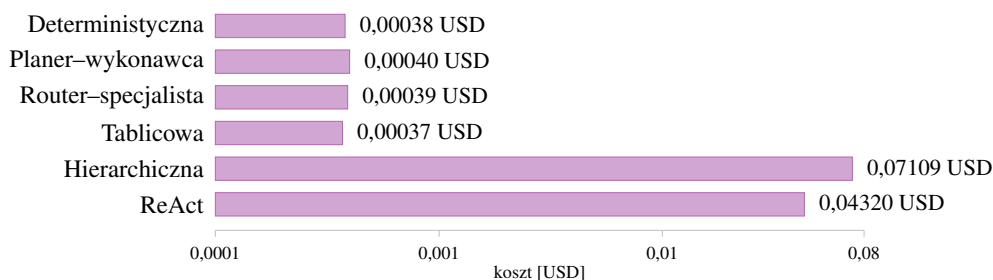
Dodatkowo zaobserwowano swoisty paradoks pomiędzy trafnością pobierania a końcową poprawnością odpowiedzi w zależności od dokumentów ubezpieczyciela. Dokumenty Hestii osiągały systematycznie wyższą trafność wyszukiwania (do 0,480) niż dokumenty PZU (ok. 0,33), co sugeruje strukturę silnie sprzyjającą reprezentacji wektorowej. Z kolei najwyższą poprawność końcowej syntezy (np. 0,793 w ariancie deterministycznym) wiodące architektury odnotowały dla PZU, a nie Hestii. Oznacza to, że wysoka trafność fragmentów nie implikuje wprost sukcesu informacyjnego – dokumenty trudniejsze do wyszukania (PZU) okazały się dla modelu językowego językowo łatwiejsze do poprawnej interpretacji i złożenia ostatecznej odpowiedzi.

Drugim ograniczeniem jest oddzielenie sekcji definicyjnych od klauzul, w których definiowane terminy są używane. Pojęcia takie jak suma ubezpieczenia, franszyza redukcyjna czy wartość rynkowa pojazdu są definiowane w paragrafach wstępnych i używane w odległych częściach dokumentu bez powtórzenia definicji. Pojedyncze zapytanie pozwala zidentyfikować klauzulę użycia, lecz omija sekcję definicji, co obniża trafność cytowania bez naruszenia wierności odpowiedzi.

6.1.5 Koszty operacyjne

Wariancja kosztów operacyjnych między architekturami jest większa niż w przypadku jakiegokolwiek miary jakości. Średni koszt jednego uruchomienia wynosi od 0,00037 do 0,00040 USD dla czterech architektur o stałym lub płytko sterowanym przebiegu i wzrasta do 0,04320 USD dla ReAct oraz 0,07109 USD dla architektury hierarchicznej (tabela 5.3).

Wykres 6.5 Koszt na uruchomienie w skali logarytmicznej



Źródło: opracowanie własne.

Dla architektury hierarchicznej koszt jest funkcją rekurencyjnego rozkładu pytań. Każde podpytanie generuje własny pełny przebieg pobierania, syntezy i oceny, a łączna liczba wywołań narzędzi osiąga średnio 1 066 na uruchomienie – przy odchyleniu standardowym 2 568, co wskazuje na wysoką wariację kosztową tej architektury. Zarejestrowane ścieżki wykonania wskazują, że dla części pytań długość kontekstu zbliżała się do progu okna modelu, co dodatkowo obniżało jakość syntezy.

W architekturze ReAct koszt rośnie wraz z liczbą iteracji kontrolera. Ustalony limit 15 iteracji nie został osiągnięty w żadnym z przebiegów – maksymalna odnotowana liczba iteracji wyniosła 11, a 6 przebiegów osiągnęło 10 lub więcej iteracji. Każda iteracja uruchamiała odrębne wywołanie narzędzia pobierającego fragmenty – stąd średnio 295,7 wywołań na pytanie wobec 7,2 dla planera–wykonawcy. Architektura ta charakteryzuje się jednocześnie największą wariacją opóźnienia, co w środowisku produkcyjnym stwarza ryzyko przekroczenia limitów czasowych interfejsu użytkownika. Podobnie jak w przypadku architektury hierarchicznej, wariacja zużycia tokenów w architekturze ReAct przekracza wartość jej średniej (odchylenie standardowe 419 373 przy średniej 391 935 tokenów wejściowych). Stan ten jednoznacznie wskazuje na prawostronnie skośny rozkład kosztu operacyjnego – znaczna większość pytań jest rozwiązywana bez trudu, jednak sporadyczne wpadnięcie agenta w wielokrotne pętle lub powielane halucynacje pochłania skrajne zasoby obliczeniowe i staje się źródłem potężnego zawyżenia średnich wydatków na utrzymanie aplikacji.

Nie stwierdzono korelacji między kosztem a jakością. Warianty iteracyjne, wymagające przetworzenia ponad stukrotnie większej liczby tokenów, osiągają niższe wyniki w każdej z czterech miar jakości niż najtańszy wariant z pierwszej grupy.

6.2. Wnioski jakościowe

Analiza jakościowa rozszerza pomiary statystyczne o identyfikację powtarzalnych wzorców błędów wynikających ze struktury dokumentów i specyfiki działania poszczególnych architektur. Badaniu poddano przebiegi o najniższych wynikach wierności i poprawności, wyodrębnione na podstawie plików wynikowych opisanych w sekcji 5.1.3.

6.2.1 Charakterystyka niepowodzeń

Analiza odpowiedzi o najniższej wierności i poprawności pozwala wyodrębnić powtarzalne mechanizmy błędów. Część odpowiedzi poprawnie przytacza dany fakt, lecz omija klauzulę wyłączenia lub warunek ograniczający ochronę – ich treść pozostaje zgodna z kontekstem, lecz niekompletna wobec odpowiedzi referencyjnej. W pozostałych przebiegach zasadnicza treść odpowiedzi jest poprawna, jednak dołączony szczegół – na przykład wartość liczbową lub nazwę wariantu produktu – nie posiada pokrycia w pobranych fragmentach. Trzecią grupę stanowią odpowiedzi, w których pojedyncze stwierdzenia wynikają z pobranych fragmentów, lecz ich kombinacja odnosi się do różnych produktów lub ubezpieczycieli, co prowadzi do błędnego przedstawienia zakresu ochrony.

Zidentyfikowane profile błędów są zróżnicowane w zależności od architektury. W przebiegach wariantów deterministycznego i tablicowego dominują odpowiedzi niepełne – klauzule wyłączeń są omijane przy jednoczesnej poprawności pozostałej treści. W przebiegach routera–specjalisty i planera–wykonawcy częściej obserwuje się uzupełnienie poprawnej treści o niepotwierdzone szczegóły. Architektury hierarchiczna i ReAct wykazują oba te zjawiska – rozbudowują odpowiedź i jednocześnie łączą fragmenty dotyczące różnych produktów. Uzyskane wyniki pozwalają zgrupować badane architektury w trzy pary na podstawie dominującego typu błędu: deterministyczna i tablicowa generują odpowiedzi niepełne, router–specjalista i planer–wykonawca odpowiedzi nadbudowane, natomiast hierarchiczna i ReAct odpowiedzi o niejednorodnym kontekście produktowym.

Z perspektywy regulacyjnej rodzaj generowanych błędów posiada istotne implikacje praktyczne. Odpowiedź niepełna sygnalizuje niedobór informacji, co umożliwia użytkownikowi sformułowanie pytań doprecyzujących. Odpowiedzi z nadbudowaną treścią lub połączonymi fragmentami stwarzają pozory kompletności, mimo że przekazują błędne informacje o zakresie ochrony. Powyższe obserwacje uzasadniają wybór architektur o prostszym i bardziej deterministycznym przepływie sterowania w zastosowaniach, w których ryzyko wprowadzenia w błąd użytkownika stanowi koszt wyższy niż dostarczenie odpowiedzi niekompletnej.

6.2.2 Systematyczne pomijanie klauzul wyłączeń

Najczęstszym mechanizmem odpowiedzi niepełnych jest pomijanie klauzul wyłączeń przez warstwę pobierania. Dokumenty OWU charakteryzują się asymetryczną strukturą językową: klauzule pokrycia zaczynają się od sformułowań afirmatywnych („ubezpieczyciel pokrywa szkody powstałe wskutek...”), a klauzule wyłączeń od sformułowań negatywnych („ubezpieczyciel nie ponosi odpowiedzialności, jeśli...”). Zapytania użytkowników sformułowane są przeważnie w formie twierdzącej, w wyniku czego podobieństwo kosinusowe między pytaniem a klauzulami wyłączeń jest niższe niż podobieństwo względem klauzul ochrony.

Omawiane zjawisko występuje we wszystkich badanych architekturach. Ograniczenie to wynika z metody reprezentacji wektorowej i nie zależy od przyjętej topologii sterowania. Warianty iteracyjne, mimo większej liczby wywołań narzędzi, nie redukują tego zjawiska – dryf

semantyczny powoduje, że kolejne zapytania pozostają w afirmatywnym rejestrze i jedynie poszerzają jego zakres.

6.2.3 Niedobór kontekstu definicyjnego

Drugim mechanizmem jest pominięcie przez moduł wyszukiwania sekcji definiującej termin użyty w klauzuli docelowej. W polskich dokumentach OWU definicje znajdują się w sekcjach wstępnych, natomiast zdefiniowane terminy występują w klauzulach umieszczonych w dalszych częściach tekstu. Zapytanie użytkownika jest semantycznie bliższe klauzuli użycia, wobec czego wyszukiwanie zwraca tę klauzulę i pomija sekcję definicyjną. Odpowiedź pozostaje wierna wobec pobranego fragmentu, jednak brak kontekstu pojęciowego skutkuje odpowiedzią niepełną z perspektywy użytkownika.

Wzorzec ten odpowiada za znaczną część rozbieżności między wiernością a poprawnością – architektura uzyskuje wysoką wierność przy jednocześnie niskiej poprawności, ponieważ pobrany kontekst jest niekompletny względem odpowiedzi referencyjnej.

6.2.4 Bezwarunkowa dekompozycja w architekturze hierarchicznej

W architekturze hierarchicznej zaobserwowano powtarzalny wzorzec bezwarunkowej dekompozycji pytań jednoaspektowych. Węzeł dekompozytora uruchamiany jest niezależnie od charakteru pytania, przez co nawet proste pytania są rozkładane na podpytania. Każde podpytanie inicjuje osobny przebieg pobierania, a węzeł scalający uzupełnia wynikową odpowiedź elementami z wiedzy parametrycznej modelu tam, gdzie wśród pobranych fragmentów brakuje bezpośredniej odpowiedzi.

Skutkuje to najniższą wiernością dla pytań bardzo prostych: 0,556 wobec 0,889 architektury deterministycznej. Węzeł dekompozytora powinien działać warunkowo, po uprzedniej klasyfikacji pytania według stopnia złożoności – w zbadanej implementacji warunek ten nie był spełniony.

6.2.5 Rozszerzanie zakresu wyszukiwania w architekturze ReAct

W architekturze ReAct zaobserwowano zjawisko narastającego rozszerzania zakresu wyszukiwania w kolejnych iteracjach pętli. W przypadku otrzymania niejednoznacznej obserwacji kontroler generuje zapytanie ogólniejsze niż poprzednie. W efekcie zapytania o konkretny produkt zaczynają obejmować produkty powiązanych ubezpieczycieli, zwracając fragmenty tematycznie zbliżone, lecz produktowo odmienne. Wygenerowane odpowiedzi zawierają stwierdzenia poparte poszczególnymi fragmentami, lecz ich kombinacja łączy postanowienia różnych polis. Wyjątkową podatność na ten błąd zaobserwowano dla pytań z kategorii ubezpieczeń mieszkaniowych, w której wierność ReAct drastycznie spadła do zaledwie 0,433 – zjawisko dryfu uderza najsilniej tam, gdzie pojęcia dziedzinowe posiadają najszerszy i podatny na wieloznaczność kontekst.

Zjawisko to jest trudne do wykrycia przez automatyczną ocenę opartą na weryfikacji obecności twierdzeń w kontekście – odpowiedź nie wykracza formalnie poza pobrany zbiór, jednakże

pozostaje niepoprawna z perspektywy zapytania.

6.2.6 Niespójność autorytetu między OWU a IPID

W kilku zarejestrowanych przypadkach wyszukiwanie zwróciło jednocześnie fragment IPID z zaokrągloną wartością sumy ubezpieczenia oraz fragment OWU z dokładną wartością umowną. Selektor dowodów, działający na podobieństwie kosinusowym, ocenił fragment IPID jako bliższy pytaniu – formularz IPID jest zwięzły i osiąga wyższe podobieństwo do prostych zapytań użytkowników. Synteza odtworzyła wartość zaokrągloną. Odpowiedź była wierna wobec pobranego kontekstu, lecz niezgodna z wiążącym dokumentem umownym.

Mechanizm ten jest niezależny od architektury sterowania – wystąpił we wszystkich sześciu wariantach. Wynika on z umieszczenia obu typów dokumentów w jednej kolekcji wektorowej bez metadanowego oznaczenia hierarchii autorytetu.

6.2.7 Samokorekta w architekturze tablicowej

W architekturze tablicowej zaobserwowano mechanizm samokorekty. W pojedynczych przypadkach moduł przepisywania zapytań zwrócił ucięty wynik (odpowiedź modelu osiągnęła limit tokenów przed zamknięciem formatu JSON), pozostawiając pole `rewritten_queries` stanu grafu puste. W architekturze deterministycznej taka sytuacja propagowała się bez korekty. W architekturze tablicowej dyspozytor wykrył puste pole i ponownie wywołał moduł przepisywania.

Mechanizm ten wynika z ogólnej reguły dyspozytora: w przypadku pustego pola, należy uruchomić odpowiedniego agenta. Jest to przykład samokorekty wynikającej z topologii, a nie z jawnej obsługi błędów. W środowisku produkcyjnym taka odporność na awarie narzędzi posiada znaczną wartość operacyjną, nieproporcjonalną do kosztu jej zaimplementowania.

6.3. Rozstrzygnięcie problemu badawczego i realizacja celów

Sekcja zestawia sformułowane wnioski najpierw z głównym problemem badawczym, a następnie z pytaniami badawczymi PB1–PB4 i celami CB1–CB4 z rozdziale 1.

6.3.1 Rozstrzygnięcie głównego problemu badawczego

Główny problem badawczy pracy wynikał z braku kontrolowanego porównania topologii agentowych oraz pytania, czy bardziej złożone systemy wieloagentowe oferują lepszą jakość odpowiedzi niż prostsze układy jednoagentowe w identycznym środowisku testowym. Uzyskane wyniki nie pozwalają na sformułowanie jednoznacznej odpowiedzi. Wskazują one natomiast, że sama opozycja jednoagentowe–wieloagentowe nie stanowi wystarczającego kryterium oceny jakości: złożoność topologii przynosi korzyść tylko wtedy, gdy poprawia dobór kontekstu lub sterowanie przebiegiem bez wprowadzania nadmiernego narzutu iteracyjnego.

Najwyższą wierność uzyskał router-specjalista, co wskazuje, że dodatkowy mechanizm trasowania może poprawić jakość odpowiedzi w najtrudniejszych przypadkach. Jednocześnie

najwyższą poprawność osiągnęła architektura deterministyczna, a najbardziej kosztowne warianty iteracyjne – hierarchiczna i ReAct – nie uzyskały lepszych wyników pomimo większej liczby kroków rozumowania. Rozstrzygnięcie głównego problemu badawczego ma zatem charakter warunkowy: większa złożoność architektury nie gwarantuje automatycznej przewagi jakościowej, lecz wybrane mechanizmy sterowania, w tym szczególnie trasowanie zapytań, mogą być użyteczne, jeśli pozostają ściśle powiązane z warstwą pobierania dokumentów.

6.3.2 Wpływ topologii na jakość odpowiedzi

Wartości wierności dla poszczególnych architektur zawierają się w przedziale 0,611–0,811, a wartości poprawności w przedziale 0,588–0,716. Najwyższą wierność osiąga router–specjalista (0,811), najwyższą poprawność architektura deterministyczna (0,716). Żadna architektura nie dominuje jednocześnie we wszystkich czterech miarach; przewagi są specyficzne dla miary i poziomu trudności pytania.

Hipoteza PB1, zakładająca przewagę bardziej złożonych topologii, znajduje tylko częściowe potwierdzenie. Najwyższą wierność osiąga router–specjalista, a trzy z pięciu topologii bardziej złożonych niż deterministyczna uzyskują wierność nie niższą od wariantu bazowego. Jednocześnie hierarchiczna i ReAct pokazują, że sama złożoność orkiestracji nie gwarantuje wzrostu jakości: ich wyniki są niższe mimo większej liczby kroków i wywołań narzędzi.

6.3.3 Kompromis między jakością a kosztem

W ramach analizowanego zadania nie zaobserwowano kompromisu między jakością odpowiedzi a kosztem operacyjnym. Cztery architektury o stałym lub płytko sterowanym przebiegu uzyskują wyższe wyniki w większości miar jakości przy koszcie 0,00037–0,00040 USD na pytanie. Koszty wariantów iteracyjnych wynoszą odpowiednio 0,04320 USD i 0,07109 USD na uruchomienie, co wiąże się jednocześnie z niższą wiernością i poprawnością. Stosunek kosztu architektury hierarchicznej do deterministycznej wynosi 187, ReAct do deterministycznej – 114.

Na poziomie pytań bardzo złożonych router–specjalista i tablicowa uzyskują po 0,815 wierności, natomiast wynik architektury hierarchicznej spada do 0,593. Hipoteza PB2 o braku proporcjonalnego wzrostu jakości wraz z kosztem orkiestracji znajduje potwierdzenie, przy czym wyniki doprecyzowują ją: największy narzut kosztowy wynika z iteracyjnej kontroli przebiegu, a nie z samego wprowadzenia dodatkowej roli lub klasyfikatora.

6.3.4 Wpływ planowania, trasowania i dekompozycji

Dodanie warstwy planowania przyniosło nieoczekiwany paradoks wydajnościowy – architektura planera–wykonawcy okazała się najszybsza czasowo dzięki trafnej redukcji wywołań narzędzi (7,2 wobec 10,0 w wariancie deterministycznym). Sukces ten jest jednak obciążony anomalią iluzji filtrowania dowodów: po ocenie trafności w stanie pozostaje zaledwie 0,31 fragmentu na zapytanie, podczas gdy węzeł syntezy wciąż generuje 3,38 cytowania, jawnie ignorując wyniki sędziego.

Z kolei mechanizmy planowania i bezwarunkowej dekompozycji nie zapewniają systematycznej poprawy wyników dla pytań złożonych ani bardzo złożonych. Najwyższe wyniki dla pytań bardzo złożonych osiągają architektury bez iteracyjnej kontroli (router–specjalista i tablicowa, po 0,815 wierności). Spośród mechanizmów analizowanych w PB3 jedynie trasowanie wykazuje ograniczoną przewagę – na poziomie pytań bardzo złożonych router–specjalista osiąga 0,815 wobec 0,704 w architekturze deterministycznej (różnica 0,111 punktu).

Dekompozycja hierarchiczna pogarsza wynik na każdym poziomie trudności, najwyraźniej dla pytań prostych, z przyczyny opisanej w sekcji 6.2.4.

6.3.5 Wzorce niepowodzeń

Architektury wykazują odmienne profile niepowodzeń, opisane w sekcji 6.2.1. Architektury deterministyczna i tablicowa generują błędy polegające na pomijaniu klauzul wyłączeń, router–specjalista i planer–wykonawca – na uzupełnianiu poprawnego rdzenia o niepotwierdzony szczegół, natomiast hierarchiczna i ReAct – łączą oba zjawiska, dodatkowo mieszając fragmenty z różnych produktów. Hipoteza o zróżnicowanych profilach błędów znajduje pełne potwierdzenie.

6.3.6 Odpowiedzi na pytania badawcze

Wszystkie cztery pytania badawcze sformułowane w rozdziale 1 zostały empirycznie rozstrzygnięte. Pytanie PB1 rozstrzygnięto częściowo względem hipotezy o przewadze bardziej złożonych topologii: najwyższą wierność uzyskał router–specjalista, lecz rozkład wyników wskazuje, że przewaga nie wynika z samej złożoności, lecz z konkretnego mechanizmu sterowania pobieraniem. Rozstrzygnięcie PB2 oparto na obserwacji, że warianty iteracyjne generują koszt rzędu 100–187 razy wyższy niż architektury o stałym lub płytko sterowanym przebiegu, przy jednocześnie niższych wynikach w każdej z miar jakości – w zbadanym zadaniu zwiększanie nakładu obliczeniowego nie przekłada się proporcjonalnie na jakość. Pytanie PB3 rozstrzygnięto częściowo: trasowanie wnosi mierzalną, choć ograniczoną przewagę na pytaniach bardzo złożonych, podczas gdy bezwarunkowa dekompozycja i iteracyjne planowanie konsekwentnie pogarszają wynik. Pytanie PB4 rozstrzygnięto twierdząco – architektury rozdzielają się na trzy odrębne profile niepowodzeń sparowane po dwie konfiguracje, co potwierdza, że topologia sterowania determinuje charakter typowego błędu w większym stopniu niż jego częstość. Tabela 6.1 podsumowuje wnioski płynące z rozstrzygnięcia poszczególnych pytań.

Tabela 6.1 Odpowiedzi na pytania badawcze

Pytanie	Wniosek
PB1	Najwyższa wierność routera–specjalisty; przewaga złożoności częściowa i zależna od konkretnego mechanizmu sterowania.
PB2	Brak proporcjonalnego związku jakość–koszt; warianty iteracyjne droższe i gorsze jakościowo.
PB3	Trasowanie daje ograniczoną przewagę na pytaniach bardzo złożonych; dekompozycja pogarsza wynik.
PB4	Trzy odrębne profile niepowodzeń; architektury grupują się parami według dominującego mechanizmu.

Źródło: opracowanie własne.

6.3.7 Realizacja celów badawczych

Wszystkie cztery cele badawcze zostały zrealizowane. Środowisko orkiestracji oparte na LangGraph i Arize Phoenix umożliwiło uruchomienie sześciu architektur w jednym przebiegu na identycznym indeksie wektorowym i przy wspólnej warstwie narzędzi (CB1). Dedykowany zestaw 90 pytań testowych związanych z polskojęzycznymi dokumentami OWU i IPID powstał na potrzeby eksperymentu (CB2). Ewaluacja wszystkich sześciu architektur z wykorzystaniem czterech miar jakości i zestawu miar operacyjnych stanowi realizację celu CB3. Analiza zebranych danych z rozróżnieniem profili niepowodzeń każdej architektury oraz rekomendacje praktyczne dla zespołów wdrożeniowych realizują CB4, którego konkluzja stanowi rozstrzygnięcie głównego problemu badawczego: złożoność architektury przynosi korzyść wtedy, gdy służy precyzyjnemu sterowaniu pobieraniem, lecz iteracyjna dekompozycja i ReAct nie zapewniają wyższej jakości pomimo większego nakładu.

6.4. Kierunki dalszych badań

Wyniki eksperymentu wskazują kilka potencjalnych kierunków kontynuacji badań, wynikających z ograniczeń zaobserwowanych w zbadanym układzie.

Pierwszym kierunkiem jest rozszerzenie zestawu testowego do co najmniej 220 pytań przy udziale co najmniej dwóch modeli bazowych od różnych dostawców. Obecna próba 90 pytań nie pozwala statystycznie rozróżnić różnic między czterema najtańszymi architekturami (rozpiętość 0,033 punktu wierności). Drugi model bazowy pozwoliłby ocenić, w jakim stopniu obserwowane wzorce są specyficzne dla gpt-5, a w jakim wynikają z samej topologii.

Drugim kierunkiem jest opracowanie strategii wyszukiwania uwzględniających asymetrię polaryzacji semantycznej między klauzulami pokrycia a klauzulami wyłączeń. Jednym z możliwych rozwiązań jest wyszukiwanie hybrydowe łączące podobieństwo gęste z indeksem leksykalnym BM25 oraz osadzenia asymetryczne trenowane na parach pytanie–klauzula z odwróconą polaryzacją. Niezależną ścieżką jest metadanowe oznaczenie roli klauzuli w indeksie i wymuszenie

pobierania zarówno klauzul pokrycia, jak i wyłączeń dla każdego pytania o zakres ochrony.

Trzecim kierunkiem jest segmentacja dokumentów oparta na strukturze prawnej, a nie na progu liczby tokenów. W roli granic fragmentów należy wykorzystać granice ustępów, podpunktów lub klauzul, co zmniejsza ryzyko łączenia merytorycznie niezwiązanych zapisów w jednym fragmencie. Wstępna analiza indeksu wskazuje, że fragmenty o rozmiarze 256 tokenów z podziałem klauzulowym mogą podnieść trafność dokumentów niezależnie od architektury.

Czwartym kierunkiem jest weryfikacja automatycznej ewaluacji przez ekspertów dziedzinowych. Próba 200–300 ocen przeprowadzonych przez prawnika lub specjalistę do spraw oceny ryzyka pozwoliłaby zmierzyć korelację między oceną sędziego opartego na modelu a oceną merytoryczną oraz oszacować systematyczne obciążenia tej formy ewaluacji w kontekście prawno-umownym.

Piątym kierunkiem jest ocena mechanizmów planowania, trasowania i dekompozycji w zadaniach wymagających rzeczywistego wnioskowania wieloskokowego – takich, w których odpowiedź wymaga zestawienia klauzul z dokumentów różnych ubezpieczycieli. W obecnym zestawie każde pytanie dotyczyło jednego produktu, co uniemożliwiło ujawnienie ewentualnej przewagi mechanizmów dekompozycji. Zadania wielodokumentowe stanowią środowisko, w którym hipoteza o przewadze bardziej złożonych topologii mogłaby zostać zweryfikowana w środowisku o wyższym stopniu złożoności.

6.5. Rekomendacje dla praktyków

Niniejsza sekcja skierowana jest do inżynierów projektujących produkcyjne systemy odpowiadania na pytania na podstawie dokumentów regulacyjnych. Zalecenia zostały uporządkowane według hierarchii priorytetów, a ich szczegółowy zestaw zebrano w tabeli 6.2.

Najwyższy wpływ na poprawę wyników wykazuje optymalizacja indeksu – segmentacja klauzulowa, oddzielne kolekcje OWU i IPID oraz metadane oznaczenie roli klauzuli (pokrycie, wyłączenie, definicja) eliminują źródła błędów wspólne dla wszystkich badanych architektur. Drugą warstwą jest strategia pobierania (wyszukiwanie hybrydowe, osadzenia asymetryczne, wymuszone pobieranie wyłączeń). Wybór topologii agentowej stanowi trzeci poziom priorytetu – w zbadanym zadaniu różnice między czterema najtańszymi architekturami mieszczą się w granicach niepewności pomiarowej. Wybór modelu bazowego ma marginalne znaczenie, pod warunkiem że model spełnia minimalne wymagania jakości syntezy w języku polskim.

Tabela 6.2 Rekomendacje dla wdrożeń produkcyjnych

Rekomendacja	Uzasadnienie
Wariant bazowy	Architektura deterministyczna: najprostsza analiza niepowodzeń; najwyższa poprawność (0,716); profil błędów ukierunkowany na odpowiedzi niepełne zamiast halucynacji, wiążący się z niższym ryzykiem w kontekście regulacyjnym.
Trasowanie warunkowo	Przewaga 0,111 punktu wierności na pytaniach bardzo złożonych; koszt dodatkowego wywołania klasyfikatora pomijalny.
Oddzielne kolekcje OWU i IPID	Eliminuje niespójność autorytetu między wartością umowną a zaokrągloną, opisaną w sekcji 6.2.6.
Segmentacja klauzulowa	Granice fragmentów na poziomie ustępów i klauzul; docelowy rozmiar 256 tokenów.
Indeksowanie wyłączeń	Metadanowe oznaczenie roli klauzuli i wymuszenie pobierania zarówno klauzul pokrycia, jak i wyłączeń.
Pominięcie wariantów iteracyjnych	Brak wnioskowania wieloskokowego w zadaniu; koszt 100–187-krotnie wyższy przy jednoczesnym braku wzrostu jakości.
Regresyjny zestaw kontrolny	Zmiana modelu po stronie dostawcy może zmienić wyniki bez modyfikacji kodu; zalecane cykliczne uruchamianie 30–50 pytań.
Przegląd ekspercki	20–30 odpowiedzi miesięcznie weryfikowanych przez prawnika lub specjalistę; sygnał o dryfcie jakości niewidocznym dla sędziego opartego na modelu językowym.

Źródło: opracowanie własne.

6.6. Zakończenie

Praca dotyczyła empirycznej weryfikacji wpływu topologii systemu agentowego i jej złożoności na jakość odpowiedzi generowanych na podstawie obszernych dokumentów regulacyjnych. Uzyskane wyniki pozwalają na doprecyzowanie hipotezy wyjściowej: przewagę uzyskują topologie, w których pojedynczy przebieg pobierania jest precyzyjnie ukierunkowany, a nie te, które wykonują największą liczbę kroków rozumowania.

Cztery architektury bez rozbudowanej pętli iteracyjnej osiągają wierność w przedziale 0,778–0,811 przy koszcie 0,00037–0,00040 USD na pytanie. Dwa warianty iteracyjne generują ponad stukrotnie wyższe koszty i jednocześnie uzyskują niższe wyniki w każdej z czterech miar jakości. Zależność ta nie potwierdza hipotezy o bezpośredniej przewadze jakościowej wynikającej ze złożoności orkiestracji, ale wskazuje na warunkową przydatność wybranych mechanizmów złożoności, zwłaszcza trasowania.

W ramach niniejszej pracy zrealizowano cztery główne wkłady badawcze. Pierwszym z nich

jest odtwarzalna platforma porównawcza sześciu architektur agentowych działających w identycznym środowisku, na tych samych danych i przy tej samej infrastrukturze ewaluacyjnej. Drugim jest oryginalny zestaw testowy 90 pytań do polskojęzycznych dokumentów ubezpieczeniowych. Trzecim jest opis powtarzalnych wzorców błędów z rozróżnieniem profili ryzyka – odpowiedź niepełna wiąże się z niższym ryzykiem niż odpowiedź halucynacyjna wykazująca pozorną kompletność w kontekście regulacyjnym. Czwartym jest empiryczne doprecyzowanie problemu złożoności topologii: sama liczba ról, kroków lub iteracji nie determinuje o jakości, natomiast ukierunkowane sterowanie pobieraniem może poprawiać wynik w najtrudniejszych pytaniach.

W środowisku regulacyjnym błędy w odpowiedziach systemu niosą za sobą istotne implikacje praktyczne. Wyniki niniejszej pracy wskazują, że budowa wiarygodnych systemów odpowiadania na pytania w tej domenie jest możliwa przy stosunkowo prostej, precyzyjnie sterowanej architekturze – pod warunkiem, że priorytetem jest jakość warstwy pobierania, indeksowanie klauzul wyłączeń i hierarchia autorytetu między typami dokumentów, a nie liczba kroków rozumowania.

Spis tabel

1.1	Zestawienie pytań badawczych, hipotez i źródeł danych	4
2.1	Porównanie architektur agentowych	12
2.2	Porównanie metryk ewaluacji systemów agentowych	17
2.3	Porównanie środowisk orkiestracji systemów agentowych	19
2.4	Porównanie typów halucynacji w modelach językowych	21
2.5	Porównanie strategii fragmentacji dokumentów	26
2.6	Porównanie metryk ewaluacji systemów RAG	28
2.7	Zestawienie struktur polskich tekstów prawnych	32
2.8	Porównanie dokumentów OWU i IPID	34
2.9	Zestawienie prac pokrewnych	40
3.1	Porównanie metod badawczych	41
3.2	Porównanie architektur agentowych	43
3.3	Zestawienie warunków kontrolowanych	44
3.4	Zestawienie korpusu dokumentów	45
3.5	Zestawienie rozkładu pytań	46
3.6	Zestawienie przykładowych pytań	47
3.7	Zestawienie metryk oceny	48
3.8	Zestawienie kategorii błędów	49
4.1	Stos technologiczny	53
4.2	Artefakty systemu	54
4.3	Moduły systemu	55
4.4	Stan grafu	57
4.5	Prompty systemu	60
4.6	Narzędzia agentowe	63
4.7	Zestawienie topologii grafów	66
4.8	Parametry fragmentacji	67
4.9	Artefakty ewaluacji	69
5.1	Czas wykonania według architektury	72
5.2	Pliki wynikowe eksperymentu	73
5.3	Tokeny i koszt na uruchomienie	74
5.4	Liczba wywołań narzędzi i iteracji na uruchomienie	74
5.5	Liczba fragmentów i cytowań	75
5.6	Wierność według poziomu trudności	76

5.7	Wierność według kategorii produktowej	76
5.8	Wierność według ubezpieczyciela	77
5.9	Poprawność według poziomu trudności	77
5.10	Poprawność według kategorii produktowej	78
5.11	Poprawność według ubezpieczyciela	78
5.12	Zwiężłość według poziomu trudności	79
5.13	Zwiężłość według kategorii produktowej	79
5.14	Zwiężłość według ubezpieczyciela	80
5.15	Trafność dokumentów według poziomu trudności	80
5.16	Trafność dokumentów według kategorii produktowej	81
5.17	Trafność dokumentów według ubezpieczyciela	81
5.18	Parametry modeli językowych	82
5.19	Parametry wyszukiwania	83
5.20	Parametry fragmentacji dokumentów	83
5.21	Parametry sterujące architektur	84
5.22	Parametry wykonania eksperymentu	84
6.1	Odpowiedzi na pytania badawcze	95
6.2	Rekomendacje dla wdrożeń produkcyjnych	97

Spis rysunków

1.1	Projekt eksperymentu	2
2.1	Pętla decyzyjna agenta	8
2.2	Podział architektur agentowych	9
2.3	Potok prostego RAG	23
2.4	Potok zaawansowanego RAG	23
2.5	Potok modularnego RAG	24
2.6	Chronologia rozwiązań	38
3.1	Etapy eksperymentu	50
4.1	Warstwy systemu	55
4.2	Potok przetwarzania dokumentów	67
B.1	Architektura deterministyczna	119
B.2	Architektura planer–wykonawca	120
B.3	Architektura router–specjalista	121
B.4	Architektura tablicowa	122
B.5	Architektura hierarchiczna	123
B.6	Architektura ReAct	124

Wykaz wykresów

6.1	Wierność według architektury	85
6.2	Poprawność według architektury	86
6.3	Zwiężłość według architektury	87
6.4	Trafność dokumentów według architektury	88
6.5	Koszt na uruchomienie w skali logarytmicznej	89

Bibliografia

- [1] A. Vaswani i in., „Attention is all you need,” *Advances in neural information processing systems*, t. 30, 2017.
- [2] T. Brown i in., „Language models are few-shot learners,” *Advances in neural information processing systems*, t. 33, s. 1877–1901, 2020.
- [3] T. Schick i in., „Toolformer: Language models can teach themselves to use tools,” *Advances in neural information processing systems*, t. 36, s. 68 539–68 551, 2023.
- [4] P. Lewis i in., „Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in neural information processing systems*, t. 33, s. 9459–9474, 2020.
- [5] L. Wang i in., „A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, t. 18, nr 6, s. 186 345, 2024.
- [6] S. Yao i in., „React: Synergizing reasoning and acting in language models,” w: *The eleventh international conference on learning representations*, 2022.
- [7] Q. Wu i in., „Autogen: Enabling next-gen LLM applications via multi-agent conversations,” w: *First conference on language modeling*, 2024.
- [8] S. Hong i in., „MetaGPT: Meta programming for a multi-agent collaborative framework,” w: *The twelfth international conference on learning representations*, 2023.
- [9] Z. Xi i in., „The rise and potential of large language model based agents: A survey,” *Science China Information Sciences*, t. 68, nr 2, s. 121 101, 2025.
- [10] M. Wooldridge i N. R. Jennings, „Intelligent agents: Theory and practice,” *The knowledge engineering review*, t. 10, nr 2, s. 115–152, 1995.
- [11] T. Guo i in., „Large language model based multi-agents: A survey of progress and challenges,” *arXiv preprint arXiv:2402.01680*, 2024.
- [12] L. Weng, „LLM-powered Autonomous Agents,” *lilianweng.github.io*, czerwiec 2023. URL: <https://lilianweng.github.io/posts/2023-06-23-agent/>
- [13] Z. Ji i in., „Survey of hallucination in natural language generation,” *ACM computing surveys*, t. 55, nr 12, s. 1–38, 2023.
- [14] L. Wang i in., „Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” w: *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 2023, s. 2609–2634.
- [15] N. Shazeer i in., „Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *arXiv preprint arXiv:1701.06538*, 2017.

- [16] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum i I. Mordatch, „Improving factuality and reasoning in language models through multiagent debate,” w: *Forty-first international conference on machine learning*, 2024.
- [17] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan i S. Yao, „Reflexion: Language agents with verbal reinforcement learning, 2023,” URL <https://arxiv.org/abs/2303.11366>, t. 8, 2024.
- [18] S. G. Patil, T. Zhang, X. Wang i J. E. Gonzalez, „Gorilla: Large language model connected with massive apis,” *Advances in Neural Information Processing Systems*, t. 37, s. 126 544–126 565, 2024.
- [19] Y. Qin i in., „Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023,” URL <https://arxiv.org/abs/2307.16789>, t. 2, 2024.
- [20] T. Sumers, S. Yao, K. R. Narasimhan i T. L. Griffiths, „Cognitive architectures for language agents,” *Transactions on Machine Learning Research*, 2023.
- [21] K. Papineni, S. Roukos, T. Ward i W.-J. Zhu, „Bleu: a method for automatic evaluation of machine translation,” w: *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, s. 311–318.
- [22] C.-Y. Lin, „Rouge: A package for automatic evaluation of summaries,” w: *Text summarization branches out*, 2004, s. 74–81.
- [23] X. Liu i in., „Agentbench: Evaluating llms as agents,” *arXiv preprint arXiv:2308.03688*, 2023.
- [24] L. Zheng i in., „Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in neural information processing systems*, t. 36, s. 46 595–46 623, 2023.
- [25] A. Panickssery, S. R. Bowman i S. Feng, „Llm evaluators recognize and favor their own generations, 2024,” URL <https://arxiv.org/abs/2404.13076>,
- [26] W. Shi i in., „Replug: Retrieval-augmented black-box language models,” w: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, s. 8371–8384.
- [27] L. Gao, X. Ma, J. Lin i J. Callan, „Precise zero-shot dense retrieval without relevance labels,” w: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, s. 1762–1777.
- [28] J. Maynez, S. Narayan, B. Bohnet i R. McDonald, „On faithfulness and factuality in abstractive summarization,” w: *Proceedings of the 58th annual meeting of the association for computational linguistics*, 2020, s. 1906–1919.
- [29] J. H. Curlin IV, „ChatGPT Didn’t Write This... Or Did It? The Emergence of Generative AI in the Legal Field and Lessons from *Mata v. Avianca*,” *Ark. L. Rev.*, t. 78, s. 123, 2025.

- [30] N. F. Liu i in., „Lost in the middle: How language models use long contexts,” *Transactions of the association for computational linguistics*, t. 12, s. 157–173, 2024.
- [31] Y. Gao i in., „Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, t. 2, nr 1, s. 32, 2023.
- [32] N. Reimers i I. Gurevych, „Sentence-bert: Sentence embeddings using siamese bert-networks,” w: *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, 2019, s. 3982–3992.
- [33] J. Johnson, M. Douze i H. Jégou, „Billion-scale similarity search with GPUs,” *IEEE transactions on big data*, t. 7, nr 3, s. 535–547, 2019.
- [34] S. Robertson i H. Zaragoza, *The probabilistic relevance framework: BM25 and beyond*. Now Publishers Inc, 2009, t. 4.
- [35] V. Karpukhin i in., „Dense passage retrieval for open-domain question answering,” w: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, 2020, s. 6769–6781.
- [36] G. V. Cormack, C. L. Clarke i S. Buettcher, „Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” w: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, 2009, s. 758–759.
- [37] R. Pradeep, S. Sharifymoghaddam i J. Lin, „Rankvicuna: Zero-shot listwise document reranking with open-source large language models,” *arXiv preprint arXiv:2309.15088*, 2023.
- [38] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly i M. Abdelrazek, „Seven failure points when engineering a retrieval augmented generation system,” w: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*, 2024, s. 194–199.
- [39] S. Es, J. James, L. E. Anke i S. Schockaert, „Ragas: Automated evaluation of retrieval augmented generation,” w: *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations*, 2024, s. 150–158.
- [40] H. Trivedi, N. Balasubramanian, T. Khot i A. Sabharwal, „Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” w: *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 2023, s. 10 014–10 037.
- [41] Z. Jiang i in., „Active retrieval augmented generation,” w: *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, s. 7969–7992.

- [42] A. Asai, Z. Wu, Y. Wang, A. Sil i H. Hajishirzi, „Self-rag: Learning to retrieve, generate, and critique through self-reflection,” w: *The Twelfth International Conference on Learning Representations*, 2023.
- [43] Y. Koreeda i C. D. Manning, „ContractNLI: A dataset for document-level natural language inference for contracts,” w: *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, s. 1907–1919.
- [44] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras i I. Androutsopoulos, „LEGAL-BERT: The muppets straight out of law school,” w: *Findings of the association for computational linguistics: EMNLP 2020*, 2020, s. 2898–2904.
- [45] D. Hendrycks, C. Burns, A. Chen i S. Ball, „Cuad: An expert-annotated nlp dataset for legal contract review,” *arXiv preprint arXiv:2103.06268*, 2021.
- [46] Z. Yang i in., „HotpotQA: A dataset for diverse, explainable multi-hop question answering,” w: *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2018, s. 2369–2380.
- [47] I. Chalkidis i in., „LexGLUE: A benchmark dataset for legal language understanding in English,” w: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, s. 4310–4330.
- [48] P. Rybak, P. Przybyła i M. Ogrodniczuk, „PolQA: Polish question answering dataset,” w: *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 2024, s. 12 846–12 855.
- [49] S. Dadas, M. Perełkiewicz i R. Poświata, „PIRB: A Comprehensive Benchmark of Polish Dense and Hybrid Text Retrieval Methods,” 2024. arXiv: 2402.13350 [cs.CL].
- [50] Sejm Rzeczypospolitej Polskiej, *Ustawa z dnia 11 września 2015 r. o działalności ubezpieczeniowej i reasekuracyjnej*, Dziennik Ustaw 2015, poz. 1844, 2015. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20150001844>
- [51] European Parliament and Council, „Directive 2009/138/EC on the Taking-up and Pursuit of the Business of Insurance and Reinsurance (Solvency II),” Official Journal of the European Union, Directive 2009/138/EC, 2009. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32009L0138>
- [52] Polska Izba Ubezpieczeń, „Raport roczny 2023,” Polska Izba Ubezpieczeń, sprawozdanie techniczne, 2023. URL: <https://piu.org.pl/raporty-roczne/>
- [53] European Parliament and Council, „Directive (EU) 2016/97 on Insurance Distribution (IDD),” Official Journal of the European Union, Directive 2016/97/EU, 2016. URL: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016L0097>

- [54] Sejm Rzeczypospolitej Polskiej, *Ustawa z dnia 15 grudnia 2017 r. o dystrybucji ubezpieczeń*, Dziennik Ustaw 2017, poz. 2486, 2017. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20170002486>
- [55] A. Mishra i S. K. Jain, „A survey on question answering systems with classification,” *Journal of King Saud University-Computer and Information Sciences*, t. 28, nr 3, s. 345–361, 2016.
- [56] D. Edge i in., „From local to global: A graph rag approach to query-focused summarization, 2025,” URL <https://arxiv.org/abs/2404.16130>,
- [57] W. Zhang i in., „A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist,” w: *Proceedings of the 30th acm sigkdd conference on knowledge discovery and data mining*, 2024, s. 4314–4325.
- [58] Y. Shao, Y. Jiang, T. Kanell, P. Xu, O. Khattab i M. Lam, „Assisting in writing wikipedia-like articles from scratch with large language models,” w: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, s. 6252–6278.
- [59] X. Ho, A.-K. D. Nguyen, S. Sugawara i A. Aizawa, „Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps,” w: *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, s. 6609–6625.

A. Prompty systemu

Dodatek zawiera moduł `sources/config/prompts.py` z promptami wykorzystywanymi w systemie. Ich wydzielenie zwiększa czytelność opisu implementacji w rozdziale 4 i umożliwia weryfikację treści instrukcji przekazywanych modelom.

Prompt `EXTRACTION_PROMPT` służy jako instrukcja systemowa dla modelu multimodalnego podczas ekstrakcji tekstu ze stron dokumentów ubezpieczeniowych metodą OCR.

Listing A.1 Prompt ekstrakcji tekstu OCR (`EXTRACTION_PROMPT`)

```
1 EXTRACTION_PROMPT: str = """Act as an expert OCR system for Polish insurance documents. Extract text VERBATIM into clean Markdown.
2
3 ## CRITICAL RULES
4 1. **Text Fidelity:** No paraphrasing or summarizing. Preserve Polish diacritics (ą,ć,ę,ł,ń,ó,ś,ż,ź), legal terminology, numbers, and
   monetary values exactly.
5 2. **Layout:** Process multi-column layouts Left-to-Right, then Top-to-Bottom. Keep related content (e.g., terms and definitions)
   together.
6 3. **Clean Output:** Return ONLY raw Markdown. No 'markdown wrappers, preamble, or commentary. Ignore logos, page numbers, QR codes,
   and watermarks.
7
8 ## MARKDOWN HIERARCHY
9 * '#' (H1): Document Titles, Major Sections (e.g., "ROZDZIAŁ I", "POSTANOWIENIA OGÓLNE").
10 * '##' (H2): OWU Paragraphs (e.g., "§ 1"), Standard IPID Questions (see below).
11 * '###' (H3): Subsections, specific product variants (e.g., "Wariant Komfort"), internal headers.
12 * '---': Use to separate major sections.
13
14 ## FORMATTING STANDARDS
15 * **Lists:** Use '*' for bullets (never '-'). Preserve exact numbering format ('1.', '1)', 'a').
16 * **Emphasis:** Use **bold** for defined terms/keywords and headers. Preserve ✓ and × symbols.
17 * **Tables:** Reconstruct using Markdown tables. Align columns left ('|:---|').
18   * *Example:* '| POJĘCIE | CO OZNACZA? |'
19
20 ## STANDARD IPID HEADERS (Use Exact Wording)
21 Treat these specific questions as '##' headers:
22 * Jakiego rodzaju jest to ubezpieczenie?
23 * Co jest przedmiotem ubezpieczenia?
24 * Czego nie obejmuje ubezpieczenie?
25 * Jakie są ograniczenia ochrony ubezpieczeniowej?
26 * Gdzie obowiązuje ubezpieczenie?
27 * Co należy do obowiązków ubezpieczonego?
28 * Jak i kiedy należy opłacać składki?
29 * Kiedy rozpoczyna się i kończy ochrona ubezpieczeniowa?
30
31 ## CONTINUITY
32 If text continues from a previous page, complete the sentence/section smoothly without repeating headers.
33 """
```

Prompt `QUESTION_PARSER_SYSTEM_PROMPT` definiuje proces wyodrębniania strukturalnych metadanych z pytania użytkownika na potrzeby filtrowania w bazie wektorowej Qdrant.

Listing A.2 Prompt parsera pytań (`QUESTION_PARSER_SYSTEM_PROMPT`)

```
1 QUESTION_PARSER_SYSTEM_PROMPT: str = """\
2 Task: extract structured metadata from one user question for Qdrant filtering.
3
4 Return ONLY valid JSON (no markdown, no commentary) with this exact schema:
5 {
6   "company": "<hestia|pzu|warta|null>",
7   "product": "<canonical product id|null>",
8   "intent": "<faq|description|coverage|exclusions|conditions|limits|comparison|claims|unknown>",
9   "entities": ["<domain entities only>"],
10  "is_multi_part": <true|false>,
11  "language": "<ISO 639-1>"
12 }
13
14 Intent guide (pick ONE best):
15 - faq: short fact (ile/jaki/kiedy/gdzie)
```

```

16 - description: opis lub podsumowanie zakresu
17 - coverage: czy coś jest objęte ochroną
18 - exclusions: czego nie obejmuje / wyłączenia
19 - conditions: warunki, obowiązki, czy mogę
20 - limits: kwoty, limity, franszyza, udział własny
21 - comparison: porównanie produktów/wariantów
22 - claims: zgłoszenie szkody, terminy, proces
23 - unknown: no good match
24
25 Company normalization:
26 - hestia aliases: ergo hestia, stu ergo hestia, sopockie towarzystwo
27 - pzu aliases: pzu, pzu sa, powszechny zakład ubezpieczeń, powszechny zakład ubezpieczeń
28 - warta aliases: warta, tuir warta, warta sa
29 - ambiguous tokens like "ergo" or "ergo7" alone do NOT set company
30
31 Product normalization:
32 - ergo7_podroz: ergo podroz, ergo podróż, ergo 7 podroz, ergo 7 podróż, ergo podroze, ergo podróże
33 - ergo7_komunikacja: ergo 7 komunikacja, ergo komunikacja
34 - ergo7_pozakomunikacyjne: ergo 7 pozakomunikacyjne, ergo pozakomunikacyjne
35 - autocasco_komfort_ack: ac komfort, autocasco komfort
36 - autocasco_standard_acs: ac standard, autocasco standard
37 - pzu_auto / pzu_dom / pzu_wojazer for PZU Auto/Dom/Wojazer/Wojażer
38 - warta_travel / warta_dom / warta_dom_komfort for Warta variants
39
40 Infer company from product:
41 - ergo7_* -> hestia
42 - pzu_* -> pzu
43 - autocasco_* and warta_* -> warta
44
45 Quality rules:
46 - If product and company conflict, trust product mapping and correct company.
47 - If unsure, use null (never guess).
48 - entities: only concrete insurance terms (risk, coverage, vehicle type, location, medical term, sport); no generic words.
49 - is_multi_part=true only for truly separate questions.
50 """

```

Prompt QUERY_REWRITER_SYSTEM_PROMPT służy do generowania trzech zróżnicowanych leksykalnie wariantów zapytania na potrzeby wyszukiwania wektorowego.

Listing A.3 Prompt przepisania zapytań (QUERY_REWRITER_SYSTEM_PROMPT)

```

1 QUERY_REWRITER_SYSTEM_PROMPT: str = """\
2 Task: generate exactly 3 diverse rewrites for vector retrieval in OWU/IPID corpus.
3
4 Return ONLY valid JSON array:
5 [
6   {"strategy": "direct", "query": "..."},
7   {"strategy": "step_back", "query": "..."},
8   {"strategy": "hyde", "query": "..."}
9 ]
10
11 Rules:
12 - Keep original question language.
13 - If company/product context exists, include those tokens in ALL rewrites exactly.
14 - direct: specific keywords matching likely clause text.
15 - step_back: broader section-level query (e.g., "Zakres ubezpieczenia", "Wyłączenia Odpowiedzialności", "Limity świadczeń").
16 - hyde: 2-3 sentence plausible legal-style snippet with clause style (§, ust., pkt) and conditional wording.
17 - Ensure lexical diversity across 3 rewrites; avoid near-duplicates.
18 - Keep each rewrite compact (prefer <= 24 words; hyde <= 2 short sentences).
19 - Output exactly 3 objects with strategies: direct, step_back, hyde (no extra keys).
20 """

```

Prompt RERANKER_SYSTEM_PROMPT definiuje kryteria dla modelu oceniającego trafność każdego pobranego fragmentu dokumentu względem zadanego pytania w skali 0–1.

Listing A.4 Prompt oceny trafności fragmentów (RERANKER_SYSTEM_PROMPT)

```

1 RERANKER_SYSTEM_PROMPT: str = """\
2 Task: score each retrieved chunk for relevance to the question on 0.0-1.0.
3
4 Use 4 equal criteria:
5 1) topical precision,
6 2) contains answer-bearing detail (clause/number/condition),
7 3) correct entity (company/product/variant),

```

```

8 4) right granularity.
9
10 Scoring bands:
11 - 0.9-1.0 direct answer chunk
12 - 0.7-0.8 strong but missing small detail
13 - 0.4-0.6 related but not answer-ready
14 - 0.1-0.3 weakly related
15 - 0.0 irrelevant/boilerplate
16
17 Penalize (minus 0.2 to 0.3): table of contents, wrong product/company, definition-only chunk without operative rule.
18
19 Input: [{"index": int, "text": str}, ...]
20 Output: [{"index": int, "score": float}, ...]
21 Return ONLY valid JSON.
22 """

```

Prompt `REFERENCE_CORRECTNESS_PROMPT` jest wykorzystywany przez ewaluator Phoenix do oceny poprawności wygenerowanej odpowiedzi względem wzorcowej odpowiedzi referencyjnej.

Listing A.5 Prompt oceny poprawności odpowiedzi (`REFERENCE_CORRECTNESS_PROMPT`)

```

1 REFERENCE_CORRECTNESS_PROMPT: str = """You are evaluating whether a candidate answer is correct with respect to a gold reference answer.
2 The answers are responses to Polish insurance document questions. The candidate answer may use Polish.
3
4 Assign one label from this graded correctness rubric (mapped to score 0.0-1.0):
5 - "fully_correct" (1.0): semantically equivalent to the gold answer; no material errors.
6 - "mostly_correct" (0.8): key conclusion is correct; allows moderate omissions, compression, or less detail than the gold answer.
7 - "partially_correct" (0.5): core direction is plausible but incomplete or somewhat mixed with inaccuracies.
8 - "mostly_incorrect" (0.3): substantial mismatch, but still contains limited aligned facts.
9 - "fully_incorrect" (0.0): contradicts the gold answer, misses the core answer, or is unsupported.
10
11 Calibration examples (quality-oriented):
12 - 1.0: no meaningful factual errors.
13 - 0.8: up to around 35% of non-critical detail may be missing.
14 - 0.5: around 35-60% of important detail missing or mixed quality.
15 - 0.3: around 60-80% mismatch.
16 - 0.0: all key facts are wrong/missing.
17
18 Edge cases:
19 - A candidate that says "the documents do not contain this information" is "fully_incorrect" when the gold answer provides a specific
    answer.
20 - A candidate with the right conclusion but a wrong clause number is at least "mostly_correct" if factual content matches.
21 - A candidate that covers only part of a multi-part gold answer should usually be "partially_correct" unless the missing part changes the
    final conclusion.
22 - Prefer the higher score when uncertainty is between adjacent labels and there is no clear contradiction.
23
24 [BEGIN DATA]
25 *****
26 [Question]: {input}
27 *****
28 [Gold Answer]: {expected_answer}
29 *****
30 [Candidate Answer]: {output}
31 [END DATA]
32
33 Return only one label from: "fully_correct", "mostly_correct", "partially_correct", "mostly_incorrect", "fully_incorrect".
34 [Label]:"""

```

Prompt `CONCISENESS_PROMPT` jest używany przez ewaluator Phoenix do oceny zwięzłości odpowiedzi generowanej przez system.

Listing A.6 Prompt oceny zwięzłości odpowiedzi (`CONCISENESS_PROMPT`)

```

1 CONCISENESS_PROMPT: str = """You are an evaluation assistant assessing whether a response is concise.
2 The responses are answers to Polish insurance document questions and may legitimately use structured Markdown (bold headers, bullet
    lists, clause references like § X ust. Y, inline citations).
3
4 Assign one label from this graded conciseness rubric (mapped to score 0.0-1.0):
5 - "very_concise" (1.0): clear, focused answer with minimal filler.
6 - "mostly_concise" (0.8): compact and readable; allows some supportive context.
7 - "mixed" (0.5): balance of useful content and extra wording.
8 - "mostly_verbos" (0.3): clearly longer than needed with repeated or low-value filler.

```

```

9 - "very_verbose" (0.0): excessive padding where signal is hard to find.
10
11 Calibration examples (verbosity-oriented):
12 - 1.0: no unnecessary words.
13 - 0.8: up to around 30% unnecessary words.
14 - 0.5: around 30-60% unnecessary words.
15 - 0.3: around 60-80% unnecessary words.
16 - 0.0: almost entirely unnecessary padding.
17
18 Assessment rules:
19 - Reward information-dense structure when it helps clarity (headers, bullets, citations).
20 - Do not penalize brief safety caveats, traceability phrasing, or short context-setting lines when they improve reliability.
21 - Penalize avoidable padding (e.g., repeated restating of the question, long generic preambles, repeated hedging).
22
23 Important: length alone does not determine verbosity. A short but padded answer can be "mostly_verbose" or "very_verbose"; a long but
    information-dense answer can be "very_concise" or "mostly_concise".
24 When unsure between adjacent labels, choose the less harsh one.
25
26 [BEGIN DATA]
27 *****
28 [Question]: {input}
29 *****
30 [Response]: {output}
31 [END DATA]
32
33 Return only one label from: "very_concise", "mostly_concise", "mixed", "mostly_verbose", "very_verbose".
34 [Label]: ""

```

Prompt `PLANNER_SYSTEM_PROMPT` definiuje zadanie agenta planisty polegające na opracowaniu optymalnego planu wywołań narzędzi dla danego pytania użytkownika.

Listing A.7 Prompt agenta planisty (`PLANNER_SYSTEM_PROMPT`)

```

1 PLANNER_SYSTEM_PROMPT: str = """\
2 You are a planning agent for Polish insurance document Q&A.
3
4 Available tools:
5 {tool_descriptions}
6
7 Return ONLY valid JSON array:
8 [{{"tool_name": "<name>", "reason": "<short reason>"}, ...]
9
10 Required order:
11 1) question_parser
12 2) query_rewriter
13 3) retriever
14 4) prompt_selector
15 5) answer_synthesizer (always last)
16
17 Optional tools:
18 - reranker: add for clause-specific / limits / exclusions / conditions questions
19 - evidence_selector: add when many chunks or noisy retrieval
20 - citation_maker: add when explicit traceability is needed
21
22 Never use unknown tool names. Prefer the shortest sufficient plan.
23 Keep reason very short (3-8 words).
24 """

```

Prompt `REACT_SYSTEM_PROMPT` definiuje pętlę rozumowania i działania agenta ReAct, określając format kroków myślenia oraz reguły korzystania z narzędzi.

Listing A.8 Prompt agenta ReAct (`REACT_SYSTEM_PROMPT`)

```

1 REACT_SYSTEM_PROMPT: str = """\
2 You are a ReAct agent for Polish insurance Q&A.
3 Use ONLY retrieved OWU/IPID context. Never use prior knowledge.
4
5 Available tools:
6 {tool_descriptions}
7
8 At each step output ONLY valid JSON:
9
10 To use a tool:
11 [{{"thought": "<your reasoning>", "action": "<tool_name>", "action_input": {<optional overrides>}}}]

```



```

12
13 To finish:
14 {"thought": "<your reasoning>", "action": "finish", "answer": "<your final answer>"}
15
16 Default flow:
17 1) question_parser
18 2) query_rewriter
19 3) retriever
20 4) optional reranker/evidence_selector/citation_maker
21 5) prompt_selector
22 6) answer_synthesizer
23 7) finish
24
25 Hard rules:
26 - Call each tool at most once; retriever may be retried once if clearly off-topic.
27 - Max 8 steps.
28 - Never fabricate facts; if context is insufficient, say so.
29 - Always produce final answer via answer_synthesizer before finish.
30 - Keep "thought" short and action-oriented.
31 - Keep "thought" <= 12 words.
32 - If no overrides are needed, set "action_input": {}.
33 - Use only actions from available tools plus "finish".
34 """

```

Prompt `ROUTER_SYSTEM_PROMPT` służy do klasyfikacji pytania użytkownika do jednej z ośmiu zdefiniowanych kategorii tematycznych.

Listing A.9 Prompt klasyfikatora kategorii pytań (`ROUTER_SYSTEM_PROMPT`)

```

1 ROUTER_SYSTEM_PROMPT: str = """\
2 Task: classify the question into exactly one category.
3
4 Return ONLY valid JSON:
5 {"category": "<faq|description|coverage|exclusions|conditions|limits|comparison|claims>"}
6
7 Quick guide:
8 - faq: short fact
9 - description: overview/summary
10 - coverage: whether covered
11 - exclusions: whether excluded / what is not covered
12 - conditions: requirements/eligibility/obligations
13 - limits: money limits, sums, deductibles
14 - comparison: compare products/variants/companies
15 - claims: claims process/deadlines/reporting
16
17 Disambiguation:
18 - "Czy pokrywane..." -> coverage
19 - "Czego nie obejmuje..." -> exclusions
20 - "Czy mogę..." -> conditions
21 - "Do jakiej kwoty..." -> limits
22 - If mixed, pick primary user intent.
23 """

```

Prompt `DECOMPOSER_SYSTEM_PROMPT` określa zasady podziału złożonego pytania użytkownika na maksymalnie trzy niezależne podpytania.

Listing A.10 Prompt dekompozycji pytań (`DECOMPOSER_SYSTEM_PROMPT`)

```

1 DECOMPOSER_SYSTEM_PROMPT: str = """\
2 Task: split a user question into 1-3 independent sub-questions.
3
4 Return ONLY valid JSON array of strings:
5 [<sub-question 1>", "<sub-question 2>", ...]
6
7 Rules:
8 - Keep single atomic questions as one element.
9 - Split only when there are distinct asks (e.g. condition + amount, or two different topics).
10 - Keep each sub-question self-contained (include product/company when needed).
11 - Preserve original language (Polish).
12 """

```

Prompt `MERGE_ANSWERS_SYSTEM_PROMPT` definiuje zasady scalania odpowiedzi cząstko-

wych na poszczególne podpytania w jedną spójną odpowiedź w języku polskim.

Listing A.11 Prompt scalania odpowiedzi cząstkowych (MERGE_ANSWERS_SYSTEM_PROMPT)

```
1 MERGE_ANSWERS_SYSTEM_PROMPT: str = """\n
2 You merge sub-answers into one coherent Polish response.\n
3\n
4 ## Mandatory output structure\n
5\n
6 **Odpowiedź:** <exactly 1 full, grammatical, and logical sentence answering the original question; no citations>\n
7\n
8 ---\n
9\n
10 **Szczegóły:**\n
11 - <key fact from sub-answer 1> [N]\n
12 - <key fact from sub-answer 2> [N]\n
13\n
14 ## Merging rules\n
15 - Synthesize, do not list sub-answers separately.\n
16 - Re-number citations globally as [1], [2], ...\n
17 - Remove duplicates.\n
18 - If contradictions exist, state both with citations.\n
19 - Omit "insufficient context" parts unless all parts are insufficient.\n
20 - No intro/filler; Polish only.\n
21 - Keep **Odpowiedź** to exactly one decisive sentence.\n
22 - Do not add any citation in **Odpowiedź**.\n
23 - Insert a Markdown separator line '---' between **Odpowiedź** and **Szczegóły**.\n
24 - In **Szczegóły**, provide fully explained details with citations and clause references when available.\n
25 - For yes/no outcomes, start with: "Tak," / "Nie," / "Warunkowo".\n
26 - Preserve exact values and constraints (kwoty, limity, terminy, wyjątki).\n
27\n
28 Original question: {question}\n
29 """
```

Szablon `_ANSWER_FORMAT` definiuje obligatoryjną strukturę wyjścia (sekcje **Odpowiedź** i **Szczegóły**) oraz zbiór reguł dla wszystkich promptów odpowiedzi domenowych.

Listing A.12 Szablon formatu odpowiedzi (`_ANSWER_FORMAT`)

```
1 _ANSWER_FORMAT: str = """\n
2 ## Mandatory output structure\n
3\n
4 **Odpowiedź:** <exactly 1 full, grammatical, and logical sentence; start with verdict/conclusion/key fact; no citations>\n
5\n
6 ---\n
7\n
8 **Szczegóły:**\n
9 - <key fact with the concrete clause reference taken from the context, e.g. (§ 12 ust. 4 pkt 2) [1]>\n
10 - <another key fact supported by the context> [2]\n
11\n
12 ## Hard rules\n
13 - ALWAYS answer in Polish (język polski).\n
14 - Use ONLY the retrieved context - no prior knowledge, no assumptions, no guessing.\n
15 - Start immediately with the verdict/direct answer. No greetings, no intro, no restating question.\n
16 - ALWAYS write **Odpowiedź** as exactly one full, grammatical, and logical sentence (never a fragment).\n
17 - NEVER add inline citations such as [1] in **Odpowiedź**.\n
18 - Insert a Markdown separator line '---' between **Odpowiedź** and **Szczegóły**.\n
19 - Every bullet in **Szczegóły** must also be a full logical sentence.\n
20 - Every factual claim MUST have an inline citation like [1] or [2] referencing a specific source from the context.\n
21 - When citing insurance clauses, include the full concrete reference copied from the context, e.g. § 12 ust. 4 pkt 2 [1]. Never output\n
22     literal placeholders such as § X ust. Y pkt Z or [N].\n
23 - Prefer concrete answer shape matching evaluation targets:\n
24   - yes/no questions: "Tak/Nie/Warunkowo, ..." + one decisive condition or exception,\n
25   - amount questions: exact amount + unit + scope,\n
26   - variant questions: explicitly name the variant/product.\n
27 - ALWAYS keep both sections: **Odpowiedź** and **Szczegóły**.\n
28 - If there is only one key fact, keep **Szczegóły** as a single bullet.\n
29 - Keep **Szczegóły** detailed and evidence-based (prefer 2-6 bullets when evidence exists).\n
30 - If multiple sources confirm the same fact, cite the most specific one (prefer clause-level over chapter-level).\n
31 - If the context contains CONTRADICTIONARY information from different products/companies, present both perspectives with their citations.\n
32 - Avoid filler and hedging (e.g., "na podstawie dokumentów", "co do zasady") unless required by the clause wording.\n
33 - If the context is insufficient, respond ONLY with:\n
34     "Dostępne dokumenty nie zawierają wystarczających informacji, aby odpowiedzieć na to pytanie."
35 """
```

Prompt FAQ_PROMPT formułuje zasady udzielania zwięzłych i precyzyjnych odpowiedzi na pytania faktograficzne dotyczące dokumentów ubezpieczeniowych.

Listing A.13 Prompt odpowiedzi FAQ (FAQ_PROMPT)

```
1 FAQ_PROMPT: str = f"""\n
2 You answer factual insurance questions briefly and precisely.\n
3 \n
4 Answering strategy:\n
5 - Extract exact datum (number/name/date/yes-no).\n
6 - For "czy": start with "Tak" or "Nie".\n
7 - For "ile": include exact unit.\n
8 - No background; only necessary fact.\n
9 - Keep **Odpowiedź** as exactly one compact sentence.\n
10 {_ANSWER_FORMAT}\n
11 """
```

Prompt DESCRIPTION_PROMPT służy do generowania zwięzłego opisu produktu lub zakresu ochrony na podstawie pobranych fragmentów kontekstu.

Listing A.14 Prompt opisu produktu (DESCRIPTION_PROMPT)

```
1 DESCRIPTION_PROMPT: str = f"""\n
2 You provide concise product/coverage descriptions from retrieved context only.\n
3 \n
4 Answering strategy:\n
5 - In **Odpowiedź**: exactly 1 sentence summary.\n
6 - In **Szczegóły**: Zakres, Wyłączenia, Warunki, Limity, Warianty.\n
7 - Clearly label facts by variant when applicable.\n
8 - Avoid broad/general summaries; include only decisive policy facts.\n
9 {_ANSWER_FORMAT}\n
10 """
```

Prompt CLAIMS_PROMPT definiuje wytyczne do opisu procedury zgłaszania i obsługi szkody, w tym kroków, terminów i wymaganych dokumentów.

Listing A.15 Prompt obsługi szkód (CLAIMS_PROMPT)

```
1 CLAIMS_PROMPT: str = f"""\n
2 You are a Polish insurance documentation assistant explaining a claims process.\n
3 \n
4 Answering strategy:\n
5 - In **Odpowiedź**: one decisive sentence with who to contact + key deadline.\n
6 - In **Szczegóły**: kroki, terminy, dokumenty, metody rozliczenia.\n
7 - Every procedural fact needs citation and clause if available.\n
8 {_ANSWER_FORMAT}\n
9 """
```

Prompt COMPARISON_PROMPT określa format zestawienia produktów, wariantów lub firm ubezpieczeniowych w postaci tabeli Markdown.

Listing A.16 Prompt porównania produktów (COMPARISON_PROMPT)

```
1 COMPARISON_PROMPT: str = f"""\n
2 You are a Polish insurance documentation assistant comparing insurance products, variants, or companies.\n
3 \n
4 Answering strategy:\n
5 - In **Odpowiedź**: one sentence key differences summary.\n
6 - In **Szczegóły** use a Markdown comparison table:\n
7   - Rows: key attributes (zakres, wyłączenia, limity, warunki, składka)\n
8   - Columns: compared products/variants\n
9   - Use "Brak danych" when evidence is missing (never invent)\n
10 - Every non-empty cell must be traceable to citation [N].\n
11 {_ANSWER_FORMAT}\n
12 """
```

Prompt ELIGIBILITY_PROMPT ustala zasady wyjaśniania kryteriów uprawnienia do ochrony,

rozdzielając warunki obowiązkowe, opcjonalne i wyłączenia podmiotowe.

Listing A.17 Prompt warunków uprawnienia do ochrony (ELIGIBILITY_PROMPT)

```
1 ELIGIBILITY_PROMPT: str = f"""\n
2 You are a Polish insurance documentation assistant explaining eligibility criteria and requirements.\n
3 \n
4 Answering strategy:\n
5 - In **Odpowiedź**: one direct Tak/Nie/Warunkowo sentence or key requirement.\n
6 - In **Szczegóły** separate:\n
7   - **Warunki obowiązkowe** - hard requirements that MUST be met (mandatory conditions)\n
8   - **Warunki opcjonalne** - conditions that can be changed via additional premium or endorsement\n
9   - **Wyłączenia podmiotowe** - who/what is NOT eligible\n
10  - Include clause references and deadlines when available.\n
11 {_ANSWER_FORMAT}\n
12 """
```

Prompt PRICING_PROMPT określa sposób prezentacji informacji cenowych i finansowych z dokumentów polisowych, takich jak sumy ubezpieczenia czy składki.

Listing A.18 Prompt informacji cenowych (PRICING_PROMPT)

```
1 PRICING_PROMPT: str = f"""\n
2 You are a Polish insurance documentation assistant providing pricing and financial information.\n
3 \n
4 Answering strategy:\n
5 - In **Odpowiedź** state the exact documented amount(s) in PLN or the pricing mechanism in one sentence.\n
6 - In **Szczegóły** list:\n
7   - All relevant amounts (sumy ubezpieczenia, składki, limity) with exact PLN values\n
8   - Factors that affect pricing (wiek pojazdu, zakres, wariant, etc.) if documented\n
9   - Any discounts or surcharges mentioned\n
10  - NEVER present figures as guaranteed prices - they are policy-documented limits/rates.\n
11  - Always include the clause reference (§ X ust. Y) for every monetary value.\n
12 {_ANSWER_FORMAT}\n
13 """
```

Prompt COVERAGE_PROMPT wymusza sformułowanie jednoznacznego werdyktu (Tak/Nie/Warunkowo) w kwestii objęcia danego zdarzenia ochroną ubezpieczeniową.

Listing A.19 Prompt zakresu ochrony (COVERAGE_PROMPT)

```
1 COVERAGE_PROMPT: str = f"""\n
2 You are a Polish insurance documentation assistant determining whether something is covered by an insurance policy.\n
3 \n
4 Answering strategy:\n
5 - In **Odpowiedź** start IMMEDIATELY with a clear verdict: **Tak** / **Nie** / **Warunkowo** (conditionally), in one sentence.\n
6   - **Tak** - the event/item IS covered under standard policy terms\n
7   - **Nie** - the event/item is explicitly EXCLUDED\n
8   - **Warunkowo** - covered only under specific conditions (additional premium, specific variant, time limit, etc.)\n
9   - Include the decisive reason in that same sentence.\n
10  - In **Szczegóły** cite the specific OWU clauses that determine coverage:\n
11    - The operative clause granting coverage with its concrete reference from the context, e.g. (§ 8 ust. 1) [1]\n
12    - Any exclusions that limit or negate coverage with their concrete references, e.g. (§ 10 ust. 3) [2]\n
13    - Any conditions required for coverage to apply (additional premium, variant, etc.) with citation [3]\n
14  - If coverage depends on the variant/add-on, explicitly name which variant provides it.\n
15  - For edge cases, prioritize exclusion clauses and exceptions.\n
16  - If both coverage and exclusion appear, conclude using the controlling rule (exclusion or explicit exception).\n
17 {_ANSWER_FORMAT}\n
18 """
```

Prompt CONDITIONS_PROMPT określa wytyczne do wyjaśniania warunków umownych, obowiązków ubezpieczonego i wymagań formalnych wynikających z dokumentów OWU.

Listing A.20 Prompt warunków umowy (CONDITIONS_PROMPT)

```
1 CONDITIONS_PROMPT: str = f"""\n
2 You are a Polish insurance documentation assistant explaining policy conditions, requirements, or contractual rules.\n
3 \n
4 Answering strategy:\n
5 - In **Odpowiedź** state the key condition or requirement directly in one sentence.
```

```

6 - In Szczegóły bullet-list all relevant conditions, organized as:
7   - Each condition as a separate bullet with its concrete clause reference copied from the context, e.g. (§ 15 ust. 2 pkt 1) [1]
8   - Mark each condition as: [Obowiązkowe] (mandatory - failure voids coverage) or [Zalecane] (recommended, non-compliance may
   reduce payout)
9   - Include specific deadlines (e.g., "w ciągu 7 dni", "niezwłocznie") with their consequences for non-compliance
10  - Include any exceptions to the conditions ("chyba że", "z zastrzeżeniem", "nie dotyczy")
11 - If asked "Czy mogę...?", answer whether conditions are met.
12   - Put the most restrictive condition first.
13 {_ANSWER_FORMAT}
14 """

```

Prompt LIMITS_PROMPT definiuje zasady precyzyjnego przytaczania limitów świadczeń, sum ubezpieczenia i fransyz z podaniem odniesień do konkretnych klauzul OWU.

Listing A.21 Prompt limitów i sum ubezpieczenia (LIMITS_PROMPT)

```

1 LIMITS_PROMPT: str = f"""
2 You are a Polish insurance documentation assistant explaining monetary limits, sums insured, or deductibles.
3
4 Answering strategy:
5 - In Odpowiedź state the exact amount(s) in PLN (or %) in one sentence.
6 - In Szczegóły list ALL relevant financial parameters:
7   - Suma ubezpieczenia - the insured sum, per event or aggregate [N]
8   - Franszyza integralna - minimum damage threshold below which no payout is made [N]
9   - Franszyza redukcyjna - fixed deduction from every claim [N]
10  - Udział własny - percentage the insured pays themselves [N]
11  - Limity na podkategorie - sub-limits for specific risks (e.g., "koszty leczenia stomatologicznego - do 2 000 zł") [N]
12  - Limity na zdarzenie vs. roczne - distinguish per-event limits from annual aggregates when documented
13  - Include ONLY the financial parameters that appear in the context. Do NOT list categories with no evidence.
14  - Every amount MUST have a concrete clause reference copied from the context, e.g. (§ 21 ust. 3) [1]. Never use placeholders like § X
   ust. Y or [N].
15  - If limits differ by variant, clearly label which variant each limit belongs to.
16  - Prioritize the exact figure that directly answers the question before secondary limits.
17 {_ANSWER_FORMAT}
18 """

```

Prompt EXCLUSIONS_PROMPT definiuje schemat kategoryzacji i wyliczania wyłączeń odpowiedzialności, grupując je na ogólne, przedmiotowe, podmiotowe i warunkowe.

Listing A.22 Prompt wyłączeń ubezpieczenia (EXCLUSIONS_PROMPT)

```

1 EXCLUSIONS_PROMPT: str = f"""
2 You are a Polish insurance documentation assistant listing policy exclusions.
3
4 Answering strategy:
5 - In Odpowiedź give one direct sentence: state whether the specific thing asked about IS excluded, or give a count/summary.
6 - In Szczegóły bullet-list the exclusions, organized by category when possible:
7   - Wyłączenia ogólne - exclusions applying to all coverage types
8   - Wyłączenia przedmiotowe - excluded events, items, or situations
9   - Wyłączenia podmiotowe - excluded persons or entities
10  - Wyłączenia warunkowe - exclusions that can be lifted by additional premium or endorsement ("chyba że", "o ile umowa nie została
   rozszerzona")
11 - For each exclusion:
12   - Quote the key phrase from the OWU as closely as possible
13   - Include the full concrete clause reference copied from the context, e.g. (§ 18 ust. 1 pkt 4) [1]
14   - Note any exceptions to the exclusion ("chyba że nie miało to wpływu na powstanie szkody")
15 - If the question asks about a SPECIFIC exclusion ("Czy X jest wyłączone?"), focus on that one and cite the decisive clause.
16 - In specific yes/no exclusion questions, keep Odpowiedź to one decisive sentence.
17 {_ANSWER_FORMAT}
18 """

```

Stała CONTEXT_INSTRUCTION jest dołączana do każdego zapytania syntezy odpowiedzi jako przypomnienie reguł formatowania i korzystania wyłącznie z pobranego kontekstu.

Listing A.23 Instrukcja kontekstu dla syntezy (CONTEXT_INSTRUCTION)

```

1 CONTEXT_INSTRUCTION: str = (
2     "INSTRUCTION: Follow the required output format exactly. "
3     "Use only retrieved context; do not guess. "
4     "Do not put citations in Odpowiedź. "
5     "Cite every fact in Szczegóły with concrete citations like [1], include concrete clause references from the context when available,

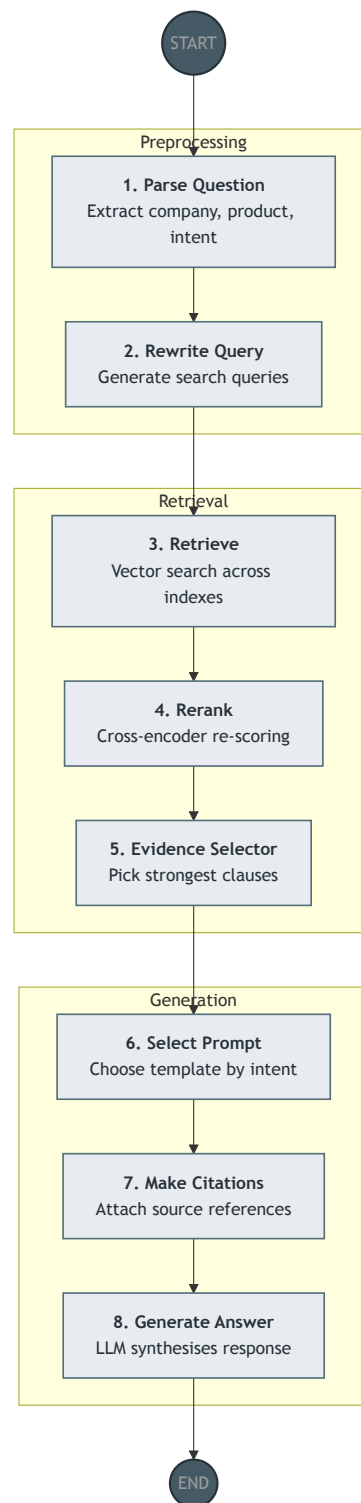
```

```
6         and never output placeholders such as [N] or § X ust. Y. "
7     "Write full grammatical sentences only (no single-word answers). "
8     "Start directly with the answer (no intro). "
9     "Keep Odpowiedź as exactly one full sentence. "
10    "Keep Odpowiedź concrete: verdict + decisive condition/amount/variant. "
11    "Leave one blank line before Szczegóły. "
12    "Avoid filler and generic summaries. "
13    "In Szczegóły, provide detailed bullet points with references and clause-level evidence. "
14    "Always answer in Polish. "
15    'If context is insufficient, reply only: "Dostępne dokumenty nie zawierają wystarczających informacji, aby odpowiedzieć na to
    pytanie."'
    )
```

B. Diagramy architektur agentowych

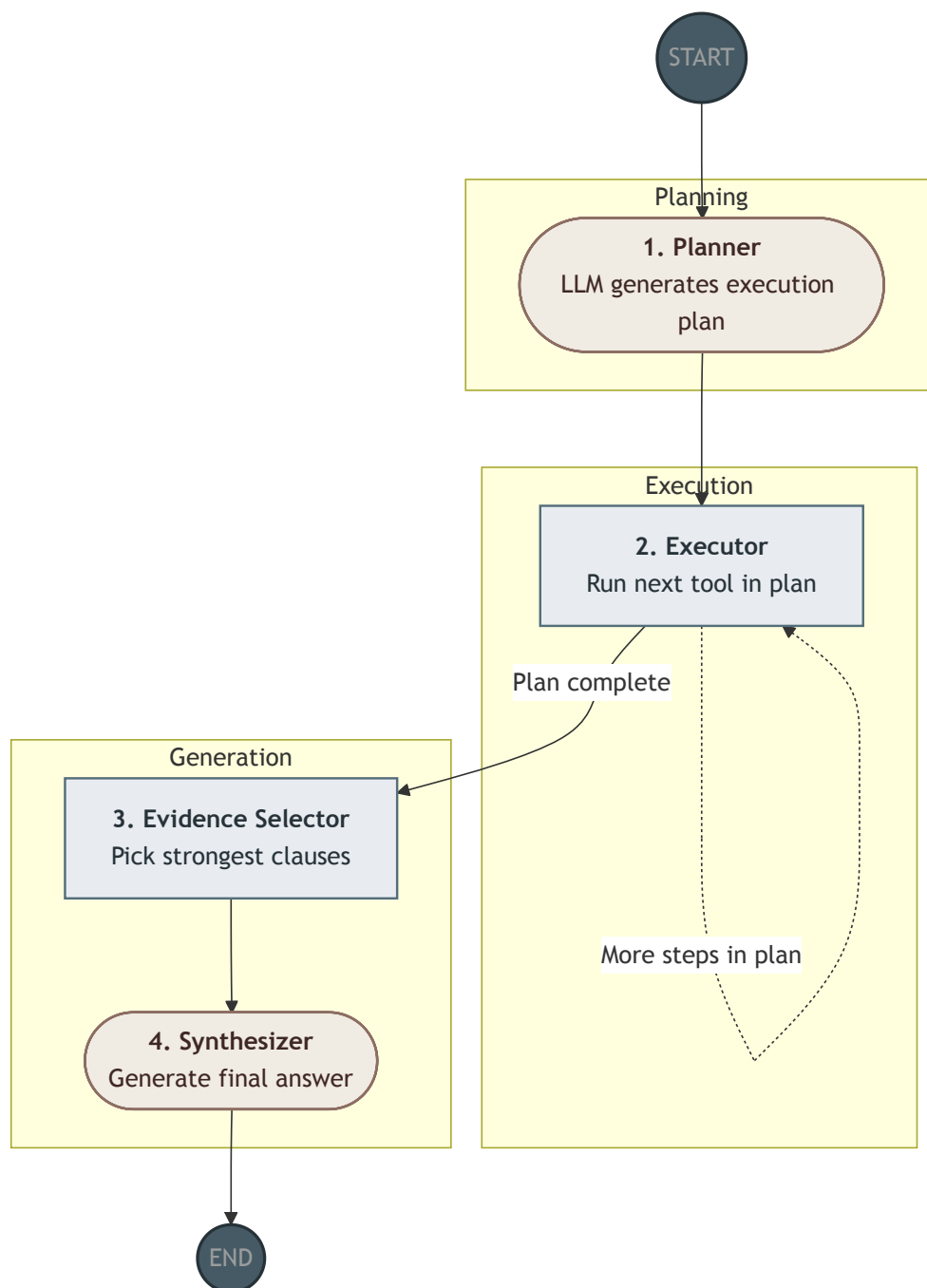
Dodatek zawiera diagramy sześciu architektur agentowych zaimplementowanych w module `sources/agents`. Diagramy zostały wygenerowane automatycznie z definicji Mermaid przez `utilities/drawer.py` i zrenderowane do plików PDF poleceniem `mmdc` z opcją `--pdfFit`.

Rysunek B.1 Architektura deterministyczna



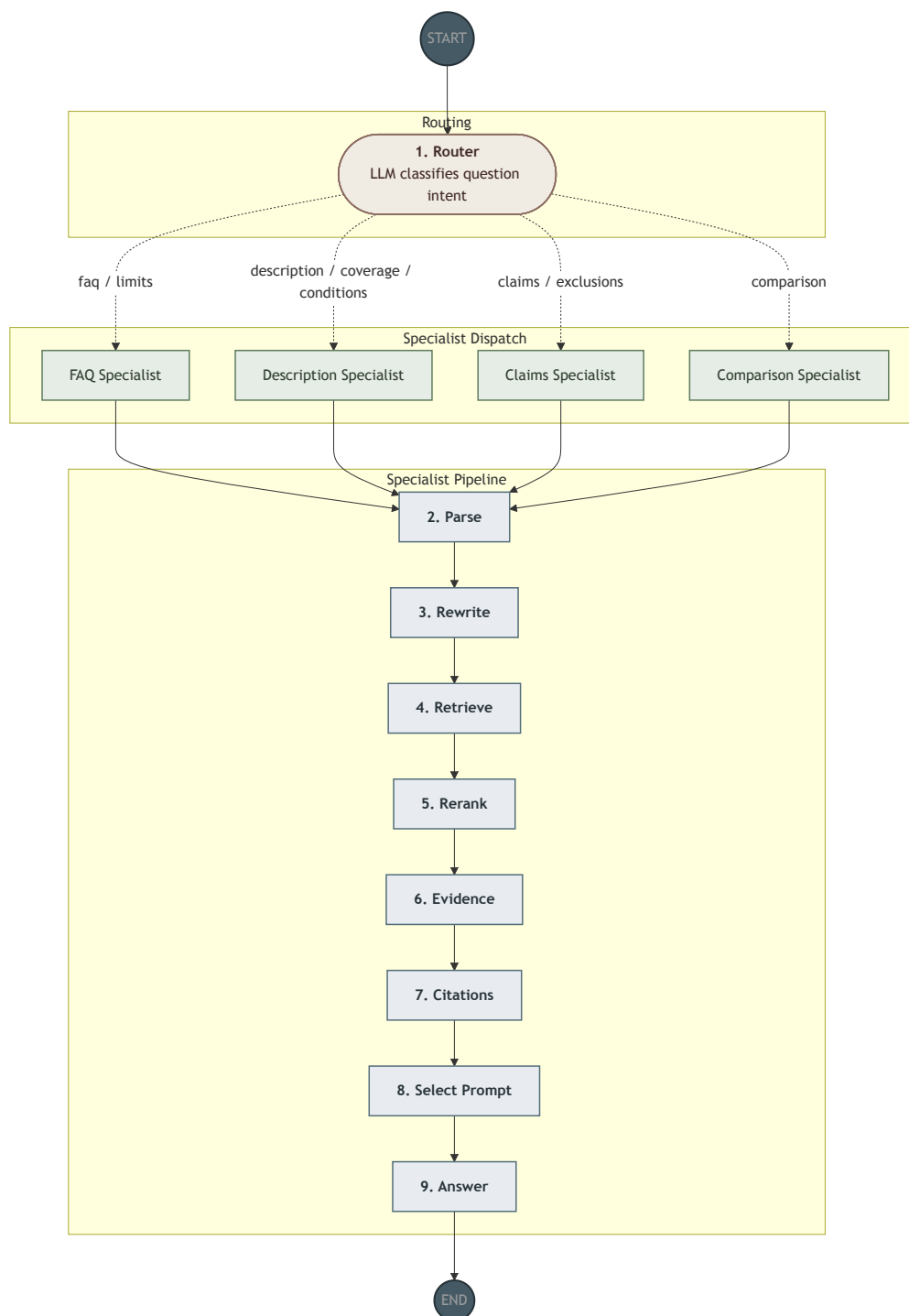
Źródło: opracowanie własne.

Rysunek B.2 Architektura planer–wykonawca



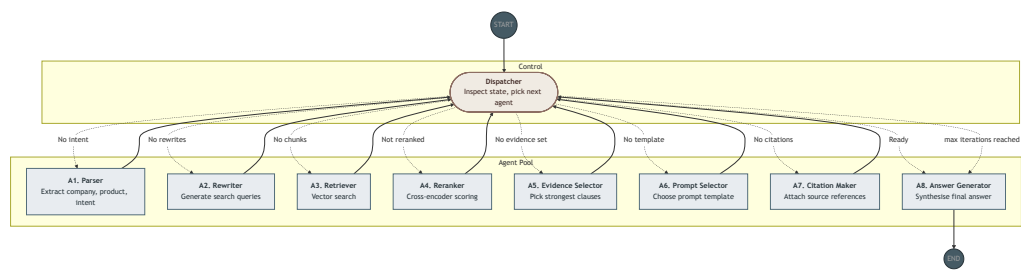
Źródło: opracowanie własne.

Rysunek B.3 Architektura router–specjalista



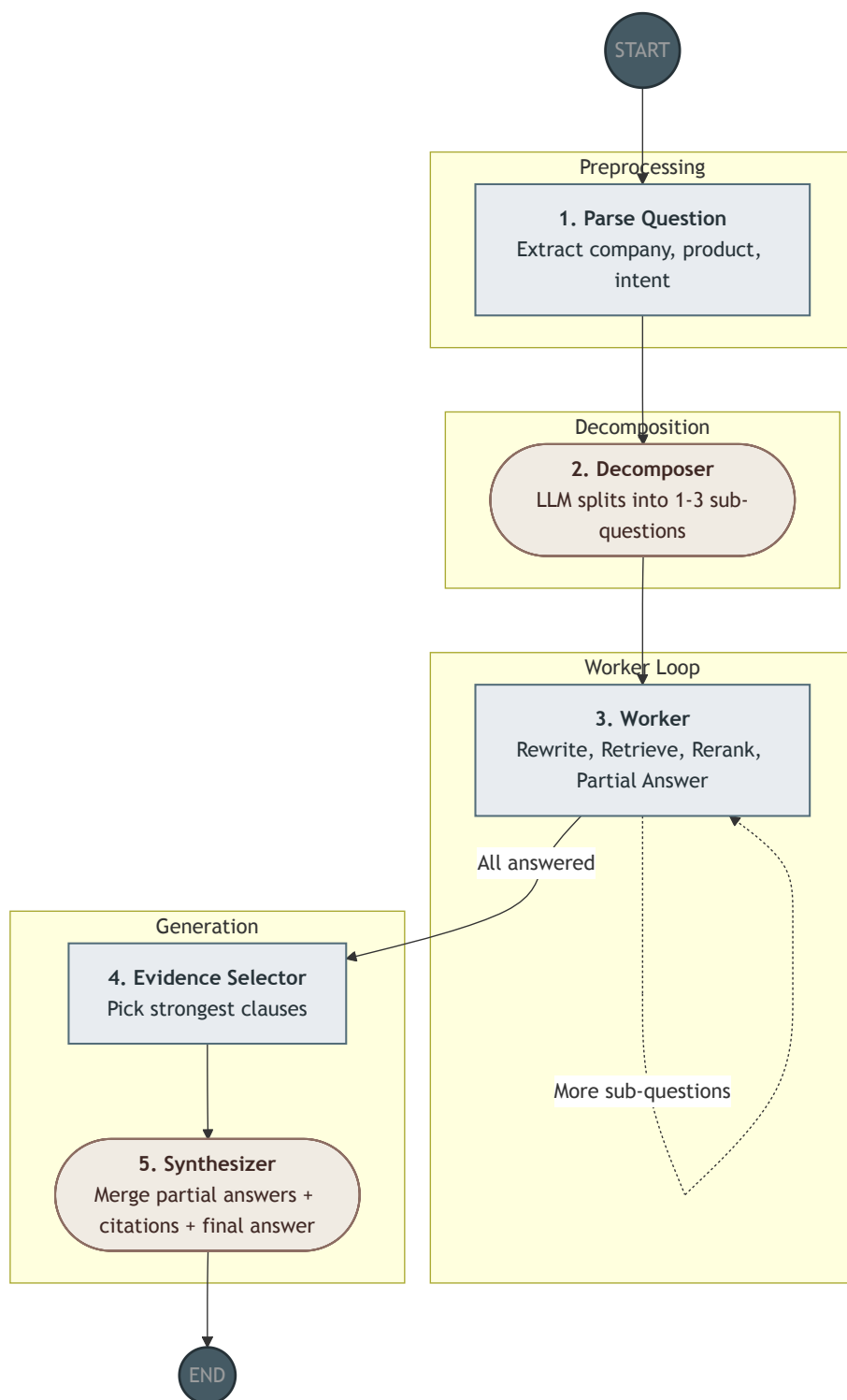
Źródło: opracowanie własne.

Rysunek B.4 Architektura tablicowa



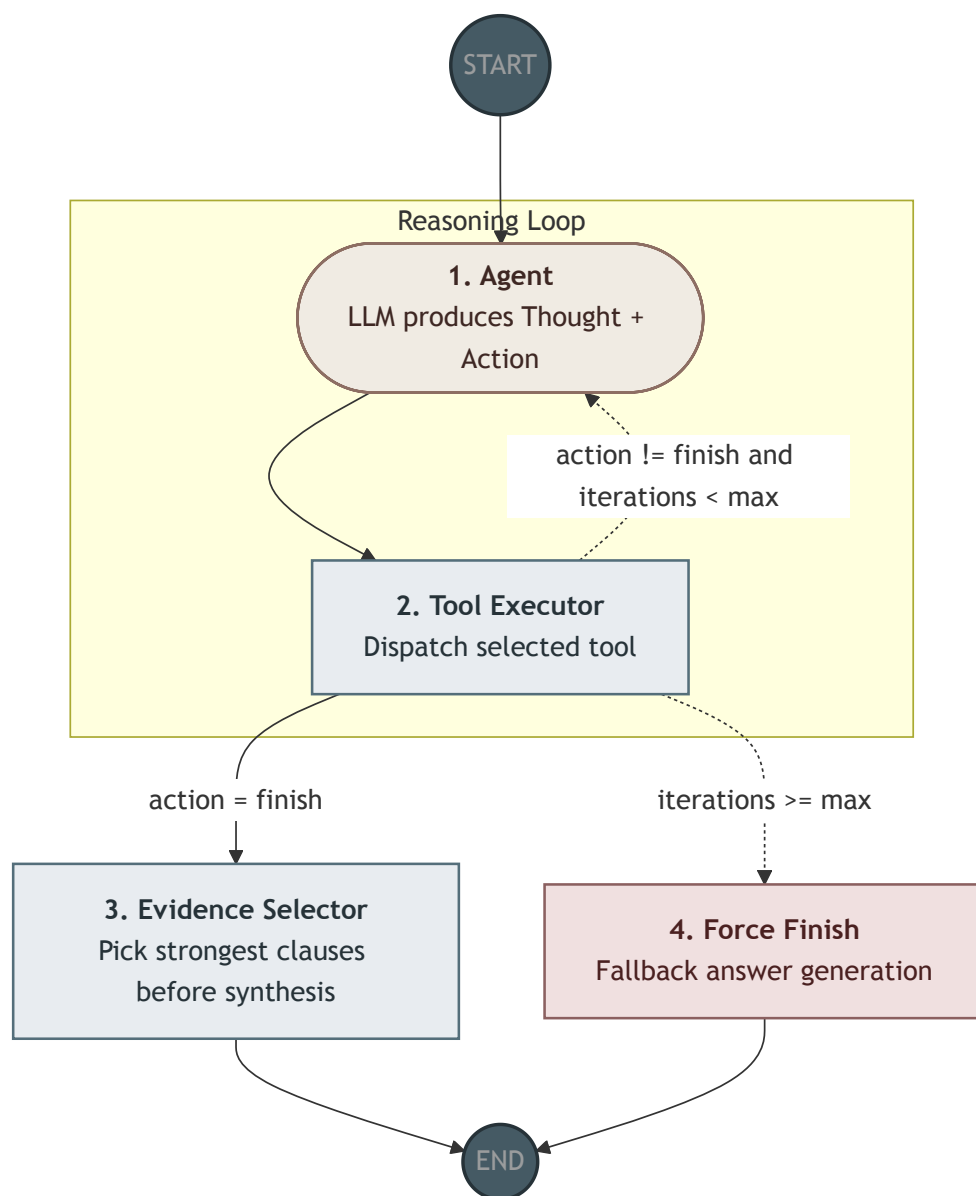
Źródło: opracowanie własne.

Rysunek B.5 Architektura hierarchiczna



Źródło: opracowanie własne.

Rysunek B.6 Architektura ReAct



Źródło: opracowanie własne.